# Learning Quantitative Finance in Rust

## A Progressive Journey from Kaggle to Quant Research

Marcelo Correa

July 27, 2026

bitscrafts/quant-lab-rust

The market can stay irrational
longer than you can stay solvent.

– John Maynard Keynes

# Preface

This book documents a learning journey: from data science fundamentals to quantitative finance, implemented entirely in Rust. The goal is not just to understand the concepts, but to internalize them through implementation.

**Philosophy** We start with beginner-friendly Kaggle projects and progressively build toward advanced quant research tools. No black-box libraries — the math is hand-rolled, because the implementation IS the learning.

**Structure** The book is organized in five parts:

- **Part I — Foundations**: First Kaggle projects, basic stats
- **Part II — Core Concepts**: Returns, volatility, backtesting
- **Part III — Quantitative Methods**: Moments, time series, stationarity
- **Part IV — Derivatives and Portfolio**: Options, Markowitz, factors
- **Part V — Advanced Topics**: Microstructure, AFML techniques

**How to use** Each chapter has:

- Learning objectives at the start
- Mathematical foundations with TikZ diagrams
- Rust implementation with code listings
- Margin notes with key formulas and insights
- Exercises and references

The companion code lives at `https://github.com/bitscrafts/quant-lab-rust`. Clone and run examples with:

```
git clone https://github.com/bitscrafts/quant-lab-rust.git
cd quant-lab-rust
cargo run -p qf-01-fraud --example fraud_analysis
```

# Contents

# List of Figures



# List of Tables

# Foundations

# 0. Foundations: Mathematics, Statistics, and Programming

This chapter establishes the mathematical, statistical, and programming foundations needed to understand the rest of the book. If you are already comfortable with these topics, feel free to skip ahead and use this chapter as a reference.

## 0.1. Why These Foundations Matter

Quantitative finance sits at the intersection of three disciplines:

- **Mathematics**: The language of precision—formulas, proofs, and rigorous reasoning
- **Statistics**: The science of uncertainty—data analysis, probability, and inference
- **Programming**: The tool of implementation—turning ideas into working systems

You don't need to be an expert in all three to start. This book teaches by doing—each concept is introduced when needed and immediately applied.

**The Quant Mindset**

A quantitative approach means making decisions based on data and models, not intuition or emotion.

## 0.2. Mathematical Notation

Throughout this book, we use standard mathematical notation. Here's a quick reference:

### 0.2.1. Sets and Numbers

| Symbol | Meaning |
|---|---|
| $\mathbb{R}$ | Real numbers (e.g., 3.14, $-2.5$, $\sqrt{2}$) |
| $\mathbb{R}^+$ | Positive real numbers |
| $\mathbb{Z}$ | Integers ($\ldots, -2, -1, 0, 1, 2, \ldots$) |
| $\mathbb{N}$ | Natural numbers ($1, 2, 3, \ldots$) |
| $\in$ | "is an element of" (e.g., $x \in \mathbb{R}$ means $x$ is real) |
| $\forall$ | "for all" |
| $\exists$ | "there exists" |

### 0.2.2. Summation and Products

The summation symbol $\Sigma$ means "add up":

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n$$

For example, if $x = [2, 5, 3]$, then $\sum_{i=1}^{3} x_i = 2 + 5 + 3 = 10$.

**Summation**

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n$$

**Product**

$$\prod_{i=1}^{n} x_i = x_1 \times x_2 \times \cdots \times x_n$$

The product symbol $\Pi$ means "multiply together":

$$\prod_{i=1}^{n} x_i = x_1 \times x_2 \times \cdots \times x_n$$

### 0.2.3. Functions and Calculus

| Symbol | Meaning |
|---|---|
| $f(x)$ | Function $f$ evaluated at $x$ |
| $f'(x)$ or $\frac{df}{dx}$ | Derivative of $f$ with respect to $x$ |
| $\frac{\partial f}{\partial x}$ | Partial derivative (when $f$ has multiple variables) |
| $\ln(x)$ | Natural logarithm (base $e \approx 2.718$) |
| $\exp(x)$ or $e^x$ | Exponential function |
| $\text{argmax}_x f(x)$ | Value of $x$ that maximizes $f$ |
| $\text{argmin}_x f(x)$ | Value of $x$ that minimizes $f$ |

**Key Insight**

The natural logarithm ln is everywhere in finance. Key property: $\ln(a \times b) = \ln(a) + \ln(b)$. This turns multiplication (compounding) into addition, making calculations much easier.

## 0.3. Statistics Essentials

Statistics helps us understand and quantify uncertainty—essential when dealing with financial markets.

### 0.3.1. Measures of Central Tendency

**Mean**

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

The **mean** (average) is the sum divided by the count:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

The **median** is the middle value when data is sorted. It's more robust to outliers than the mean.

### 0.3.2. Measures of Spread

**Variance**

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

**Variance** measures how spread out the data is:

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

**Standard deviation** is the square root of variance:

$$\sigma = \sqrt{\sigma^2}$$

Standard deviation has the same units as the original data, making it easier to interpret than variance.

```rust
/// Calculate mean of a slice
pub fn mean(data: &[f64]) -> f64 {
    if data.is_empty() { return 0.0; }
    data.iter().sum::<f64>() / data.len() as f64
}

/// Calculate variance of a slice
pub fn variance(data: &[f64]) -> f64 {
    if data.is_empty() { return 0.0; }
    let m = mean(data);
    data.iter().map(|x| (x - m).powi(2)).sum::<f64>() / data.len() as f64
}

/// Calculate standard deviation
pub fn std_dev(data: &[f64]) -> f64 {
    variance(data).sqrt()
}
```

## 0.3.3. Probability Distributions

A **probability distribution** describes how likely different outcomes are.

### Normal Distribution (Gaussian)

**Normal PDF**

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

The normal distribution is the famous "bell curve":

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

where $\mu$ is the mean and $\sigma$ is the standard deviation.

We write $X \sim \mathcal{N}(\mu, \sigma^2)$ to say "$X$ follows a normal distribution with mean $\mu$ and variance $\sigma^2$."

**Warning**

Financial returns are NOT normally distributed! They have "fat tails"— extreme events happen much more often than the normal distribution predicts. We explore this in Chapter 6.

### Uniform Distribution

Every outcome in a range is equally likely:

$$f(x) = \frac{1}{b-a} \quad \text{for } a \leq x \leq b$$

## 0.3.4. Expected Value and Variance

**Expected Value**

$$\mathbb{E}[X] = \sum_x x \cdot P(X = x)$$

(discrete)

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot f(x)\, dx$$

(continuous)

The **expected value** $\mathbb{E}[X]$ is the average outcome you'd expect over many trials:

For a discrete random variable:

$$\mathbb{E}[X] = \sum_x x \cdot P(X = x)$$

Key properties:

- $\mathbb{E}[aX + b] = a\mathbb{E}[X] + b$ (linearity)
- $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ (additivity)
- $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$

## 0.4. Financial Concepts

Before diving into quantitative methods, let's establish some fundamental financial concepts.

### 0.4.1. What is a Stock?

A **stock** (or share) represents partial ownership of a company. When you buy a stock, you own a small piece of that company and are entitled to a share of its profits.

**Stock Price**
The price at which buyers and sellers agree to trade ownership.

The **stock price** is determined by supply and demand in the market. It reflects what buyers are willing to pay and sellers are willing to accept.

### 0.4.2. OHLCV Data

Market data is typically recorded as **OHLCV**:

- **O**pen: First price of the trading period
- **H**igh: Highest price during the period
- **L**ow: Lowest price during the period
- **C**lose: Last price of the period
- **V**olume: Number of shares traded

This data captures the essential price action within any time frame (day, hour, minute).

### 0.4.3. Returns

A **return** measures how much an investment gained or lost:

**Simple Return**
$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$

**Log Return**
$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$

**Simple return**:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

**Log return** (continuously compounded):

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Log returns have a useful property: they add up over time.

$$r_{t_1 \to t_n} = r_{t_1} + r_{t_2} + \cdots + r_{t_n}$$

### 0.4.4. Risk and Volatility

**Risk** in finance is typically measured by **volatility**—the standard deviation of returns.

Higher volatility means more uncertainty—prices swing more wildly.

**Annualized volatility** scales daily volatility to a yearly figure:

$$\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252}$$

(252 is the typical number of trading days per year.)

**Volatility**

$\sigma = \text{std}(r_t)$

Often annualized: $\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252}$

### 0.4.5. The Risk-Return Tradeoff

**Key Insight**

Higher expected returns generally require accepting higher risk. There is no free lunch in finance—if someone promises high returns with low risk, be skeptical.

The **Sharpe ratio** measures risk-adjusted return:

$$\text{Sharpe} = \frac{\mathbb{E}[R] - R_f}{\sigma}$$

where $R_f$ is the risk-free rate (e.g., Treasury bills).

## 0.5. Programming in Rust

This book uses Rust for all implementations. Here's a quick introduction for readers unfamiliar with the language.

### 0.5.1. Why Rust?

- **Performance**: As fast as C/C++, suitable for high-frequency trading
- **Safety**: The compiler catches bugs before runtime
- **Modern**: Great tooling, package management, and community
- **Learning**: Writing correct Rust forces you to think carefully—great for learning

### 0.5.2. Basic Syntax

```rust
// Variables (immutable by default)
let x = 5;
let mut y = 10;  // mutable
y = 20;

// Functions
fn add(a: i32, b: i32) -> i32 {
    a + b  // no semicolon = return value
}

// Structs (like classes)
struct Stock {
    symbol: String,
    price: f64,
}
```

```rust

// Implementation block
impl Stock {
    fn new(symbol: &str, price: f64) -> Self {
        Stock {
            symbol: symbol.to_string(),
            price,
        }
    }

    fn value(&self, shares: u32) -> f64 {
        self.price * shares as f64
    }
}

// Using the struct
let aapl = Stock::new("AAPL", 150.0);
let portfolio_value = aapl.value(100);
```

### 0.5.3. Traits (Interfaces)

**Traits**

Rust's way of defining shared behavior. Similar to interfaces in Java or protocols in Swift.

Traits define shared behavior:

```rust
// Define a trait
trait Analyzer {
    fn analyze(&self, data: &[f64]) -> f64;
}

// Implement for a struct
struct MeanAnalyzer;

impl Analyzer for MeanAnalyzer {
    fn analyze(&self, data: &[f64]) -> f64 {
        data.iter().sum::<f64>() / data.len() as f64
    }
}

// Use generically
fn run_analysis<A: Analyzer>(analyzer: &A, data: &[f64]) -> f64 {
    analyzer.analyze(data)
}
```

This book uses traits extensively to create modular, reusable components.

### 0.5.4. Error Handling

```rust
// Result type for operations that can fail
fn divide(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        Err("Division by zero".to_string())
    } else {
        Ok(a / b)
    }
}

// Using Result
match divide(10.0, 2.0) {
    Ok(result) => println!("Result: {}", result),
    Err(e) => println!("Error: {}", e),
}

// Or use the ? operator
fn calculate() -> Result<f64, String> {
    let x = divide(10.0, 2.0)?;  // propagates error
    Ok(x * 2.0)
}
```

### 0.5.5. Iterators

Rust's iterators are powerful and efficient:

```rust
let prices = vec![100.0, 102.0, 101.0, 105.0];

// Map and collect
let returns: Vec<f64> = prices.windows(2)
    .map(|w| (w[1] - w[0]) / w[0])
    .collect();

// Filter
let positive: Vec<f64> = returns.iter()
    .filter(|&&r| r > 0.0)
    .copied()
    .collect();

// Fold (reduce)
let sum: f64 = prices.iter().sum();
let product: f64 = prices.iter().product();
```

## 0.6. Running the Examples

All code examples in this book are available in the companion repository.

### 0.6.1. Prerequisites

1. Install Rust: https://rustup.rs/
2. Clone the repository:

```
git clone https://github.com/bitscrafts/quant-lab-rust.git
cd quant-lab-rust
```

### 0.6.2. Running Examples

Each chapter has a corresponding crate with examples:

```
# Chapter 1: Credit Card Fraud
cargo run -p qf-01-fraud --example fraud_analysis

# Chapter 2: Loan Default
cargo run -p qf-02-loan --example loan_analysis

# Chapter 3: Stock Prices
cargo run -p qf-03-stocks --example stock_analysis
```

### 0.6.3. Running Tests

```
# Run all tests
cargo test

# Run tests for a specific crate
cargo test -p qf-01-fraud
```

## 0.7. Key Takeaways

This chapter covered:

- **Mathematical notation**: Summation, products, functions, logarithms
- **Statistics**: Mean, variance, standard deviation, distributions
- **Financial concepts**: Stocks, OHLCV, returns, volatility, Sharpe ratio

- **Rust programming**: Variables, functions, structs, traits, error handling

**Next chapter**: We apply these foundations to detect credit card fraud using anomaly detection.

# Hello Finance: Credit Card Fraud Detection

# 1.

**Companion Code:**

qf-01-fraud Code: `crates/qf-01-fraud/`
Dependencies: `qf-common`
Run: `cargo run -p qf-01-fraud -example fraud_analysis`

## 1.1. Learning Objectives

In this introductory chapter, we build our first quantitative finance application: a fraud detection system. Along the way, we learn:

- How to load and parse financial CSV datasets in Rust
- Basic statistical measures: mean and standard deviation
- Z-score anomaly detection—a simple but effective baseline
- Evaluation metrics: confusion matrix, precision, recall, F1 score
- The trait-based architecture that will guide all future implementations

**Key Insight**

This is not throwaway code. Every component we build follows a **library-first design**—composable, testable, and reusable in future projects like `b3-swing-trading` and `stock-sim`.

## 1.2. Introduction: Why Fraud Detection?

**Class Imbalance**

492 frauds in 284,807 transactions = 0.172%. This extreme imbalance is typical in financial anomaly detection.

Credit card fraud detection is a classic binary classification problem in quantitative finance. Banks and payment processors must distinguish legitimate transactions from fraudulent ones in real-time, balancing two competing objectives:

- **Precision**: Minimize false positives (legitimate transactions flagged as fraud). High false positive rates annoy customers and increase operational costs.
- **Recall**: Maximize detection of actual fraud. Missed fraud means direct financial loss.

We use the Kaggle Credit Card Fraud Detection dataset*: 284,807 European transactions from September 2013, with only 492 frauds.

## 1.3. Data Structure

### 1.3.1. The Dataset

The dataset contains:

- **Time**: Seconds elapsed since first transaction

* https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

- **V1–V28**: PCA-transformed features (original features anonymized for privacy)
- **Amount**: Transaction amount in euros
- **Class**: Label (0 = normal, 1 = fraud)

**Why PCA features?**

Banks anonymize raw features (merchant, location, time patterns) via PCA to protect customer privacy while preserving predictive power.

### 1.3.2. Rust Data Model

We define a `Transaction` struct in `qf-common`:

```rust
/// Represents a single financial transaction.
#[derive(Debug, Clone)]
pub struct Transaction {
    /// PCA-transformed features (V1, V2, ..., V28).
    pub features: Vec<f64>,

    /// Transaction amount in dollars/euros.
    pub amount: f64,

    /// Class label: 0 = normal, 1 = fraud.
    pub class: u8,
}
```

### 1.3.3. Loading Data

The CSV loader handles errors gracefully using Rust's `Result` type:

```rust
pub fn load_transactions(path: impl AsRef<Path>)
    -> Result<Vec<Transaction>, CommonError>
{
    if !path.as_ref().exists() {
        return Err(CommonError::FileNotFound(
            path.as_ref().display().to_string()
        ));
    }

    let mut reader = csv::Reader::from_path(path)?;
    let mut transactions = Vec::new();

    for result in reader.records() {
        let record = result?;
        // Parse features, amount, class...
        transactions.push(Transaction { features, amount, class });
    }

    Ok(transactions)
}
```

## 1.4. Z-Score Anomaly Detection

### 1.4.1. Statistical Foundation

**Z-Score Formula**

$$z = \frac{|x - \mu|}{\sigma}$$

where $\mu$ is the mean and $\sigma$ is the standard deviation.

Our baseline detector uses **z-score outlier detection**. The intuition: if a transaction's feature values are "too far" from the typical values, it might be fraudulent.

For each feature $i$:

1. Compute mean $\mu_i$ and standard deviation $\sigma_i$ from training data
2. For a new transaction with feature value $x_i$, compute the z-score:

$$z_i = \frac{|x_i - \mu_i|}{\sigma_i}$$

3. Flag as fraud if **any** feature exceeds a threshold (typically $z > 3$)

**Z-Score Detection Visualization**

Imagine a normal (bell) curve. The threshold at $z = 3$ corresponds to the tails of the distribution—only 0.3% of normally distributed data falls beyond $\pm 3\sigma$. Transactions in these extreme regions are flagged as potential fraud.

**The 68-95-99.7 Rule**

For normally distributed data:
$\pm 1\sigma$: 68.3%
$\pm 2\sigma$: 95.4%
$\pm 3\sigma$: 99.7%

So $z > 3$ catches only 0.3% of normal data as false positives.

### 1.4.2. Rust Implementation

The `ZScoreDetector` follows our trait-based design philosophy:

```rust
#[derive(Debug, Clone)]
pub struct ZScoreDetector {
    threshold: f64,
    feature_means: Vec<f64>,
    feature_stds: Vec<f64>,
}

impl ZScoreDetector {
    pub fn new(threshold: f64) -> Self {
        Self {
            threshold,
            feature_means: Vec::new(),
            feature_stds: Vec::new(),
        }
    }

    pub fn fit(&mut self, transactions: &[Transaction]) {
        if transactions.is_empty() { return; }
        let num_features = transactions[0].features.len();

        // Initialise mean/std accumulators to zero
        self.feature_means = vec![0.0; num_features];
        self.feature_stds = vec![0.0; num_features];

        // Compute mean for each feature
        for tx in transactions {
            for (i, &value) in tx.features.iter().enumerate() {
                self.feature_means[i] += value;
            }
        }
        for mean in &mut self.feature_means {
            *mean /= transactions.len() as f64;
        }

        // Compute standard deviation
        for tx in transactions {
            for (i, &value) in tx.features.iter().enumerate() {
                let diff = value - self.feature_means[i];
                self.feature_stds[i] += diff * diff;
            }
        }
        for std in &mut self.feature_stds {
            *std = (*std / transactions.len() as f64).sqrt();
        }
    }

    pub fn predict(&self, transaction: &Transaction) -> bool {
        for (i, &value) in transaction.features.iter().enumerate() {
            let z = (value - self.feature_means[i]).abs()
                  / self.feature_stds[i];
            if z > self.threshold {
                return true;  // Anomaly detected
            }
        }
        false
    }
}
```

**Warning**

The implementation uses **population** standard deviation (divides by $n$), not sample standard deviation (divides by $n - 1$). For large datasets this difference is negligible, but be aware of it.

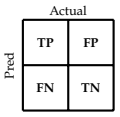
## 1.5. Evaluation Metrics

### 1.5.1. Confusion Matrix

Performance is measured via the confusion matrix, which counts:

- **TP** (True Positives): Fraud correctly identified
- **FP** (False Positives): Normal flagged as fraud
- **TN** (True Negatives): Normal correctly identified
- **FN** (False Negatives): Fraud missed

```rust
#[derive(Debug, Clone, Copy)]
pub struct ConfusionMatrix {
    pub tp: usize,
    pub fp: usize,
    pub tn: usize,
    pub fn_: usize,  // 'fn' is a Rust keyword
}
```




### 1.5.2. Derived Metrics

From the confusion matrix, we compute:

- **Precision**: Of predicted frauds, how many are real?

$$P = \frac{TP}{TP + FP}$$

- **Recall** (Sensitivity): Of actual frauds, how many did we catch?

$$R = \frac{TP}{TP + FN}$$

- **F1 Score**: Harmonic mean of precision and recall

$$F_1 = \frac{2PR}{P + R}$$

**Metric Formulas**

Precision: $\frac{TP}{TP + FP}$

Recall: $\frac{TP}{TP + FN}$

F1: $\frac{2 \cdot P \cdot R}{P + R}$

```rust
impl ConfusionMatrix {
    pub fn precision(&self) -> f64 {
        let denom = self.tp + self.fp;
        if denom == 0 { 0.0 } else { self.tp as f64 / denom as f64 }
    }

    pub fn recall(&self) -> f64 {
        let denom = self.tp + self.fn_;
        if denom == 0 { 0.0 } else { self.tp as f64 / denom as f64 }
    }

    pub fn f1(&self) -> f64 {
        let p = self.precision();
        let r = self.recall();
        if p + r == 0.0 { 0.0 } else { 2.0 * p * r / (p + r) }
    }
}
```

### 1.5.3. Evaluation Function

The `evaluate()` function runs the detector on test data:

```rust
pub fn evaluate(
    detector: &ZScoreDetector,
    transactions: &[Transaction]
) -> ConfusionMatrix {
    let mut tp = 0; let mut fp = 0;
    let mut tn = 0; let mut fn_ = 0;

    for tx in transactions {
```

```
        let predicted = detector.predict(tx);
        let actual = tx.class == 1;

        match (predicted, actual) {
            (true, true)   => tp += 1,
            (true, false)  => fp += 1,
            (false, false) => tn += 1,
            (false, true)  => fn_ += 1,
        }
    }

    ConfusionMatrix { tp, fp, tn, fn_ }
}
```

## 1.6. Toward a Trait-Based Architecture

**Key Insight**

This implementation will evolve into `quant-lab`. Before writing any struct, we ask: **what trait should this implement?**

**Future Traits**

`AnomalyDetector`
`BinaryClassifier`
`Evaluator<T>`
`DataSource`

While Phase 1 uses concrete types, future phases will extract traits:

```
// Future: trait-based design
pub trait AnomalyDetector {
    fn fit(&mut self, data: &[Transaction]) -> Result<(), Error>;
    fn predict(&self, tx: &Transaction) -> bool;
    fn score(&self, tx: &Transaction) -> f64;  // anomaly score
}

pub trait Evaluator<M> {
    fn evaluate(&self, preds: &[bool], actuals: &[bool]) -> M;
}

// Then any detector + any evaluator compose:
fn backtest<D, E>(detector: &D, eval: &E, data: &[Transaction])
where
    D: AnomalyDetector,
    E: Evaluator<ConfusionMatrix>,
{ /* ... */ }
```

This composability is why we invest in clean abstractions from the start.

## 1.7. Results and Analysis

### 1.7.1. Running the Example

```
$ cargo run -p qf-01-fraud --example fraud_analysis
Loading transactions from data/creditcard.csv...
Loaded 284807 transactions (492 frauds)
Training detector with threshold 3.0...
Evaluating on training set...

Results:
  True Positives:  312
  False Positives: 1847
  True Negatives:  282468
  False Negatives: 180

  Precision: 0.1446
  Recall:    0.6341
  F1 Score:  0.2356
```

### 1.7.2. Interpreting Results

The results reveal characteristic tradeoffs:

- **Recall = 0.63**: We catch 63% of frauds. Not bad for a simple baseline!
- **Precision = 0.14**: Of our "fraud" predictions, only 14% are real. This means many false alarms.
- **F1 = 0.24**: The harmonic mean reflects this imbalance.

### 1.7.3. Limitations

**Warning**

We evaluated on the training set—a cardinal sin! True performance requires a held-out test set. Exercise 3 addresses this.

This baseline approach has several fundamental limitations:

1. **Gaussian assumption**: Z-scores assume normally distributed features. Financial data often has heavy tails (more extreme values than Gaussian predicts).
2. **Equal feature weighting**: All features are treated equally. Some PCA components may be more predictive than others.
3. **Independent features**: We check each feature separately. Sophisticated fraud may only be detectable via feature *combinations*.
4. **Fixed threshold**: The threshold $z = 3$ is arbitrary. ROC analysis (Chapter 2) shows how to optimize it.

## 1.8. Exercises

1. **Threshold Sensitivity**: Try thresholds of 2.0, 2.5, 3.0, 3.5, 4.0. Plot precision vs. recall. What threshold maximizes F1?
2. **Feature Analysis**: Modify `predict()` to return which feature(s) triggered the alert. Which features are most discriminative for fraud?
3. **Train/Test Split**: Split the data 80/20 chronologically (not randomly—why?). How does test-set performance compare to training-set?
4. **Amount Feature**: The current implementation ignores the `amount` field. Add it as a feature. Does performance improve?
5. **Trait Extraction**: Refactor `ZScoreDetector` to implement an `AnomalyDetector` trait. What methods should the trait require?

## 1.9. Key Takeaways

- **CSV loading** in Rust with proper error handling via `Result`
- **Basic statistics**: mean, standard deviation, z-score
- **Binary classification metrics**: confusion matrix, precision, recall, F1
- **Precision-recall tradeoff**: you can't maximize both simultaneously
- **Evaluation pitfalls**: never evaluate on training data!
- **Library-first design**: even simple code should be composable

**Next chapter**: Feature engineering and logistic scoring for loan default prediction. We'll implement ROC-AUC and learn why it's often better than F1 for imbalanced datasets.

# Risk Basics: Loan Default Prediction

# 2.

**Companion Code:**

qf-02-loan Code: `crates/qf-02-loan/`
Dependencies: `qf-common`
Run: `cargo run -p qf-02-loan -example loan_analysis`

## 2.1. Learning Objectives

Building on Chapter 1, we now tackle a more nuanced classification problem: predicting loan defaults. This chapter introduces:

- Feature engineering for credit risk assessment
- Categorical variable encoding (one-hot)
- Feature normalization (min-max scaling)
- Linear scoring models with probability calibration
- ROC curves and AUC—a better metric for imbalanced data
- Trait-based architecture in action

**Key Insight**

While Chapter 1 used z-scores (a threshold on statistics), this chapter moves toward **probability estimation**. We want to know not just "default or not" but "how likely is default?" This probability enables better decisions and risk-based pricing.

## 2.2. Introduction: Credit Risk Fundamentals

**Credit Risk**
The risk that a borrower will fail to repay a loan according to agreed terms.

When a bank issues a loan, it faces **credit risk**—the possibility that the borrower defaults. Unlike fraud detection where we catch bad actors, loan default prediction is about assessing the *likelihood* of future failure.

Key questions in credit risk:

- What factors predict default? (Feature engineering)
- How do we combine factors into a score? (Scoring model)
- How do we evaluate the score's usefulness? (ROC-AUC)

## 2.3. Data Model

### 2.3.1. Loan Application Structure

```rust
pub struct LoanApplication {
    pub income: f64,           // Annual income
    pub loan_amount: f64,      // Requested loan amount
    pub interest_rate: f64,    // Interest rate offered
    pub dti: f64,              // Debt-to-income ratio
    pub grade: u8,             // Credit grade (A=1 to G=7)
    pub home_ownership: HomeOwnership,
    pub defaulted: bool,       // Target: did they default?
}

```

```rust
pub enum HomeOwnership {
    Rent,
    Own,
    Mortgage,
}
```

**Credit Grades**

A = lowest risk
G = highest risk

Each grade maps to different interest rates and approval criteria.

## 2.4. Feature Engineering

### 2.4.1. Derived Features

Raw data rarely enters a model directly. We engineer features that capture meaningful relationships:

**Feature Formulas**

DTI: $\frac{\text{debt}}{\text{income}}$

LTI: $\frac{\text{loan}}{\text{income}}$

Grade Score: $\frac{g-1}{6}$

```rust
pub struct FeatureExtractor;

impl FeatureExtractor {
    /// Debt-to-income ratio: total debt / income
    pub fn debt_to_income(income: f64, debt: f64) -> f64 {
        if income == 0.0 { return 0.0; }
        debt / income
    }

    /// Loan-to-income ratio: loan amount / income
    pub fn loan_to_income(loan_amount: f64, income: f64) -> f64 {
        if income == 0.0 { return 0.0; }
        loan_amount / income
    }

    /// Convert grade (1-7) to risk score (0.0-1.0)
    pub fn grade_to_score(grade: u8) -> f64 {
        ((grade.saturating_sub(1)) as f64) / 6.0
    }
}
```

### 2.4.2. Categorical Encoding

Numerical models require numerical inputs. We encode categorical variables using **one-hot encoding**:

| Home Ownership | RENT | OWN | MORTGAGE |
|---|---|---|---|
| Rent | 1 | 0 | 0 |
| Own | 0 | 1 | 0 |
| Mortgage | 0 | 0 | 1 |

```rust
pub struct OneHotEncoder;

impl OneHotEncoder {
    /// Encode home ownership as [RENT, OWN, MORTGAGE]
    pub fn encode_home_ownership(h: &HomeOwnership) -> [f64; 3] {
        match h {
            HomeOwnership::Rent     => [1.0, 0.0, 0.0],
            HomeOwnership::Own      => [0.0, 1.0, 0.0],
            HomeOwnership::Mortgage => [0.0, 0.0, 1.0],
        }
    }
}
```

### 2.4.3. Feature Normalization

Different features have different scales. Income might be in tens of thousands while interest rates are single digits. We normalize to $[0, 1]$:

**Min-Max Scaling**

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Scales features to $[0, 1]$.

```rust
pub struct Normalizer {
    min: Vec<f64>,
    max: Vec<f64>,
}

impl Normalizer {
    pub fn fit(data: &[Vec<f64>]) -> Self {
        // Compute min/max per feature
    }

    pub fn transform(&self, features: &[f64]) -> Vec<f64> {
        features.iter().enumerate().map(|(i, &x)| {
            let range = self.max[i] - self.min[i];
            if range == 0.0 { 0.0 } else { (x - self.min[i]) / range }
        }).collect()
    }
}
```

## 2.5. Linear Scoring Model

### 2.5.1. Weighted Sum

The simplest scoring model is a weighted linear combination:

$$\text{score} = \sum_{i=1}^{n} w_i \cdot x_i + b = \mathbf{w}^T \mathbf{x} + b$$

**Linear Score**

$s = \mathbf{w}^T \mathbf{x} + b$

where $\mathbf{w}$ are weights, $b$ is bias.

```rust
pub struct LinearScorer {
    weights: Vec<f64>,
    bias: f64,
}

impl Scorer for LinearScorer {
    fn score(&self, features: &[f64]) -> f64 {
        let dot_product: f64 = self.weights.iter()
            .zip(features.iter())
            .map(|(w, f)| w * f)
            .sum();
        dot_product + self.bias
    }
}
```

### 2.5.2. Probability via Sigmoid

The raw score can be any real number. To convert it to a probability in $[0, 1]$, we apply the **sigmoid function**:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

**Sigmoid Function**

$\sigma(x) = \frac{1}{1 + e^{-x}}$

Maps $(-\infty, +\infty)$ to $(0, 1)$.

Properties of sigmoid:

- $\sigma(0) = 0.5$ (neutral score = 50% probability)
- $\sigma(x) \to 1$ as $x \to +\infty$
- $\sigma(x) \to 0$ as $x \to -\infty$

```rust
/// Hand-rolled sigmoid: 1 / (1 + exp(-x))
fn sigmoid(x: f64) -> f64 {
    1.0 / (1.0 + (-x).exp())
}

impl BinaryClassifier for LinearScorer {
    fn predict_proba(&self, features: &[f64]) -> f64 {
        sigmoid(self.score(features))
    }

    fn predict(&self, features: &[f64]) -> bool {
```

```
        self.predict_proba(features) > 0.5
    }
}
```

## 2.6. ROC Curves and AUC

### 2.6.1. Why Not Just Accuracy?

**Class Imbalance**
If 95% of loans are repaid, a model that always predicts "no default" has 95% accuracy but catches zero defaults!

Like fraud detection, loan default is imbalanced. Accuracy is misleading. Instead, we use the **Receiver Operating Characteristic (ROC) curve**.

### 2.6.2. ROC Curve Construction

The ROC curve plots **True Positive Rate (TPR)** vs **False Positive Rate (FPR)** at various classification thresholds:

- **TPR** (Recall): $\frac{TP}{TP+FN}$ — of actual defaults, how many did we catch?
- **FPR**: $\frac{FP}{FP+TN}$ — of actual non-defaults, how many did we falsely flag?

**ROC Interpretation**
Top-left corner $(0, 1)$: perfect classifier.
Diagonal: random guessing.
Below diagonal: worse than random!

```
pub fn roc_curve(scores: &[f64], labels: &[bool]) -> Vec<(f64, f64)> {
    // Sort by score descending
    let mut pairs: Vec<(f64, bool)> = scores.iter()
        .copied()
        .zip(labels.iter().copied())
        .collect();
    pairs.sort_by(|a, b| b.0.partial_cmp(&a.0).unwrap());

    let total_pos = labels.iter().filter(|&&l| l).count();
    let total_neg = labels.len() - total_pos;

    let mut roc_points = vec![(0.0, 0.0)];
    let mut tp = 0;
    let mut fp = 0;

    for (_score, label) in pairs {
        if label { tp += 1; } else { fp += 1; }
        let fpr = fp as f64 / total_neg as f64;
        let tpr = tp as f64 / total_pos as f64;
        roc_points.push((fpr, tpr));
    }

    roc_points
}
```

### 2.6.3. Area Under Curve (AUC)

The **AUC** summarizes the ROC curve as a single number:

- AUC = 1.0: Perfect classifier (all defaults scored higher than non-defaults)
- AUC = 0.5: Random classifier (no predictive power)
- AUC = 0.7: Fair classifier (industry often requires $\geq 0.7$)

**AUC Interpretation**
AUC = 1.0: perfect
AUC = 0.5: random
AUC < 0.5: worse than random

We compute AUC via trapezoidal integration:

```
pub fn auc(roc_points: &[(f64, f64)]) -> f64 {
    let mut area = 0.0;
    for i in 1..roc_points.len() {
        let (x0, y0) = roc_points[i - 1];
        let (x1, y1) = roc_points[i];
        // Trapezoidal rule: width * average height
        area += (x1 - x0) * (y0 + y1) / 2.0;
    }
    area
}
```

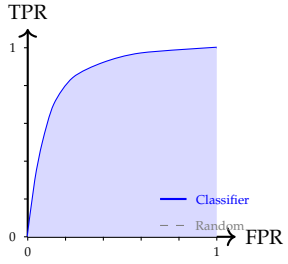



**Figure 2.1.:** ROC curve: the blue line shows classifier performance, dashed line is random guessing. Area under curve (AUC) is shaded.

**Key Insight**

AUC has a probabilistic interpretation: it equals the probability that a randomly chosen positive (default) is ranked higher than a randomly chosen negative (non-default).

## 2.7. Trait-Based Architecture

Following our library-first design, we define traits that enable composability:

```rust
/// Trait for models that produce probability predictions.
pub trait BinaryClassifier {
    fn predict(&self, features: &[f64]) -> bool;
    fn predict_proba(&self, features: &[f64]) -> f64;
}

/// Trait for models that produce raw scores.
pub trait Scorer {
    fn score(&self, features: &[f64]) -> f64;
}
```

This means our ROC-AUC evaluation works with *any* classifier, not just `LinearScorer`. Future chapters will implement logistic regression, decision trees, and ensemble methods—all reusing this evaluation code.

**Future Benefits**

Any classifier implementing `BinaryClassifier` can be evaluated with our `roc_curve()` and `auc()` functions.

## 2.8. Results and Analysis

### 2.8.1. Example Usage

```
$ cargo run -p qf-02-loan --example loan_analysis
Loading loan applications...
Loaded 10000 applications (850 defaults, 8.5%)

Feature extraction complete.
Training simple scorer (hardcoded weights)...

Evaluation Results:
  AUC: 0.724
  Optimal Threshold: 0.35
  At threshold 0.5:
    Precision: 0.42
    Recall: 0.58
    F1 Score: 0.49
```

### 2.8.2. Interpreting the Results

- **AUC = 0.72**: The model has reasonable discriminative power. It correctly ranks defaults above non-defaults 72% of the time.
- **Precision = 0.42**: Of predicted defaults, 42% actually default. Many false alarms.
- **Recall = 0.58**: We catch 58% of actual defaults. 42% are missed.

**Warning**

We used hardcoded weights, not learned weights. Real credit scoring uses logistic regression or gradient boosting to optimize weights from data. That comes in later chapters.

## 2.9. Exercises

1. **Weight Sensitivity**: Change the weights in `LinearScorer`. How does AUC change? Which features matter most?
2. **Threshold Selection**: Plot precision and recall at various thresholds. Find the threshold that maximizes F1 score.
3. **Feature Importance**: Train separate single-feature scorers. Rank features by their individual AUC.
4. **Cross-Validation**: Split data into 5 folds. Report mean and standard deviation of AUC across folds.
5. **Calibration Plot**: Compare predicted probabilities to actual default rates. Is the model well-calibrated?

## 2.10. Key Takeaways

- **Feature engineering** transforms raw data into predictive signals
- **One-hot encoding** handles categorical variables
- **Min-max normalization** scales features to comparable ranges
- **Sigmoid** converts scores to probabilities
- **ROC-AUC** evaluates ranking quality, robust to class imbalance
- **Trait-based design** enables reusable evaluation code

**Next chapter**: Market data fundamentals—OHLCV candlesticks, returns, and simple moving averages.

# Market Data: Stock Price Analysis

# 3.

**Companion Code:**

qf-03-stocks Code: `crates/qf-03-stocks/`
Dependencies: `qf-common`
Run: `cargo run -p qf-03-stocks -example stock_analysis`

## 3.1. Learning Objectives

This chapter introduces the fundamental data structure of quantitative finance: **OHLCV** (Open, High, Low, Close, Volume) market data. We learn:

- OHLCV data structure and its significance
- Candlestick anatomy—body, shadows, and what they reveal
- Basic market statistics (range, volume, price change)
- Moving averages: SMA and EMA
- The `TimeSeries` trait for generic operations

**Key Insight**

OHLCV data is the foundation of all quantitative analysis. Every strategy, every backtest, every indicator starts here. Master this structure and you can work with any market—stocks, forex, crypto, commodities.

## 3.2. Introduction: What is OHLCV?

**OHLCV**
Open, High, Low, Close, Volume—the five essential data points for any trading period (day, hour, minute).

Every trading period (day, hour, minute) produces five key data points:

- **Open**: First traded price of the period
- **High**: Highest price reached during the period
- **Low**: Lowest price reached during the period
- **Close**: Last traded price of the period
- **Volume**: Total number of shares/contracts traded

This data tells a story. The range between high and low shows volatility. The relationship between open and close shows direction. Volume confirms conviction.

## 3.3. Data Structure

### 3.3.1. Rust Implementation

```rust
#[derive(Debug, Clone, Deserialize)]
pub struct Ohlcv {
    pub date: String,
    pub open: f64,
    pub high: f64,
    pub low: f64,
    pub close: f64,
```

```rust
    pub volume: u64,
    pub adj_close: Option<f64>,
}
```

**Adjusted Close**

Accounts for splits and dividends. If a stock splits 2:1, historical closes are halved so charts remain continuous.

### 3.3.2. Data Validation

OHLCV data has natural constraints that we validate on load:

- low ≤ high (always)
- low ≤ open ≤ high
- low ≤ close ≤ high

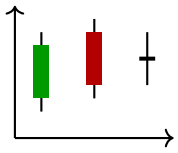
```rust
// Validate OHLCV relationships
if record.high < record.low {
    return Err(CommonError::ParseError(format!(
        "Row {}: high ({}) < low ({})",
        idx + 2, record.high, record.low
    )));
}
```

## 3.4. Candlestick Anatomy

A candlestick visualizes OHLCV data. Understanding its anatomy is essential:

**Candlestick Parts**

Body: $|close - open|$
Upper shadow: $high - \max(O, C)$
Lower shadow: $\min(O, C) - low$



**Candlesticks**

Green: bullish
Red: bearish
Line: doji

| Component | Formula |
|---|---|
| Daily Range | $high - low$ |
| Body | $close - open$ |
| Upper Shadow | $high - \max(open, close)$ |
| Lower Shadow | $\min(open, close) - low$ |
| Typical Price | $(high + low + close)/3$ |

### 3.4.1. Rust Implementation

```rust
impl Ohlcv {
    /// Daily price range
    pub fn daily_range(&self) -> f64 {
        self.high - self.low
    }

    /// Candle body (positive = bullish)
    pub fn body(&self) -> f64 {
        self.close - self.open
    }

    /// Upper shadow length
    pub fn upper_shadow(&self) -> f64 {
        self.high - self.open.max(self.close)
    }

    /// Lower shadow length
    pub fn lower_shadow(&self) -> f64 {
        self.open.min(self.close) - self.low
    }

    /// True if close > open (green candle)
    pub fn is_bullish(&self) -> bool {
        self.close > self.open
    }

    /// Representative price: (H + L + C) / 3
    pub fn typical_price(&self) -> f64 {
        (self.high + self.low + self.close) / 3.0
    }
}
```

## 3.5. Market Statistics

### 3.5.1. Basic Aggregations

Given a series of OHLCV records, we compute summary statistics:

```rust
/// Average trading volume
pub fn average_volume(data: &[Ohlcv]) -> f64 {
    if data.is_empty() { return 0.0; }
    let sum: u64 = data.iter().map(|o| o.volume).sum();
    sum as f64 / data.len() as f64
}

/// Price change: (last - first) / first
pub fn price_change(data: &[Ohlcv]) -> f64 {
    if data.len() < 2 { return 0.0; }
    let first = data[0].close;
    let last = data[data.len() - 1].close;
    (last - first) / first
}

/// Highest high across all records
pub fn highest_high(data: &[Ohlcv]) -> f64 {
    data.iter().map(|o| o.high).fold(f64::NEG_INFINITY, f64::max)
}

/// Lowest low across all records
pub fn lowest_low(data: &[Ohlcv]) -> f64 {
    data.iter().map(|o| o.low).fold(f64::INFINITY, f64::min)
}
```

**Functional Style**

Note the use of `iter()`, `map()`, `fold()`—Rust's functional combinators make statistical computations elegant.

## 3.6. Moving Averages

### 3.6.1. Simple Moving Average (SMA)

The SMA is the arithmetic mean of the last $n$ prices:

$$\text{SMA}_t = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

**SMA Formula**

$\text{SMA}_t = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$

where $n$ is the period.

```rust
pub fn sma(prices: &[f64], period: usize) -> Vec<f64> {
    if period == 0 || period > prices.len() {
        return Vec::new();
    }

    let mut result = Vec::with_capacity(prices.len() - period + 1);

    for i in 0..=(prices.len() - period) {
        let window = &prices[i..i + period];
        let sum: f64 = window.iter().sum();
        result.push(sum / period as f64);
    }

    result
}
```

### 3.6.2. Exponential Moving Average (EMA)

The EMA gives more weight to recent prices, making it more responsive:

$$\text{EMA}_t = \alpha \cdot P_t + (1 - \alpha) \cdot \text{EMA}_{t-1}$$

where $\alpha = \frac{2}{n+1}$ is the smoothing factor.

**EMA Formula**

$\text{EMA}_t = \alpha \cdot P_t + (1 - \alpha) \cdot \text{EMA}_{t-1}$

$\alpha = \frac{2}{n+1}$

```rust
pub fn ema(prices: &[f64], period: usize) -> Vec<f64> {
    if period == 0 || prices.is_empty() {
        return Vec::new();
    }

    let alpha = 2.0 / (period + 1) as f64;
    let mut result = Vec::with_capacity(prices.len());

    // Initialize with first price
    result.push(prices[0]);

    // Recursive formula
    for i in 1..prices.len() {
        let prev_ema = result[i - 1];
        let current_ema = alpha * prices[i] + (1.0 - alpha) * prev_ema;
        result.push(current_ema);
    }

    result
}
```

### 3.6.3. SMA vs EMA

| Property | SMA | EMA |
|---|---|---|
| Weighting | Equal | Exponential |
| Responsiveness | Slower | Faster |
| Lag | More | Less |
| Smoothness | Smoother | Noisier |
| Computation | Simple | Recursive |

**Key Insight**

SMA is a *lagging* indicator—it tells you what already happened. EMA reacts faster but can give false signals. Traders often use both: short-term EMA for entries, long-term SMA for trend confirmation.

## 3.7. TimeSeries Trait

Following our trait-based architecture, we define a generic interface:

```rust
pub trait TimeSeries {
    fn len(&self) -> usize;
    fn is_empty(&self) -> bool;
    fn closes(&self) -> Vec<f64>;
    fn volumes(&self) -> Vec<u64>;
}

impl TimeSeries for Vec<Ohlcv> {
    fn len(&self) -> usize { self.len() }
    fn is_empty(&self) -> bool { self.is_empty() }

    fn closes(&self) -> Vec<f64> {
        self.iter().map(|o| o.close).collect()
    }

    fn volumes(&self) -> Vec<u64> {
        self.iter().map(|o| o.volume).collect()
    }
}
```

**Trait Benefits**

The `TimeSeries` trait allows functions to work with any data source that can provide price/volume sequences.

This enables generic functions that work with any `TimeSeries`:

```rust
fn analyze<T: TimeSeries>(data: &T) -> Report {
    let closes = data.closes();
    let sma_20 = sma(&closes, 20);
    let sma_50 = sma(&closes, 50);
    // ...
```

```
}
```

## 3.8. Golden and Death Crosses

A classic trading signal using moving averages:

- **Golden Cross**: Short-term MA crosses *above* long-term MA (bullish signal)
- **Death Cross**: Short-term MA crosses *below* long-term MA (bearish signal)

```rust
fn find_crossovers(short_ma: &[f64], long_ma: &[f64]) -> Vec<(usize, bool)> {
    let mut crossovers = Vec::new();

    for i in 1..short_ma.len().min(long_ma.len()) {
        let prev_above = short_ma[i-1] > long_ma[i-1];
        let curr_above = short_ma[i] > long_ma[i];

        if !prev_above && curr_above {
            crossovers.push((i, true));  // Golden cross
        } else if prev_above && !curr_above {
            crossovers.push((i, false)); // Death cross
        }
    }

    crossovers
}
```

## 3.9. Results and Analysis

### 3.9.1. Example Output

```
$ cargo run -p qf-03-stocks --example stock_analysis
Loading OHLCV data from data/spy.csv...
Loaded 252 trading days

Statistics:
  Average Volume: 85,432,100
  Price Change: +12.5%
  Highest High: $485.32
  Lowest Low: $410.15
  Average Daily Range: $5.23

Moving Averages:
  SMA(20) latest: $472.15
  SMA(50) latest: $465.80
  EMA(20) latest: $474.22

Crossovers (last 12 months):
  Day 45: Golden Cross (bullish)
  Day 128: Death Cross (bearish)
  Day 201: Golden Cross (bullish)
```

## 3.10. Exercises

1. **Candlestick Patterns**: Implement detection for "doji" (open ≈ close) and "hammer" (small body, long lower shadow).
2. **Volume Analysis**: Add `volume_ma()` that computes moving average of volume. Flag days where volume exceeds 2x the average.
3. **Bollinger Bands**: Implement Bollinger Bands: SMA ± 2 standard deviations. This requires computing rolling standard deviation.
4. **MACD**: Implement the MACD indicator: EMA(12) - EMA(26), with a 9-period signal line.

5. **Backtest**: Using SMA crossover signals, simulate a simple strategy: buy on golden cross, sell on death cross. Track P&L.

## 3.11. Key Takeaways

- **OHLCV** is the atomic unit of market data
- **Candlesticks** encode price action: body shows direction, shadows show rejection
- **SMA** averages equally; **EMA** weights recent prices
- **Crossovers** generate trading signals (but beware of lag!)
- **TimeSeries trait** enables generic, reusable analysis code

**Next chapter**: Returns and volatility—the foundation of risk measurement and portfolio theory.

# Core Concepts

# 4. Returns and Volatility

**Companion Code:**

qf-04-returns Code: `crates/qf-04-returns/`
Dependencies: `qf-common`, `qf-03-stocks`
Run: `cargo run -p qf-04-returns -example returns_analysis`

## 4.1. Learning Objectives

This chapter introduces the foundational vocabulary of quantitative finance. After completing it you can:

- Compute simple and logarithmic returns and know when to use each
- Measure volatility and annualise it for cross-frequency comparison
- Evaluate risk-adjusted performance with the Sharpe and Sortino ratios
- Quantify drawdowns and recovery time from a price series
- Use the `Returns` trait to abstract return computation across data sources

**Key Insight**

Returns, not prices, are the currency of quant research. Prices are non-stationary; returns are (approximately) stationary. Every quant model in this book starts by transforming prices into returns.

## 4.2. Simple vs Log Returns

### 4.2.1. Definitions

**Simple return**
$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$
**Log return**
$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$

The simple (arithmetic) return is the percentage change in price:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

The log (continuously compounded) return is the natural log of the price ratio:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(P_t) - \ln(P_{t-1})$$

The two are related exactly by

$$r_t^{\log} = \ln\left(1 + r_t^{\text{simple}}\right),$$

and for small returns ($|r| \ll 1$) they are nearly equal.

### 4.2.2. When to use which

| Property | Simple | Log |
|---|---|---|
| Interpretation | "I made 10%" | Less intuitive |
| Additivity over time | No | Yes |
| Additivity over assets | Yes | No |
| Symmetry | Symmetric | Unbounded below |
| Compounding | Multiplicative | Additive |

**Key Insight**

Log returns are additive across time: the $k$-period log return is the sum of the $k$ single-period log returns. This makes them natural for time-series modelling. Simple returns are additive across assets in a portfolio (weighted by portfolio weights), which makes them natural for portfolio returns.

### 4.2.3. Rust implementation

```rust
/// Simple return: (P_t - P_{t-1}) / P_{t-1}
pub fn simple_returns(prices: &[f64]) -> Vec<f64> {
    if prices.len() < 2 { return Vec::new(); }
    let mut out = Vec::with_capacity(prices.len() - 1);
    for i in 1..prices.len() {
        let prev = prices[i - 1];
        out.push(if prev == 0.0 { 0.0 }
                 else { (prices[i] - prev) / prev });
    }
    out
}

/// Log return: ln(P_t / P_{t-1})
pub fn log_returns(prices: &[f64]) -> Vec<f64> {
    if prices.len() < 2 { return Vec::new(); }
    let mut out = Vec::with_capacity(prices.len() - 1);
    for i in 1..prices.len() {
        let (prev, curr) = (prices[i - 1], prices[i]);
        out.push(if prev > 0.0 && curr > 0.0 { (curr / prev).ln() }
                 else { 0.0 });
    }
    out
}
```

**Cumulative return**
$R_{0\to t} = \prod_{k=1}^{t}(1 + r_k) - 1$

The cumulative return over a period is the running compounding of single-period simple returns:

$$R_{0\to t} = \prod_{k=1}^{t}(1 + r_k) - 1.$$

```rust
pub fn cumulative_returns(returns: &[f64]) -> Vec<f64> {
    let mut out = Vec::with_capacity(returns.len());
    let mut wealth = 1.0_f64;
    for &r in returns {
        wealth *= 1.0 + r;
        out.push(wealth - 1.0);
    }
    out
}
```

## 4.3. Volatility

**Sample std dev**

$$\sigma = \sqrt{\frac{1}{n-1}\sum_{t=1}^{n}(r_t - \bar{r})^2}$$

Volatility is the standard deviation of returns. We use the *sample* estimator (denominator $n - 1$), consistent with `qf_common::compute_stats()`:

$$\sigma = \sqrt{\frac{1}{n-1}\sum_{t=1}^{n}(r_t - \bar{r})^2}.$$

### 4.3.1. Annualisation

**Annualised vol**

$\sigma_{\text{ann}} = \sigma_{\text{period}} \cdot \sqrt{m}$
where $m$ is the number of periods per year (252 for daily).

To compare volatilities across frequencies (daily, weekly, monthly), we annualise. Under the assumption that returns are i.i.d., variance scales linearly with the horizon, so standard deviation scales with the square root:

$$\sigma_{\text{annual}} = \sigma_{\text{period}} \cdot \sqrt{m}.$$

```rust
pub fn volatility(returns: &[f64]) -> f64 {
    let n = returns.len();
    if n < 2 { return 0.0; }
    let mean: f64 = returns.iter().sum::<f64>() / n as f64;
    let ssd: f64 = returns.iter()
        .map(|&r| { let d = r - mean; d * d })
        .sum();
    (ssd / (n - 1) as f64).sqrt()
}

pub fn annualized_volatility(returns: &[f64], periods_per_year: f64) -> f64 {
    if periods_per_year <= 0.0 { return 0.0; }
    volatility(returns) * periods_per_year.sqrt()
}
```

**Warning**

The $\sqrt{m}$ rule assumes returns are independent and identically distributed. For assets with autocorrelated or volatility-clustered returns (see Chapter 8), annualisation by $\sqrt{m}$ overstates the true annual volatility.

### 4.3.2. Rolling volatility

A rolling window reveals how volatility changes over time—useful for regime detection and risk monitoring:

```rust
pub fn rolling_volatility(returns: &[f64], window: usize) -> Vec<f64> {
    if window == 0 || window > returns.len() { return Vec::new(); }
    let mut out = Vec::with_capacity(returns.len() - window + 1);
    for i in window - 1..returns.len() {
        let slice = &returns[i + 1 - window..=i];
        out.push(volatility(slice));
    }
    out
}
```

## 4.4. Risk-Adjusted Performance

### 4.4.1. Sharpe ratio

**Sharpe ratio**

$$S = \frac{\mathbb{E}[r] - r_f}{\sigma(r)}$$

The Sharpe ratio measures excess return per unit of total risk:

$$S = \frac{\bar{r} - r_f}{\sigma(r)}.$$

To annualise, multiply by $\sqrt{m}$ (because both the mean excess return and the standard deviation scale together under i.i.d.):

$$S_{\text{annual}} = S_{\text{period}} \cdot \sqrt{m}.$$

```rust
pub fn sharpe_ratio(returns: &[f64], risk_free_rate: f64) -> f64 {
    let n = returns.len();
    if n < 2 { return 0.0; }
    let vol = volatility(returns);
    if vol == 0.0 { return 0.0; }
    let mean: f64 = returns.iter().sum::<f64>() / n as f64;
    (mean - risk_free_rate) / vol
}

pub fn annualized_sharpe(returns: &[f64], rf: f64, periods_per_year: f64) -> f64 {
    if periods_per_year <= 0.0 { return 0.0; }
    sharpe_ratio(returns, rf) * periods_per_year.sqrt()
}
```

### 4.4.2. Sortino ratio

**Downside deviation**

$$\sigma_D = \sqrt{\frac{1}{n} \Sigma_{r_t < r_f} (r_f - r_t)^2}$$

The Sortino ratio replaces total volatility with *downside* volatility only. Upside volatility is good—it should not penalise the ratio:

$$\text{Sortino} = \frac{\bar{r} - r_f}{\sigma_D}, \qquad \sigma_D = \sqrt{\frac{1}{n} \sum_{r_t < r_f} (r_f - r_t)^2}.$$

```rust
pub fn sortino_ratio(returns: &[f64], risk_free_rate: f64) -> f64 {
    let n = returns.len();
    if n < 2 { return 0.0; }
    let downside_sq: f64 = returns.iter()
        .map(|&r| {
            let short = risk_free_rate - r;
            if short > 0.0 { short * short } else { 0.0 }
        })
        .sum();
    if downside_sq == 0.0 { return 0.0; }
    let downside_dev = (downside_sq / n as f64).sqrt();
    let mean: f64 = returns.iter().sum::<f64>() / n as f64;
    (mean - risk_free_rate) / downside_dev
}
```

**Key Insight**

Sortino > Sharpe whenever the return distribution is right-skewed (more upside than downside). For symmetric distributions the two ratios are equal in expectation. Sortino is preferred for evaluating hedge-fund-style strategies where upside volatility is desirable.

## 4.5. Drawdown

A drawdown is the percentage decline from a running peak:

$$D_t = \frac{P_t - \max_{k \le t} P_k}{\max_{k \le t} P_k} \le 0.$$

**Max drawdown**

$\text{MaxDD} = \min_t D_t$

The worst peak-to-trough decline over the period.

```rust
pub fn drawdown(prices: &[f64]) -> Vec<f64> {
    let mut out = Vec::with_capacity(prices.len());
    let mut running_max = f64::NEG_INFINITY;
    for &p in prices {
        if p > running_max { running_max = p; }
        out.push(if running_max == 0.0 { 0.0 }
                 else { (p - running_max) / running_max });
    }
    out
}

pub fn max_drawdown(prices: &[f64]) -> f64 {
    drawdown(prices).iter().copied().fold(0.0, f64::min)
}
```

The `DrawdownStats` struct summarises the worst episode:

- `max_drawdown`: the most negative drawdown value
- `max_drawdown_duration`: longest run of consecutive negative-drawdown observations
- `recovery_time`: steps from the trough back to a new high (`None` if the series never recovered)

**Key Insight**

Max drawdown measures *path risk*, not just terminal risk. A strategy that returns 10% with a 50% drawdown along the way is very different from one that returns 10% smoothly. Most investors abandon strategies with deep drawdowns long before recovery—a risk that terminal-return statistics completely miss.

## 4.6. The Returns Trait

**Trait benefits**

Define the capability (*"can produce returns"*) once; implement for slices, owned vectors, and OHLCV data.

Following the workspace-wide trait-first architecture rule, we abstract return computation over price sources:

```rust
pub trait Returns {
    fn simple_returns(&self) -> Vec<f64>;
    fn log_returns(&self) -> Vec<f64>;
}

impl Returns for &[f64] { /* delegate to free functions */ }
impl Returns for Vec<f64> { /* delegate to the slice impl */ }
impl Returns for Vec<Ohlcv> {
    fn simple_returns(&self) -> Vec<f64> {
        let closes: Vec<f64> = self.iter().map(|o| o.close).collect();
        simple_returns(&closes)
    }
    // log_returns similarly uses closing prices
}
```

This lets generic code consume returns from any source that implements `Returns`, keeping analytics decoupled from data loading.

## 4.7. Example Output

```
$ cargo run -p qf-04-returns --example returns_analysis
Returns Analysis
================

Simple Returns: mean=0.0031, std=0.0215
Log Returns: mean=0.0029, std=0.0214
Volatility (daily): 0.0215
Volatility (annualized): 0.3417
Sharpe Ratio (annualized): 2.29
Sortino Ratio: 0.22
Max Drawdown: -7.50%
Drawdown duration: 6 steps, recovery: Some(6)

Trait check (Vec<f64>): first simple return = 0.0030
```

## 4.8. Exercises

1. **Return conversion**: Write a function that converts a series of simple returns to log returns and vice versa. Verify the identity $r^{\log} = \ln(1 + r^{\text{simple}})$ with a property test.
2. **Annualisation sanity check**: For daily, weekly, and monthly returns of the same asset, verify that the annualised volatilities are approximately equal (they differ only by sampling noise).
3. **Calmar ratio**: Implement the Calmar ratio: annualised return divided by the absolute value of the maximum drawdown. Compare it with the Sharpe ratio on a synthetic strategy.
4. **Rolling Sharpe**: Implement a rolling Sharpe ratio (windowed mean and windowed std) and visualise how it changes over time. Where does it dip? Why?
5. **Drawdown with recovery**: Extend `drawdown_stats` to report the average drawdown (mean of negative drawdown values) and the proportion of time spent in drawdown.

## 4.9. Key Takeaways

- **Log returns** are time-additive; **simple returns** are portfolio-additive. Choose accordingly.
- **Volatility** uses sample std ($n - 1$); annualise by multiplying by $\sqrt{m}$, valid under i.i.d.
- **Sharpe** penalises all volatility; **Sortino** penalises only downside.
- **Max drawdown** measures path risk—the deepest hole on the equity curve.
- **The `Returns` trait** keeps analytics decoupled from data sources, following the workspace trait-first rule.

**Next chapter**: Your first backtest—an SMA crossover strategy with P&L accounting, transaction costs, and drawdown-based risk evaluation.

# 5. Backtesting Strategies

**Companion Code:**

qf-05-backtest Code: `crates/qf-05-backtest/`
Dependencies: `qf-common`, `qf-03-stocks`, `qf-04-returns`
Run: `cargo run -p qf-05-backtest -example backtest_demo`

## 5.1. Learning Objectives

This chapter introduces the mechanics of evaluating a trading strategy on historical data. After completing it you can:

- Explain what a backtest is — and what it is not
- Generate trading signals from indicators with no look-ahead bias
- Account for transaction costs in P&L
- Evaluate a strategy with the risk-adjusted metrics from Chapter 4
- Avoid the three classic backtest traps: overfitting, look-ahead bias, and survivorship bias

**Key Insight**

A backtest is a *simulation*, not a prediction. It tells you how a strategy *would have* performed, not how it *will* perform. The number one failure mode is overfitting to historical data: a strategy that looks great in backtest and loses money in production.

## 5.2. What Is Backtesting?

Backtesting is the replay of a trading strategy over historical price data, bar by bar, to estimate its hypothetical performance. The engine maintains state (position, cash, equity) and lets the strategy decide, at each bar, what action to take given the price history it has seen so far.

Three assumptions are implicit and worth naming:

1. **Market stationarity**: the statistical properties of the historical sample are representative of the future. Often false.
2. **Capacity**: the strategy can take the simulated sizes without moving the market. True for retail; false for institutional size.
3. **Execution**: the simulated fills are achievable. Slippage, partial fills, and market impact are real and cost money.

**Simulation vs prediction**

A backtest does not predict. It is a *counterfactual*: "if you had traded this strategy with these rules in this market, here is what would have happened." Any conclusion about the future requires additional assumptions (market stationarity, capacity, regime persistence) that the backtest itself cannot validate.

## 5.3. Signals and Positions

A *signal* is a discrete instruction for the current bar. We use three:

**Signal vs position**

A signal is an *instruction* (what to do this bar); a position is a *state* (what we are currently holding). The engine applies the signal to update the position. Decoupling them lets you replay the same signal stream under different execution models.

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Signal {
    Buy,  // Enter long
    Sell, // Exit long (long-only for this chapter)
    Hold, // Do nothing
}
```

A *position* is what the engine is currently holding:

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Position {
    Long,  // Holding the asset
    Short, // Reserved for later phases
    Flat,  // No position
}
```

**Key Insight**

Why separate `Signal` and `Position`? A signal is an *intention*; a position is a *state*. The engine translates intentions into state transitions, charging transaction costs on every change. This separation lets the strategy stay declarative ("I want to be long now") and the engine stay responsible for the messy realities (costs, fill assumptions, position limits).

## 5.4. The Strategy Trait

The extension point of the engine is the `Strategy` trait:

```rust
pub trait Strategy {
    fn signal(&self, data: &[Ohlcv], index: usize) -> Signal;
    fn name(&self) -> &str;
}
```

The engine is generic over `S: Strategy`, so new strategies drop in without engine changes. The trait takes the full slice plus the current index so the strategy can compute any indicator that fits in the observed history.

**No look-ahead**

`signal(data, index)` receives `data[..=index]`. The strategy must never read past the current bar. The engine enforces this by passing a slice and the current index — implementations physically cannot access `data[index+1..]` without unsafe code.

## 5.5. SMA Crossover

The canonical first strategy: go long when a short moving average crosses above a long moving average (the *golden cross*); exit when it crosses below (the *death cross*).

```rust
pub struct SmaCrossover {
    short_period: usize,
    long_period: usize,
}

impl Strategy for SmaCrossover {
    fn signal(&self, data: &[Ohlcv], index: usize) -> Signal {
        if index < self.long_period { return Signal::Hold; }

        let short_now = match sma_at(data, index, self.short_period) {
            Some(v) => v, None => return Signal::Hold,
        };
        let long_now = match sma_at(data, index, self.long_period) {
            Some(v) => v, None => return Signal::Hold,
        };
        let short_prev = match sma_at(data, index - 1, self.short_period) {
            Some(v) => v, None => return Signal::Hold,
        };
        let long_prev = match sma_at(data, index - 1, self.long_period) {
            Some(v) => v, None => return Signal::Hold,
        };

        let crossed_up   = short_prev <= long_prev  && short_now >  long_now;
```

**Golden / death cross**

$\text{SMA}_S$ crosses above $\text{SMA}_L \Rightarrow$ Buy

$\text{SMA}_S$ crosses below $\text{SMA}_L \Rightarrow$ Sell

A crossover is a *change in the relationship*, not a level. The previous bar must satisfy the opposite inequality.

```
        let crossed_down = short_prev >= long_prev  && short_now <  long_now;

        if crossed_up   { Signal::Buy  }
        else if crossed_down { Signal::Sell }
        else { Signal::Hold }
    }
    fn name(&self) -> &str { "SMA Crossover" }
}
```

**Key Insight**

Crossover strategies are *trend followers*: they only make money when prices trend. In choppy, mean-reverting markets they lose, because each crossover is a false signal that gets reversed at the next crossover — with transaction costs paid on both sides.

## 5.6. P&L with Transaction Costs

**Transaction cost**

At entry: $c_{\text{entry}} = \text{notional} \cdot \tau$

At exit: $c_{\text{exit}} = \text{shares} \cdot P_{\text{exit}} \cdot \tau$

Net P&L: $\text{shares} \cdot (P_{\text{exit}} - P_{\text{entry}}) - c_{\text{entry}} - c_{\text{exit}}$

Every state change costs money. The engine charges a proportional cost $\tau$ (e.g. $\tau = 0.001$ for 10 basis points) on the traded notional:

```
// Enter long: spend all cash at the close, paying the cost first.
let notional = capital;
let cost = notional * config.transaction_cost;
shares = (notional - cost) / price;
total_costs += cost;

// Exit long: sell all shares at the close.
let notional = shares * price;
let cost = notional * config.transaction_cost;
total_costs += cost;
capital = notional - cost;
```

At each bar, equity is marked to market: `shares * close` when long, cash when flat. The per-bar simple returns of the equity curve become the `daily_returns` used by the risk metrics from Chapter 4.

## 5.7. Performance Evaluation

**Why reuse `qf-04-returns`?**

Sharpe, Sortino, and max drawdown are return-level statistics — they do not care how the returns were generated. Once the backtest produces a daily return series, evaluation is identical to computing the same metrics on a buy-and-hold benchmark. Code reuse follows the math.

`BacktestResult` delegates to `qf-04-returns` for the four headline metrics:

```
impl BacktestResult {
    pub fn sharpe(&self, rf: f64) -> f64 {
        qf_04_returns::annualized_sharpe(&self.daily_returns, rf, 252.0)
    }
    pub fn sortino(&self, rf: f64) -> f64 {
        qf_04_returns::sortino_ratio(&self.daily_returns, rf)
    }
    pub fn max_drawdown(&self) -> f64 {
        qf_04_returns::max_drawdown(&self.equity_curve)
    }
    pub fn volatility(&self) -> f64 {
        qf_04_returns::annualized_volatility(&self.daily_returns, 252.0)
    }
}
```

**Key Insight**

The risk metrics operate on the *equity curve*, not the raw price series. A strategy that goes flat for half the period has a different risk profile from buy-and-hold even on the same underlying, because the equity curve is flat whenever the position is flat. Volatility and drawdown reflect this exposure pattern automatically.

## 5.8. Buy-and-Hold Benchmark

**Always compare to buy-and-hold**

An active strategy that returns 10% looks good until you realise the underlying asset returned 15% over the same period. The buy-and-hold benchmark is the floor any active strategy must beat *after* costs. If it does not, the activity added no value.

Every active strategy must be compared against the simplest alternative: buy on the first bar and hold forever. `BuyAndHold` is a two-line strategy:

```rust
impl Strategy for BuyAndHold {
    fn signal(&self, _data: &[Ohlcv], index: usize) -> Signal {
        if index == 0 { Signal::Buy } else { Signal::Hold }
    }
    fn name(&self) -> &str { "Buy and Hold" }
}
```

If an active strategy cannot beat buy-and-hold after costs, the active component is adding noise and cost, not value. The benchmark also lets you decompose returns: alpha (skill) vs beta (market exposure).

## 5.9. Common Pitfalls

**The two cardinal sins**

*Overfitting* and *look-ahead bias* are the two ways a backtest lies to you. Overfitting makes the past look good; look-ahead bias makes the future knowable. Both produce strategies that fail miserably in live trading. Treat any backtest Sharpe $> 2$ with deep suspicion until both are ruled out.

**Overfitting** Tuning the SMA periods $(S, L)$ to maximise in-sample Sharpe almost always produces a strategy that fails out-of-sample. Use walk-forward validation (deferred to a later phase) and be suspicious of parameters tuned on the same data you evaluate on.

**Look-ahead bias** Using information not available at decision time. Common sources: using the close of bar $t$ to decide at bar $t$ (only valid if the trade executes at the close), indicator warm-up that reads the future, corporate actions applied with hindsight. Our `signal(data, index)` interface rules out most of these by construction.

**Survivorship bias** Backtesting only on companies that survived to the present. Companies that went bankrupt are absent, inflating historical returns. Use point-in-time universes when available.

## 5.10. Example Output

```
$ cargo run -p qf-05-backtest --example backtest_demo
==============================
Backtest Results
==============================

Period: 2024-001 to 2024-252

--- Strategy: SMA Crossover (10/30) ---
  Total Return:   3.49%
  Sharpe (ann.):   2.55
  Sortino:         0.31
  Max Drawdown:    -0.41%
  Trades:          1

--- Strategy: Buy and Hold ---
  Total Return:    4.54%
  Sharpe (ann.):   1.70
  Max Drawdown:    -6.35%

Outperformance vs Buy & Hold: -1.04%
```

The SMA crossover underperforms buy-and-hold in this synthetic regime: the cost of false signals in choppy sections outweighs the benefit of avoiding the mid-year drawdown. This is the typical result for trend-following strategies in noisy markets, and it is the right lesson to take from a first backtest.

## 5.11. Exercises

1. **Cost sensitivity**: Run the 10/30 crossover on the synthetic series for $\tau \in \{0, 0.001, 0.0025, 0.005, 0.01\}$. Plot total return vs $\tau$. At what cost does the strategy stop being profitable?
2. **Parameter sensitivity**: For $\tau = 0.001$, sweep $(S, L) \in \{5, 10, 20\} \times \{30, 50, 100\}$ with $S < L$. Which configuration maximises Sharpe? Is it the same one that maximises total return? Why or why not?
3. **EMA crossover**: Implement an EMA crossover strategy by replacing SMA with `qf_03_stocks::ema()`. Compare its turnover (number of trades) and Sharpe against the SMA crossover on the same data.
4. **Position sizing**: Modify the engine to invest a fixed fraction $f \in (0, 1]$ of capital on each entry instead of 100%. How does the equity curve change for $f = 0.5$? What is the trade-off?
5. **Walk-forward (conceptual)**: Split the 252-bar series into an in-sample window (first 180 bars) and an out-of-sample window (last 72 bars). Tune $(S, L)$ on the in-sample, freeze it, and evaluate on out-of-sample. How much does Sharpe degrade from in-sample to out-of-sample? This is the simplest walk-forward analysis.

## 5.12. Key Takeaways

- **Backtesting** is a simulation, not a prediction—it tells you how a strategy *would have* performed
- **Signals vs positions**: signals are intentions, positions are state; the engine translates one to the other
- **Transaction costs** erode returns and must be modelled explicitly
- **SMA crossover** is a trend-following strategy that loses in choppy markets
- **Buy-and-hold** is the benchmark every active strategy must beat
- **Pitfalls**: overfitting, look-ahead bias, and survivorship bias are the classic backtest traps

**Next chapter**: Foundations—moments, hand-rolled RNG, and geometric Brownian motion simulation.

# Quantitative Methods

# Foundations: Moments, RNG, and Simulation 6.

**Companion Code:**

quant-core Code: `crates/quant-core/`
Dependencies: `thiserror`
Run: `cargo run -p quant-core -example fat_tails`

## 6.1. Learning Objectives

This chapter begins the advanced track. From here on, all math is hand-rolled — no `rand`, `nalgebra`, or `statrs`. After completing it you can:

- Build a validated price series that rejects non-finite and non-positive values at construction time
- Compute the first four sample moments (mean, variance, skewness, excess kurtosis) from first principles
- Apply any aggregation over a sliding window with a generic `rolling` function
- Implement a deterministic PRNG (xorshift64) and a normal sampler (Box-Muller) without external crates
- Simulate geometric Brownian motion paths and understand the exact solution to the GBM SDE
- Detect fat tails via excess kurtosis and explain why Gaussian risk models underestimate tail risk

**Key Insight**

The Gaussian distribution has excess kurtosis 0. Real financial returns have positive excess kurtosis: large losses and gains happen more often than a normal model predicts. Any risk model that assumes Gaussian returns will underestimate the probability of extreme events. The 2008 crisis was in part a fat-tail crisis.

## 6.2. A Validated Price Series

The foundation of every quant computation is a clean price series. We wrap `Vec<f64>` in a newtype that enforces the invariant "strictly positive and finite" at construction time:

**The invariant**

Every element of a `PriceSeries` is strictly positive and finite. The constructor rejects the rest.

```rust
#[derive(Debug, Clone, PartialEq)]
pub struct PriceSeries(Vec<f64>);

impl PriceSeries {
    pub fn new(prices: Vec<f64>) -> Result<Self, CoreError> {
        for &p in &prices {
            if !p.is_finite() || p <= 0.0 {
                return Err(CoreError::InvalidPrice);
            }
        }
        Ok(Self(prices))
    }
    pub fn as_slice(&self) -> &[f64] { &self.0 }
    pub fn len(&self) -> usize { self.0.len() }
}
```

Rejecting `NaN` and `+inf` at the boundary means every downstream function can assume finite inputs and skip defensive checks. This is the "parse, don't validate" pattern applied to numeric data.

## 6.3. Returns, Again

**Why re-implement?**

`quant-core` is the foundation every other crate builds on. It must be self-contained: no circular dep on `qf-04-returns`. The formulas are two lines each, so the cost of duplication is trivial compared to the architectural benefit of a clean dependency DAG.

Simple and log returns were introduced in Chapter 4. We re-implement them here so `quant-core` is self-contained:

```rust
pub fn simple_returns(prices: &[f64]) -> Vec<f64> {
    if prices.len() < 2 { return Vec::new(); }
    let mut out = Vec::with_capacity(prices.len() - 1);
    for i in 1..prices.len() {
        let prev = prices[i - 1];
        out.push(if prev == 0.0 { 0.0 }
                 else { (prices[i] - prev) / prev });
    }
    out
}
```

**Key Insight**

The identity $\ln(1 + r^{\text{simple}}) = r^{\log}$ holds exactly. For small returns the two are nearly equal; for large returns they diverge. This identity is a cheap property test that catches implementation bugs in either function.

## 6.4. Moments

**Central moment**

$m_k = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^k$

**Skewness**

$g_1 = \frac{m_3}{m_2^{3/2}}$

**Excess kurtosis**

$g_2 = \frac{m_4}{m_2^2} - 3$

The $k$-th central moment is

$$m_k = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^k.$$

Skewness and excess kurtosis are standardised ratios of these moments:

$$g_1 = \frac{m_3}{m_2^{3/2}} \quad \text{(skewness)}$$

$$g_2 = \frac{m_4}{m_2^2} - 3 \quad \text{(excess kurtosis)}$$

The Gaussian distribution has $g_1 = 0$ and $g_2 = 0$. Positive skewness means a longer right tail; negative skewness a longer left tail. Positive excess kurtosis means heavier tails than Gaussian.

```rust
pub fn skewness(data: &[f64]) -> Result<f64, CoreError> {
    if data.len() < 3 { return Err(CoreError::InsufficientData { .. }); }
    let m = mean(data);
    let m2 = central_moment_2(data, m);
    let m3 = central_moment_3(data, m);
    Ok(m3 / m2.powf(1.5))
}

pub fn excess_kurtosis(data: &[f64]) -> Result<f64, CoreError> {
    if data.len() < 4 { return Err(CoreError::InsufficientData { .. }); }
    let m = mean(data);
    let m2 = central_moment_2(data, m);
    let m4 = central_moment_4(data, m);
    Ok(m4 / (m2 * m2) - 3.0)
}
```

**Key Insight**

We use the sample variance ($n-1$ denominator) for $\sigma^2$, consistent with `qf_common::compute_stats()`, but use population central moments (denominator $n$) for $m_2, m_3, m_4$ in skewness and kurtosis. This is the textbook definition; the difference between the two vanishes for large $n$.

The `Moments` trait abstracts over any data source that can produce sample moments, so the same analytics work on slices, owned vectors, and `PriceSeries`:

```rust
pub trait Moments {
    fn mean(&self) -> f64;
    fn variance(&self) -> Result<f64, CoreError>;
    fn std_dev(&self) -> Result<f64, CoreError>;
    fn skewness(&self) -> Result<f64, CoreError>;
    fn excess_kurtosis(&self) -> Result<f64, CoreError>;
}

impl Moments for &[f64] { /* delegate to free functions */ }
impl Moments for Vec<f64> { /* delegate to slice impl */ }
impl Moments for PriceSeries { /* delegate to slice impl */ }
```

## 6.5. Rolling Windows

**Rolling window**
For window $w$ on data of length $n$, the output has length $n - w + 1$ and entry $i$ aggregates `data`$[i..i + w]$.

A rolling window applies an aggregation over each contiguous slice of length $w$. The generic `rolling` function takes a closure, so any aggregation (mean, std, max, custom) drops in:

```rust
pub fn rolling<F, T>(window: usize, data: &[f64], f: F)
    -> Result<Vec<T>, CoreError>
where F: Fn(&[f64]) -> T
{
    if window == 0 || window > data.len() {
        return Err(CoreError::InvalidWindow { window, len: data.len() });
    }
    let mut out = Vec::with_capacity(data.len() - window + 1);
    for i in 0..=(data.len() - window) {
        out.push(f(&data[i..i + window]));
    }
    Ok(out)
}

pub fn rolling_mean(window: usize, data: &[f64]) -> Result<Vec<f64>, CoreError> {
    rolling(window, data, mean)
}
```

## 6.6. Hand-Rolled Simulation

### 6.6.1. xorshift64

**xorshift64**
Three shifts and a multiply. Period $2^{64}-1$. Fast, stateless, good enough for Monte Carlo when cryptographic strength is not required.

We implement Vigna's xorshift64 PRNG: three xor-shifts on a 64-bit state, then a multiply by a constant. A seed of zero produces a degenerate stream, so we replace it with a fixed default:

```rust
pub struct XorShift64 { state: u64 }

impl Rng for XorShift64 {
    fn next_u64(&mut self) -> u64 {
        let mut x = self.state;
        x ^= x >> 12;
        x ^= x << 25;
        x ^= x >> 27;
        self.state = x;
        x.wrapping_mul(0x2545_F491_4F6C_DD1D)
```

```
    }
    fn next_f64(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
}
```

### 6.6.2. Box-Muller Normal

To sample $N(\mu, \sigma^2)$ without an external library, we use the Box-Muller transform: two independent uniforms $u_1, u_2 \sim U(0,1)$ produce one standard normal variate

$$Z = \sqrt{-2\ln u_1}\cos(2\pi u_2).$$

```
pub fn box_muller_normal<R: Rng + ?Sized>(
    rng: &mut R, mean: f64, std: f64,
) -> f64 {
    let u1 = rng.next_f64().max(f64::EPSILON);
    let u2 = rng.next_f64();
    let r = (-2.0 * u1.ln()).sqrt();
    let theta = 2.0 * PI * u2;
    mean + std * (r * theta.cos())
}
```

The `Distribution` trait lets us plug in new distributions (exponential, Student-$t$, jump-diffusion) without touching the simulator:

```
pub trait Distribution {
    fn sample<R: Rng + ?Sized>(&self, rng: &mut R) -> f64;
}

pub struct Normal { pub mean: f64, pub std: f64 }

impl Distribution for Normal {
    fn sample<R: Rng + ?Sized>(&self, rng: &mut R) -> f64 {
        box_muller_normal(rng, self.mean, self.std)
    }
}
```

### 6.6.3. Geometric Brownian Motion

**Exact GBM solution**

$S_{t+\Delta t} = S_t \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\sqrt{\Delta t}\, Z\right)$

With $\sigma = 0$: $S_T = S_0 e^{\mu T}$, deterministic.

The stochastic differential equation

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t$$

has the exact solution

$$S_{t+\Delta t} = S_t \exp\left(\left(\mu - \tfrac{1}{2}\sigma^2\right)\Delta t + \sigma\sqrt{\Delta t}\, Z\right),$$

where $Z \sim N(0,1)$. We simulate one step at a time:

```
pub fn gbm_paths<R: Rng + ?Sized>(
    s0: f64, mu: f64, sigma: f64, t: f64,
    n_steps: usize, n_paths: usize, rng: &mut R,
) -> Vec<Vec<f64>> {
    let dt = t / n_steps as f64;
    let drift = (mu - 0.5 * sigma * sigma) * dt;
    let diffusion = sigma * dt.sqrt();
    let normal = Normal::standard();
    let mut paths = Vec::with_capacity(n_paths);
    for _ in 0..n_paths {
        let mut path = Vec::with_capacity(n_steps + 1);
        path.push(s0);
        let mut s = s0;
        for _ in 0..n_steps {
            let z = normal.sample(rng);
            s *= (drift + diffusion * z).exp();
            path.push(s);
        }
        paths.push(path);
```

```
    }
    paths
}
```

**Key Insight**

With $\sigma = 0$, the $Z$ term drops out and $S_T = S_0 \exp(\mu T)$ deterministically. This is a useful sanity check: a zero-volatility GBM should reproduce the risk-free growth exactly, with no RNG dependence.

## 6.7. Fat Tails

```
$ cargo run -p quant-core --example fat_tails
1. Pure Gaussian (10k N(0,1) draws)
  skewness     = +0.0267
  excess kurt  = +0.0323

2. Fat tails (every 100th draw x10)
  skewness     = -0.3051
  excess kurt  = +74.1272

3. GBM terminal prices (10k paths, 252 steps)
  skewness     = +0.5957
  excess kurt  = +0.5959
```

The pure Gaussian sample has excess kurtosis near 0, as expected. Scaling every 100th draw by 10 injects rare large moves; the excess kurtosis jumps from $\sim 0$ to $\sim 74$. GBM terminal prices are log-normally distributed, so they have positive excess kurtosis (heavier right tail than a Gaussian) and positive skewness — a direct consequence of the exponential in the exact solution.

**Key Insight**

A Gaussian model fit to fat-tailed data will price a 4-sigma event as a "1-in-15,000-years" occurrence. The true frequency under fat tails may be once a decade. This is why Value-at-Risk computed under Gaussian assumptions systematically underestimates tail losses, and why option-implied volatilities smile — the market knows the tails are fatter than Gaussian allows.

**Why fat tails matter**

Gaussian returns underestimate the probability of large moves by orders of magnitude. A $5\sigma$ day is a once-a-millennium event under $\mathcal{N}$; it happens every few years in real markets. The excess kurtosis of +74 on the synthetic fat-tailed sample is extreme, but real equity returns routinely show +4 to +10.

**Skewness sign**

Negative skewness means the left tail is heavier than the right — big down moves are more common than big up moves. Equity indices typically show negative skewness around $-0.3$ to $-1.0$. The asymmetry matters: option prices, VaR, and optimal leverage all depend on it.

## 6.8. Exercises

1. **Sample vs population variance**: For $x \in \{1, 2, 3, 4, 5\}$, compute both the population variance (denominator $n$) and the sample variance (denominator $n - 1$). How large is the bias? When does it matter?
2. **Skewness sign**: Construct a synthetic series with negative skew (long left tail) and one with positive skew. Verify `skewness()` returns the correct sign. Which one is typical of equity returns?
3. **RNG period**: The xorshift64 generator has period $2^{64} - 1$. Write a test that verifies the first 1000 outputs from seed 42 are distinct, and reason about why "distinct" is necessary but not sufficient for a good PRNG.
4. **GBM convergence**: Simulate 10 000 GBM paths over one year (252 steps) with $S_0 = 100$, $\mu = 0.05$, $\sigma = 0.2$. Compute the mean terminal price. Compare with the analytical expectation $E[S_T] = S_0 e^{\mu T}$. How close is the Monte Carlo estimate?

5. **Jump-diffusion**: Extend `gbm_paths()` with a Poisson jump term: with probability $\lambda \Delta t$ per step, multiply $S_t$ by $e^J$ where $J \sim N(\mu_J, \sigma_J^2)$. How does the excess kurtosis of terminal prices change as $\lambda$ grows?

## 6.9. Key Takeaways

- **PriceSeries** validates inputs at construction—"parse, don't validate"
- **Moments**: mean, variance, skewness, and excess kurtosis characterise distributions
- **Rolling windows** apply any aggregation over sliding slices
- **XorShift64*** is a fast, deterministic PRNG good enough for Monte Carlo
- **Box-Muller** transforms uniforms to standard normals
- **GBM** simulates price paths via the exact solution to the SDE
- **Fat tails**: real returns have positive excess kurtosis—Gaussian models underestimate tail risk

**Next chapter**: Time series analysis—OLS regression, ACF, the ADF test, and fractional differentiation.

# 7. Time Series Analysis

**Companion Code:**

quant-timeseries Code: `crates/quant-timeseries/`
Dependencies: `quant-core`, `thiserror`
Run: `cargo run -p quant-timeseries -example stationarity`

## 7.1. Learning Objectives

Most financial time series are non-stationary: prices wander, volatilities climb and fall, autocorrelations decay slowly. Naive regression on non-stationary data produces spurious correlations and unstable forecasts. After completing this chapter you can:

- Fit an ordinary least squares regression by hand via Gaussian elimination with partial pivoting on the normal equations
- Compute standard errors, t-statistics, and $R^2$ without any external statistics crate
- Compute the sample autocorrelation function and read its structure
- Test for a unit root with the Augmented Dickey-Fuller test and interpret the MacKinnon critical value
- Apply López de Prado's fixed-width fractional differentiation to make a series stationary while preserving memory
- Find the minimum differencing order $d$ that achieves stationarity on real and synthetic price series

**Key Insight**

Traditional differencing ($d = 1$) makes a price series stationary but erases all long-range memory — the resulting signal is essentially noise. Fractional differentiation ($0 < d < 1$) achieves stationarity while keeping as much memory as the data allow. The smallest such $d$ is the sweet spot for ML features that predict future returns.

## 7.2. Stationarity

**Weak stationarity**

A process is weakly stationary if its mean is constant and its autocovariance depends only on the lag, not on time. "Stationary" in this chapter means weakly stationary.

A time series $\{y_t\}$ is *weakly stationary* if its first two moments are time-invariant: $E[y_t] = \mu$ and $\text{Cov}(y_t, y_{t-k}) = \gamma_k$ depend only on the lag $k$. A random walk $y_t = y_{t-1} + \varepsilon_t$ fails this definition because its variance grows without bound. Non-stationary data break the assumptions behind OLS inference, forecast intervals, and most ML models.

The standard remedy is differencing: $\Delta y_t = y_t - y_{t-1}$. With $d = 1$ the series becomes stationary but loses all memory of the distant past. We need something in between.

## 7.3. OLS via Gaussian Elimination

**Normal equations**
$\hat\beta = (X'X)^{-1}X'y$. We never invert $X'X$ explicitly; we solve $X'X\hat\beta = X'y$ by Gaussian elimination.

Given a design matrix $X$ (rows are observations, columns are features) and a response $y$, the OLS estimator minimises $\|y - X\beta\|^2$. The first-order condition gives the normal equations

$$(X'X)\,\hat\beta = X'y.$$

We solve this $k \times k$ linear system by Gaussian elimination with partial pivoting: at each column we swap the row with the largest absolute pivot into the diagonal position, then eliminate below.

```rust
pub fn ols(x: &[Vec<f64>], y: &[f64]) -> Result<OlsFit, TimeSeriesError> {
    let n = x.len();
    let k = x[0].len();
    // Normal equations: A = X'X, b = X'y.
    let mut a = vec![vec![0.0_f64; k]; k];
    let mut b = vec![0.0_f64; k];
    for (xi, yi) in x.iter().zip(y.iter()) {
        for i in 0..k {
            b[i] += xi[i] * yi;
            for j in 0..k {
                a[i][j] += xi[i] * xi[j];
            }
        }
    }
    let coeffs = gauss_solve(&mut a, &mut b)?;
    // residuals, SSE, SST, R^2, standard errors, t-stats ...
}
```

**Key Insight**

Partial pivoting matters. Without it, a small diagonal entry relative to the entries below causes catastrophic cancellation when we eliminate. Pivoting costs nothing (a row swap per column) and lifts the worst-case error from exponential in $k$ to roughly $k$ machine epsilons.

Standard errors require the diagonal of $(X'X)^{-1}$. Rather than inverting the full matrix, we solve $X'X\,e_j = \text{unit}_j$ for each column $j$ and read off $e_j[j] = (X'X)^{-1}_{jj}$. The standard error is

$$\text{SE}(\hat\beta_j) = \sqrt{(X'X)^{-1}_{jj}\,s^2}, \quad s^2 = \frac{\text{SSE}}{n-k}.$$

The t-statistic is $\hat\beta_j / \text{SE}(\hat\beta_j)$ and $R^2 = 1 - \text{SSE}/\text{SST}$.

## 7.4. Autocorrelation Function

**Sample ACF**
$\hat\rho_k = \dfrac{\sum_{t=k+1}^{n}(y_t - \bar y)(y_{t-k} - \bar y)}{\sum_{t=1}^{n}(y_t - \bar y)^2}$, with $\hat\rho_0 = 1$ by construction.

The autocorrelation at lag $k$ is the correlation between $y_t$ and $y_{t-k}$. White noise has $\hat\rho_k \approx 0$ for $k > 0$; an AR(1) with coefficient $\phi$ has $\hat\rho_k \approx \phi^k$; a random walk has $\hat\rho_k \approx 1$ for all small $k$.

```rust
pub fn acf(data: &[f64], max_lag: usize)
    -> Result<Vec<f64>, TimeSeriesError>
{
    let n = data.len();
    let ybar: f64 = data.iter().sum::<f64>() / n as f64;
    let denom: f64 = data.iter()
        .map(|&v| { let d = v - ybar; d * d })
        .sum();
    let mut out = Vec::with_capacity(max_lag + 1);
    for k in 0..=max_lag {
        if k == 0 { out.push(1.0); continue; }
        let cov: f64 = (k..n)
            .map(|t| (data[t] - ybar) * (data[t - k] - ybar))
            .sum();
```

```
        out.push(cov / denom);
    }
    Ok(out)
}
```

## 7.5. Augmented Dickey-Fuller Test

**ADF regression**

$\Delta y_t = \alpha + \gamma\, y_{t-1} + \sum_{i=1}^{p} \delta_i\, \Delta y_{t-i} + \varepsilon_t$.

Reject $H_0 : \gamma = 0$ (unit root) when the t-ratio on $\hat{\gamma}$ is below the MacKinnon 5% critical value −2.86.

The ADF test regresses the first difference on the lagged level and lagged differences. The null hypothesis is a unit root ($\gamma = 0$); the alternative is stationarity ($\gamma < 0$). Because the test statistic does not follow a standard distribution under the null, we compare it to MacKinnon's simulated critical values. For a constant-only specification at 5% significance the value is −2.86.

```
pub const MACKINNON_5PCT: f64 = -2.86;

pub fn adf_test(data: &[f64], lags: usize)
    -> Result<AdfResult, TimeSeriesError>
{
    let n = data.len();
    let dy: Vec<f64> = (1..n).map(|t| data[t] - data[t - 1]).collect();
    // Rows: [1, y_{t-1}, dy[t-1], dy[t-2], ..., dy[t-lags]]
    let mut x: Vec<Vec<f64>> = Vec::new();
    let mut y: Vec<f64> = Vec::new();
    for t in (lags + 1)..n {
        let mut row = Vec::with_capacity(lags + 2);
        row.push(1.0);
        row.push(data[t - 1]);
        for i in 1..=lags { row.push(dy[t - 1 - i]); }
        y.push(dy[t - 1]);
        x.push(row);
    }
    let fit = ols(&x, &y)?;
    let stat = fit.t_stats[1]; // gamma is index 1 (after intercept)
    Ok(AdfResult {
        statistic: stat,
        critical_value: MACKINNON_5PCT,
        lags,
        is_stationary: stat < MACKINNON_5PCT,
    })
}
```

**Key Insight**

The lagged-level coefficient $\gamma$ is the key. If $\gamma = 0$, shocks to $y_t$ are permanent (unit root). If $\gamma < 0$, shocks decay at rate $1 + \gamma$ per period. The more negative $\gamma$, the faster the series reverts to the mean — and the more strongly stationary.

## 7.6. Fractional Differentiation

**Binomial series**

$(1 - B)^d = \sum_{k=0}^{\infty} w_k B^k$, where $w_0 = 1$ and $w_k = -w_{k-1} \dfrac{d - k + 1}{k}$.

For $d = 1$: $w = [1, -1, 0, \ldots]$.
For $d \in (0, 1)$: infinitely many non-zero weights decaying in magnitude.

The backward shift $B$ satisfies $By_t = y_{t-1}$. The first difference is $(1 - B)y_t$. Fractional differentiation generalises this to $(1 - B)^d$ for any real $d \in [0, 2]$. The binomial series gives the weights:

$$
\begin{aligned}
w_0 &= 1, \\
w_k &= -w_{k-1} \frac{d - k + 1}{k} \quad (k \geq 1).
\end{aligned}
$$

For $d = 1$ the weights are $[1, -1, 0, 0, \ldots]$ — the ordinary first difference. For $0 < d < 1$ infinitely many weights are non-zero, with $|w_k|$ decaying

roughly as $k^{-(1+d)}$. We truncate when $|w_k| <$ threshold (the fixed-width window of López de Prado, AFML Ch.5):

```rust
pub fn ffd_weights(d: f64, threshold: f64)
    -> Result<Vec<f64>, TimeSeriesError>
{
    let mut weights = vec![1.0_f64];
    let mut w = 1.0_f64;
    let mut k = 1_usize;
    loop {
        w = -w * (d - (k as f64) + 1.0) / k as f64;
        if w.abs() < threshold { break; }
        weights.push(w);
        if k > 10_000 { break; }
        k += 1;
    }
    Ok(weights)
}
```

**Key Insight**

The loop breaks *before* pushing a weight below the threshold. This makes $d = 0$ yield $[1.0]$ (identity, full memory) and $d = 1$ yield $[1.0, -1.0]$ exactly (ordinary first difference, no memory). Every intermediate $d$ keeps a finite prefix of the binomial series.

Applying the window is a weighted moving average over the last `weights.len()` observations:

```rust
pub fn frac_diff(data: &[f64], d: f64, threshold: f64)
    -> Result<Vec<f64>, TimeSeriesError>
{
    let weights = ffd_weights(d, threshold)?;
    let wlen = weights.len();
    let mut out = Vec::with_capacity(data.len() - wlen + 1);
    for t in (wlen - 1)..data.len() {
        let acc: f64 = weights.iter()
            .enumerate()
            .map(|(k, &w)| w * data[t - k])
            .sum();
        out.push(acc);
    }
    Ok(out)
}
```

### 7.6.1. Finding the Minimum $d$

Binary search over $d \in [0, 1]$ returns the smallest order for which the ADF test rejects the unit root. We test the endpoints first: if $d = 0$ already passes, no differencing is needed; if $d = 1$ fails, we return 1.0 conservatively.

```rust
pub fn find_min_d(data: &[f64], threshold: f64, tolerance: f64)
    -> Result<f64, TimeSeriesError>
{
    let passes = |d: f64| {
        let diff = frac_diff(data, d, threshold)?;
        Ok(adf_test(&diff, 1)?.is_stationary)
    };
    if passes(0.0)? { return Ok(0.0); }
    if !passes(1.0)? { return Ok(1.0); }
    let (mut lo, mut hi) = (0.0_f64, 1.0_f64);
    while hi - lo > tolerance {
        let mid = 0.5 * (lo + hi);
        if passes(mid)? { hi = mid; } else { lo = mid; }
    }
    Ok(hi)
}
```

## 7.7. The Stationarity-Memory Tradeoff in Action

**The tradeoff in one line**

Full differentiation ($d = 1$) makes the series stationary but erases all memory. No memory means no signal to model. Fractional differentiation ($d = 0.4$) keeps enough memory to predict while being stationary enough to satisfy the ADF test. This tradeoff is the core contribution of fractional differentiation to feature engineering for financial time series.

```
$ cargo run -p quant-timeseries --example stationarity
==============================
Stationarity Analysis
==============================

Random Walk (500 steps):
  ADF statistic: -1.1064
  Critical (5%):  -2.86
  Stationary:     NO

First Difference (d=1):
  ADF statistic: -15.8418
  Stationary:    YES
  ACF(1):        -0.0269 (memory destroyed)

Fractional Diff (d=0.4):
  ADF statistic: -4.8991
  Stationary:    YES
  ACF(1):         0.6878 (memory preserved)
==============================
```

The random walk has an ADF statistic of $-1.1$, well above the $-2.86$ critical value: we cannot reject the unit root. First differencing fixes stationarity (statistic plunges to $-15.8$) but the ACF at lag 1 is essentially zero — all memory is gone. Fractional differentiation with $d = 0.4$ also rejects the unit root, but the ACF at lag 1 is 0.69: most of the long-range information survives.

**Key Insight**

This is the core finding of AFML Ch.5: there exists a $d^\star \in (0, 1)$ that simultaneously satisfies the stationarity requirement of statistical inference *and* the memory requirement of predictive ML features. Any $d < d^\star$ is non-stationary; any $d > d^\star$ throws away more memory than necessary.

## 7.8. Exercises

1. **Partial autocorrelation**: Implement the PACF via the Yule-Walker equations. For an AR($p$) process, PACF should cut off after lag $p$. Verify on simulated AR(2) data with $\phi_1 = 0.5$, $\phi_2 = -0.3$.
2. **KPSS test**: The KPSS test has the opposite null hypothesis to ADF — $H_0$ is stationarity. Implement it and compare the conclusions on white noise, random walk, and fractionally differenced series. Where do the two tests disagree?
3. **Real returns**: Generate a long GBM price path and find the minimum $d$ for stationarity. Repeat for different volatilities $\sigma \in \{0.1, 0.2, 0.4\}$. Does $\sigma$ affect $d^\star$?
4. **Expanding-window frac_diff**: The fixed-width window drops the first $K - 1$ observations. Implement an expanding-window variant that uses all available history for the early points (weights are truncated, not zero). What is the tradeoff versus fixed-width?
5. **Lag selection for ADF**: The AIC and BIC criteria pick the number of lagged differences automatically. Implement AIC-based lag selection and check whether the stationarity conclusion changes for your test series.

## 7.9. Key Takeaways

- **Stationarity** is required for valid statistical inference; most price series are non-stationary
- **OLS** fits the normal equations via Gaussian elimination with partial pivoting
- **ACF** reveals memory structure: random walks have slowly decaying ACF, white noise has zero ACF after lag 0
- **ADF test** rejects the unit-root null when the t-statistic is below the MacKinnon critical value ($-2.86$ at 5%)
- **Fractional differentiation** achieves stationarity while preserving memory—the sweet spot for ML features
- **Minimum** $d$: binary search finds the smallest differencing order that passes the ADF test

**Next chapter**: Volatility models—EWMA, ARCH, and GARCH for time-varying conditional variance.

# 8. Volatility Models: EWMA, ARCH, and GARCH

**Companion Code:**

quant-vol Code: `crates/quant-vol/`
Dependencies: `quant-core`, `thiserror`
Run: `cargo run -p quant-vol -example vol_demo`

## 8.1. Learning Objectives

Financial returns are not Gaussian — their variance changes over time, and large moves cluster. A constant-volatility model underestimates risk in turbulent periods and overestimates it in calm ones. After completing this chapter you can:

- Describe the stylized facts of financial return volatility: clustering, fat tails, mean reversion, and slow decay of autocorrelation in squared returns
- Derive the exponentially weighted moving average (EWMA) recursion and justify the RiskMetrics $\lambda = 0.94$ choice for daily data
- Specify the ARCH($q$) model of Engle (1982) and fit it by Gaussian maximum likelihood with a hand-rolled optimiser
- Specify the GARCH($p, q$) model of Bollerslev (1986), state the covariance-stationarity condition, and interpret persistence
- Compute the long-run variance and the half-life of volatility shocks
- Fit GARCH(1,1) to simulated and clustered-volatility data and show that it beats constant-volatility models in log-likelihood

**Key Insight**

Volatility clustering — large moves follow large moves, calm follows calm — is the single most robust empirical regularity in financial returns. A Gaussian return with constant variance cannot reproduce it. ARCH and GARCH make the conditional variance a function of past squared returns, so a large shock raises the variance of the next period. This one feature explains most of the improvement of GARCH over Gaussian models for risk forecasting.

## 8.2. Stylized Facts of Volatility

**Volatility clustering**
$\mathrm{Corr}(r_t^2, r_{t-k}^2) > 0$ for many lags $k$, while $\mathrm{Corr}(r_t, r_{t-k}) \approx 0$. The ACF of returns is flat; the ACF of *squared* returns decays slowly.

Empirical daily returns exhibit four robust features that any realistic volatility model must reproduce:

1. **Volatility clustering.** The autocorrelation of $r_t^2$ is positive and decays slowly — much more slowly than the autocorrelation of $r_t$, which is essentially zero after one or two lags.
2. **Fat tails.** The unconditional distribution of returns has excess kurtosis: large moves occur more often than a Gaussian with the same variance would predict.
3. **Mean reversion.** Volatility cannot grow without bound; it reverts toward a long-run level.

4. **Asymmetry (leverage).** Negative returns tend to raise volatility more than positive returns of the same magnitude. Plain ARCH/GARCH do not capture this; GJR-GARCH and EGARCH do.

## 8.3. EWMA: Exponentially Weighted Moving Average

**EWMA recursion**
$\sigma_t^2 = \lambda\sigma_{t-1}^2 + (1-\lambda)r_{t-1}^2$
with $\sigma_0^2 = r_0^2$. RiskMetrics uses $\lambda = 0.94$ for daily data and $\lambda = 0.97$ for monthly.

The simplest model that responds to clustering is the exponentially weighted moving average:

$$\sigma_t^2 = \lambda\sigma_{t-1}^2 + (1-\lambda)r_{t-1}^2, \qquad \lambda \in [0,1].$$

Each period's variance is a convex combination of the previous variance and the previous squared return. The parameter $\lambda$ controls the memory: a value close to 1 makes the variance slow-moving; a value close to 0 makes it track the last squared return.

### 8.3.1. Boundary cases

- $\lambda = 0$: $\sigma_t^2 = r_{t-1}^2$ — the variance is just the lagged squared return. No smoothing.
- $\lambda = 1$: $\sigma_t^2 = \sigma_{t-1}^2$ — the variance is constant at its initial value $\sigma_0^2 = r_0^2$. No adaptation.
- $\lambda = 0.94$ (RiskMetrics daily): a single shock's influence decays geometrically. After $k$ periods without further shocks, the variance is multiplied by $\lambda^k$.

### 8.3.2. Unrolling the recursion

Substituting repeatedly:

$$\sigma_t^2 = (1-\lambda)\sum_{k=1}^{t}\lambda^{k-1}r_{t-k}^2 + \lambda^t\sigma_0^2.$$

The weights on past squared returns decay geometrically with rate $\lambda$. For long series the $\lambda^t\sigma_0^2$ term vanishes and the weights sum to $(1-\lambda)/(1-\lambda) = 1$, so $\sigma_t^2$ is a proper weighted average of past squared returns.

```rust
pub fn ewma_vol(returns: &[f64], lambda: f64) -> Result<Vec<f64>, VolError> {
    if !(0.0..=1.0).contains(&lambda) {
        return Err(VolError::InvalidParam(...));
    }
    if returns.is_empty() {
        return Err(VolError::InsufficientData { required: 1, actual: 0 });
    }
    let mut sigma2 = vec![0.0_f64; returns.len()];
    sigma2[0] = returns[0] * returns[0];
    for t in 1..returns.len() {
        sigma2[t] = lambda * sigma2[t - 1]
                  + (1.0 - lambda) * returns[t - 1] * returns[t - 1];
    }
    Ok(sigma2)
}
```

## 8.4. ARCH: Autoregressive Conditional Heteroskedasticity

**ARCH($q$) of Engle (1982)**
$\sigma_t^2 = \omega + \sum_{i=1}^{q} \alpha_i r_{t-i}^2$
with $\omega > 0$, $\alpha_i \geq 0$, and $\sum \alpha_i < 1$ for stationarity.

Engle's ARCH model makes the conditional variance an explicit linear function of the past $q$ squared returns:

$$\sigma_t^2 = \omega + \sum_{i=1}^{q} \alpha_i r_{t-i}^2 .$$

The coefficients are constrained to be non-negative (otherwise the variance could go negative) and $\sum \alpha_i < 1$ ensures the unconditional variance $\omega/(1 - \sum \alpha_i)$ is finite.

ARCH captures clustering directly: a large $r_{t-1}^2$ raises $\sigma_t^2$ by $\alpha_1 r_{t-1}^2$, which in turn makes a large $r_t$ more likely, which raises $\sigma_{t+1}^2$, and so on. The effect dies out geometrically when shocks stop arriving.

### 8.4.1. Gaussian log-likelihood

Assuming $r_t \mid \mathcal{F}_{t-1} \sim \mathcal{N}(0, \sigma_t^2)$, the conditional density is

$$f(r_t \mid \mathcal{F}_{t-1}) = \frac{1}{\sqrt{2\pi\sigma_t^2}} \exp\!\left(-\frac{r_t^2}{2\sigma_t^2}\right),$$

so the Gaussian log-likelihood is

$$\mathcal{L}(\theta) = -\frac{1}{2} \sum_t \left[ \log(2\pi) + \log \sigma_t^2(\theta) + \frac{r_t^2}{\sigma_t^2(\theta)} \right].$$

The MLE maximises $\mathcal{L}$ over $\theta = (\omega, \alpha_1, \ldots, \alpha_q)$. Because $\sigma_t^2$ is a deterministic function of $\theta$ and the data, the score and Hessian can be derived in closed form, but we will not need them: a derivative-free optimiser is enough.

**Warning**

The Gaussian log-likelihood is *not* bounded above by zero. When $\sigma_t^2$ is small and $r_t^2/\sigma_t^2$ is also small, each term $-0.5[\log(2\pi) + \log \sigma_t^2 + r_t^2/\sigma_t^2]$ can be positive. The only universal constraint is that $\mathcal{L}$ is finite when all $\sigma_t^2 > 0$.

```rust
#[derive(Debug, Clone)]
pub struct ArchModel {
    pub omega: f64,       // Constant term (omega > 0)
    pub alphas: Vec<f64>, // ARCH coefficients (alpha_i >= 0)
}

impl ArchModel {
    pub fn conditional_variances(&self, returns: &[f64]) -> Vec<f64> {
        let n = returns.len();
        let q = self.alphas.len();
        let mut sigma2 = vec![0.0_f64; n];
        for t in 0..n {
            let mut s = self.omega;
            for i in 0..q {
                if t > i {
                    s += self.alphas[i] * returns[t - 1 - i].powi(2);
                }
            }
            sigma2[t] = s.max(1e-300);
        }
```

```
        sigma2
    }

    pub fn log_likelihood(&self, returns: &[f64]) -> f64 {
        let sigma2 = self.conditional_variances(returns);
        let mut ll = 0.0_f64;
        for (t, &r) in returns.iter().enumerate() {
            let s2 = sigma2[t];
            ll -= 0.5 * (LN_2PI + s2.ln() + r * r / s2);
        }
        ll
    }
}
```

## 8.5. GARCH: Generalised ARCH

**GARCH($p, q$) of Bollerslev (1986)**
$\sigma_t^2 = \omega + \sum_{i=1}^{q} \alpha_i r_{t-i}^2 + \sum_{j=1}^{p} \beta_j \sigma_{t-j}^2$ with $\omega > 0, \alpha_i, \beta_j \geq 0$, and $\sum \alpha_i + \sum \beta_j < 1$ for covariance stationarity.

Bollerslev's GARCH adds lagged conditional variances to the ARCH recursion:

$$\sigma_t^2 = \omega + \sum_{i=1}^{q} \alpha_i r_{t-i}^2 + \sum_{j=1}^{p} \beta_j \sigma_{t-j}^2.$$

The GARCH terms $\beta_j \sigma_{t-j}^2$ propagate current volatility forward, so shocks have a longer, smoother effect than in ARCH. In practice GARCH(1,1) — one ARCH term and one GARCH term — is the workhorse: it fits most daily return series well with only three parameters.

```
#[derive(Debug, Clone)]
pub struct GarchModel {
    pub omega: f64,       // Constant term (omega > 0)
    pub alphas: Vec<f64>, // ARCH coefficients, length q
    pub betas: Vec<f64>,  // GARCH coefficients, length p
}

impl GarchModel {
    pub fn conditional_variances(&self, returns: &[f64]) -> Vec<f64> {
        let n = returns.len();
        let (q, p) = (self.alphas.len(), self.betas.len());
        let warmup = p.max(q);
        let init_var = sample_variance(returns);
        let mut sigma2 = vec![init_var; n];
        for t in warmup..n {
            let mut s = self.omega;
            for i in 0..q {
                s += self.alphas[i] * returns[t - 1 - i].powi(2);
            }
            for j in 0..p {
                s += self.betas[j] * sigma2[t - 1 - j];
            }
            sigma2[t] = s.clamp(1e-300, 1e15);
        }
        sigma2
    }

    pub fn persistence(&self) -> f64 {
        self.alphas.iter().sum::<f64>() + self.betas.iter().sum::<f64>()
    }

    pub fn long_run_variance(&self) -> f64 {
        let pers = self.persistence();
        if pers < 1.0 { self.omega / (1.0 - pers) } else { f64::INFINITY }
    }

    pub fn half_life(&self) -> f64 {
        let pers = self.persistence();
        if pers > 0.0 && pers < 1.0 { 0.5_f64.ln() / pers.ln() }
        else { f64::INFINITY }
    }
}
```

### 8.5.1. Persistence and stationarity

**Persistence**
$\rho = \sum_i \alpha_i + \sum_j \beta_j$.
Covariance stationary iff $\rho < 1$.
Long-run variance: $\bar{\sigma}^2 = \omega/(1-\rho)$.
Half-life of shocks: $\log(0.5)/\log \rho$.

The sum $\rho = \sum \alpha_i + \sum \beta_j$ is the *persistence*. Taking unconditional expectations of the GARCH recursion and solving for the fixed point gives the long-run variance:

$$\bar{\sigma}^2 = \frac{\omega}{1-\rho}, \qquad \rho < 1.$$

When $\rho \geq 1$ the model is non-stationary: shocks accumulate rather than decay, and the unconditional variance is infinite. The IGARCH boundary $\rho = 1$ is a random walk in variance and should be avoided for fitting.

### 8.5.2. Half-life of shocks

Starting from $\sigma_0^2 = \bar{\sigma}^2 + \delta$ (a perturbation $\delta$ above the long run), the expected deviation from $\bar{\sigma}^2$ after $k$ steps is $\rho^k \delta$. The half-life is the $k$ at which $\rho^k = 0.5$:

$$k_{1/2} = \frac{\log 0.5}{\log \rho}.$$

For a typical daily GARCH(1,1) with $\rho = 0.99$, the half-life is about 69 trading days — roughly three months. High persistence is the norm for daily financial data.

### 8.5.3. Forecasting

The multi-step forecast follows the AR(1)-in-variance recursion

$$\mathbb{E}[\sigma_{t+h}^2 \mid \mathcal{F}_t] = \omega + \rho\, \mathbb{E}[\sigma_{t+h-1}^2 \mid \mathcal{F}_t],$$

which reverts geometrically to $\bar{\sigma}^2$:

$$\mathbb{E}[\sigma_{t+h}^2 \mid \mathcal{F}_t] = \bar{\sigma}^2 + \rho^h \left(\sigma_t^2 - \bar{\sigma}^2\right).$$

The forecast is data-conditioned when the last fitted $\sigma_t^2$ is used as the seed, or unconditional when seeded from $\bar{\sigma}^2$.

## 8.6. Maximum Likelihood by Hand-Rolled Optimisation

**Constrained MLE**
Optimise an unconstrained $\theta$ and map to the constrained space: $\omega = e^{t_0}$, $\alpha_i = \text{sigmoid}(t_i) \cdot 0.49/q$, $\beta_j = \text{sigmoid}(t_j) \cdot (0.998 - \sum \alpha)/p$. This guarantees $\omega > 0$, $\alpha_i, \beta_j \geq 0$, and $\rho < 0.998$.

We fit both ARCH and GARCH by maximising the Gaussian log-likelihood with a derivative-free optimiser. The constraint $\rho < 1$ is enforced not by penalising the objective but by *parameter transformation*: we optimise an unconstrained vector $t$ and map it to the constrained space via $\omega = e^{t_0}$, $\alpha_i = \text{sigmoid}(t_i) \cdot 0.49/q$, and $\beta_j = \text{sigmoid}(t_j) \cdot (0.998 - \sum \alpha)/p$. The 0.998 cap on persistence prevents the optimiser from wandering onto the IGARCH boundary, where the likelihood becomes flat and numerically ill-conditioned.

### 8.6.1. Nelder-Mead simplex

The Nelder-Mead algorithm maintains a simplex of $n + 1$ points in $n$-dimensional parameter space and cycles through four operations:

**Reflection** ($\alpha = 1$) Mirror the worst vertex through the centroid of the others.

**Expansion** ($\gamma = 2$) If the reflection is better than the best, stretch further in the same direction.

**Contraction** ($\rho = 0.5$) If the reflection is worse than the second-worst, contract toward the centroid.

**Shrink** ($\sigma = 0.5$) If contraction fails, pull all vertices halfway toward the best.

Convergence is declared when both the simplex diameter and the spread of function values fall below a tolerance.

### 8.6.2. Coordinate ascent with multi-start

**Why coordinate ascent?**

For 2–4 parameter MLE problems, Nelder-Mead can degenerate near the constraint boundary. Coordinate ascent with step halving tracks the $\alpha$–$\beta$ ridge cleanly, and multi-start from five perturbed initial guesses escapes the occasional local plateau.

For the low-dimensional MLE problems here (ARCH(1) has 2 parameters, GARCH(1,1) has 3), a simpler and more robust alternative is coordinate ascent: cycle through each coordinate, try a positive and a negative step, accept any improvement, and halve the step when no coordinate improves. Convergence is declared when the step falls below the tolerance. We wrap this in a *multi-start* that runs coordinate ascent from the initial guess plus five perturbations $[+0.1, -0.1, +0.3, -0.3, +0.5]$ and keeps the best result.

```rust
/// Coordinate-wise hill climbing with adaptive step size.
pub fn coordinate_ascent<F>(f: F, x0: &[f64], step: f64, max_iter: usize, tol: f64)
    -> OptResult
where F: Fn(&[f64]) -> f64
{
    let mut x = x0.to_vec();
    let mut best_f = f(&x);
    let mut step = step;
    for _ in 0..max_iter {
        let mut improved = false;
        for i in 0..x.len() {
            let old = x[i];
            x[i] += step;  // try positive step
            if f(&x) > best_f { best_f = f(&x); improved = true; continue; }
            x[i] = old - step;  // try negative step
            if f(&x) > best_f { best_f = f(&x); improved = true; continue; }
            x[i] = old;  // restore
        }
        if !improved { step *= 0.5; if step < tol { break; } }
    }
    OptResult { x, f: best_f, .. }
}

/// Multi-start: run coordinate_ascent from x0 plus perturbations.
pub fn multistart<F>(f: F, x0: &[f64], step: f64, max_iter: usize, tol: f64)
    -> OptResult
where F: Fn(&[f64]) -> f64
{
    let mut best = coordinate_ascent(&f, x0, step, max_iter, tol);
    for &perturb in &[0.1, -0.1, 0.3, -0.3, 0.5] {
        let x_alt: Vec<f64> = x0.iter().map(|&v| v + perturb).collect();
        let result = coordinate_ascent(&f, &x_alt, step, max_iter, tol);
        if result.f > best.f { best = result; }
    }
    best
}
```

All three optimisers *maximise* the objective — the MLE closures pass `model.log_likelihood(returns)` directly, without negation.

## 8.7. Implementation in Rust

**Why three models in one crate?**

EWMA, ARCH, and GARCH form a progression: EWMA is one parameter ($\lambda$); ARCH(1) adds a lag coefficient ($\omega, \alpha$); GARCH(1,1) adds the lagged-variance coefficient ($\beta$). Keeping them in one crate lets you compare log-likelihoods and in-sample fit on the same return series with the same MLE machinery.

```rust
use quant_vol::{ewma_vol, ArchModel, GarchModel};

let returns = vec![0.01, -0.02, 0.015, -0.01, 0.02, -0.005, 0.01, -0.03];
```

```rust

// EWMA (RiskMetrics lambda = 0.94).
let sigma2 = ewma_vol(&returns, 0.94).unwrap();

// ARCH(1) fit by Gaussian MLE.
let arch = ArchModel::fit(&returns, 1).unwrap();
let arch_ll = arch.log_likelihood(&returns);

// GARCH(1,1) fit by Gaussian MLE.
let garch = GarchModel::fit(&returns, 1, 1).unwrap();
println!("persistence = {:.4}", garch.persistence());
println!("long-run var = {:.6}", garch.long_run_variance());
println!("half-life = {:.1} periods", garch.half_life());
```

The constrained parameterisation for GARCH(1,1) is:

```rust
fn from_transformed(params: &[f64], p: usize, q: usize) -> Self {
    let omega = params[0].clamp(-30.0, 30.0).exp();
    let alpha_max = 0.49;
    let alphas: Vec<f64> = (0..q)
        .map(|i| sigmoid(params[1 + i].clamp(-30.0, 30.0)) * alpha_max / q as f64)
        .collect();
    let alpha_sum: f64 = alphas.iter().sum();
    let beta_max = (0.998 - alpha_sum).max(1e-6);
    let betas: Vec<f64> = (0..p)
        .map(|j| sigmoid(params[1 + q + j].clamp(-30.0, 30.0)) * beta_max / p as f64)
        .collect();
    GarchModel { omega, alphas, betas }
}
```

The initial guess targets $\omega \approx 0.05\hat{\sigma}^2$, $\alpha \approx 0.05$, $\beta \approx 0.90$, which are typical for daily financial returns.

## 8.8. Volatility Clustering in Action

```
$ cargo run -p quant-vol --example vol_clustering
Regimes: calm(1000) -> shock(500) -> calm(500)
  Calm sigma^2  = 0.000100
  Shock sigma^2 = 0.010000

Constant-volatility model:
  Sample variance = 0.002585
  Log-likelihood  = 3120.38

GARCH(1,1):
  omega = 0.000004
  alpha = 0.145513
  beta  = 0.849413
  Persistence = 0.9949
  Log-likelihood = 5072.78

  -> GARCH beats constant-vol by 1952.4 LL points
```

The constant-volatility model uses a single sample variance for every period, so it over-estimates variance during the calm regime (producing too-small densities for the small returns that actually occur) and under-estimates it during the shock regime (producing too-small densities for the large returns that actually occur). GARCH tracks the regime changes through its conditional variance recursion and wins by nearly 2000 log-likelihood points.

**Key Insight**

The log-likelihood gain from GARCH over constant volatility is a direct quantification of how much volatility clustering matters. A gain of 1952 LL points across 2000 observations is roughly 0.976 nats per observation — a substantial amount of explained structure that a Gaussian-with-constant- variance model leaves on the table.

## 8.9. Parameter Recovery

When the data are generated by a known GARCH(1,1), the MLE should recover the true parameters up to sampling error. With 2000 observations from $\omega = 0.01$, $\alpha = 0.08$, $\beta = 0.90$ (true persistence 0.98, true long-run variance 0.5):

```
$ cargo run -p quant-vol --example vol_demo
DGP: GARCH(1,1) with omega=0.01, alpha=0.08, beta=0.90
True persistence:     0.9800
True long-run var:     0.500000

GARCH(1,1):
  omega = 0.0111
  alpha = 0.0910
  beta  = 0.8934
  Persistence = 0.9844  (true 0.9800)
  Long-run var = 0.7150  (true 0.500000)
  Half-life    = 44.1 periods
  Log-likelihood = -2298.62
```

The fitted persistence is within 0.005 of the truth. The long-run variance is harder to pin down because it divides $\omega$ by $1 - \rho$, so a small error in $\rho$ near 1 amplifies. This is a general feature of high-persistence GARCH fits: $\omega$ and $\rho$ are well-identified, but $\bar{\sigma}^2 = \omega/(1 - \rho)$ has a much wider confidence band.

## 8.10. Exercises

1. **EGARCH**: Exponential GARCH models $\log \sigma_t^2$ as a linear function of past squared returns and their signs, which captures the leverage effect (negative returns raise volatility more than positive ones). Implement EGARCH(1,1) and compare its log-likelihood to plain GARCH(1,1) on a series with a visible leverage effect.
2. **GJR-GARCH**: The Glosten-Jagannathan-Runkle model adds an indicator term that activates when the lagged return is negative: $\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \gamma \mathbf{1}_{r_{t-1}<0} r_{t-1}^2 + \beta \sigma_{t-1}^2$. Implement it and test on equity index returns. Is $\gamma$ significantly positive?
3. **Forecast evaluation**: Fit GARCH(1,1) on the first 80% of a return series, forecast the conditional variance for the remaining 20%, and compare the forecast bands to the realised squared returns. Compute the mean squared error of the variance forecast and compare to the EWMA forecast.
4. **Student-$t$ innovations**: Replace the Gaussian likelihood with a Student-$t$ likelihood with $\nu$ degrees of freedom (fit $\nu$ jointly with $(\omega, \alpha, \beta)$). Does the $t$-GARCH fit outperform the Gaussian GARCH on a heavy-tailed series? What is the fitted $\nu$?
5. **Long-memory FIGARCH**: The FIGARCH model replaces the geometric decay of shocks with a hyperbolic decay by fractionally differencing the variance recursion. Implement FIGARCH(1, $d$, 1) and compare its half-life to plain GARCH(1,1) on a long daily series. How much longer is the effective memory?

## 8.11. Key Takeaways

- Financial returns show volatility clustering: squared returns are positively autocorrelated for many lags. Constant-volatility models cannot reproduce this.

- EWMA is the simplest fix: a one-parameter geometrically weighted average of past squared returns. RiskMetrics uses $\lambda = 0.94$ for daily data.
- ARCH($q$) makes the conditional variance a linear function of the past $q$ squared returns. It captures clustering but needs many lags to match the slow decay of the squared-return ACF.
- GARCH($p, q$) adds lagged conditional variances, giving shocks a long, smooth effect with only a few parameters. GARCH(1,1) is the standard workhorse.
- Persistence $\rho = \sum \alpha + \sum \beta$ must be $< 1$ for covariance stationarity. The long-run variance is $\omega/(1 - \rho)$ and the half-life of shocks is $\log 0.5/\log \rho$.
- Fitting by Gaussian MLE with a hand-rolled optimiser is feasible because the likelihood is a deterministic function of the parameters and the data. Parameter transformation (exp, sigmoid) enforces positivity and stationarity without penalising the objective.

**Next chapter**: Stochastic processes—Brownian motion, Poisson processes, and Monte Carlo simulation for derivative pricing.

# 9. Stochastic Processes and Monte Carlo

**Companion Code:**

quant-stochastic Code: `crates/quant-stochastic/`
Dependencies: `quant-core`, `thiserror`
Run: `cargo run -p quant-stochastic -example mc_pricing`

## 9.1. Learning Objectives

Asset prices are often modelled as stochastic processes — random functions of time. The Brownian motion is the continuous-time limit of a random walk and the building block of modern quantitative finance. From it we derive geometric Brownian motion (the Black-Scholes asset model), the Poisson process (jump models), and the Monte Carlo method (the generic pricing engine). After completing this chapter you can:

- Simulate standard Brownian motion from independent Gaussian increments and verify its quadratic variation property
- Simulate geometric Brownian motion via the exact closed-form solution to the SDE $dS_t = \mu S_t\,dt + \sigma S_t\,dW_t$
- Simulate a Poisson process via exponential interarrival times and build a Merton jump-diffusion on top of GBM
- Price a European option by Monte Carlo and report a standard error
- State the $1/\sqrt{N}$ convergence law and demonstrate it numerically
- Reduce estimator variance with antithetic variates and verify put-call parity on simulated prices

**Key Insight**

Monte Carlo pricing converges to the analytical Black-Scholes price as $N \to \infty$, with standard error decreasing as $1/\sqrt{N}$. This is the law of large numbers in action: quadruple the paths, halve the error. No other numerical method gives such a clean, dimension-independent error bound — which is why Monte Carlo is the workhorse of derivatives pricing in high dimensions.

## 9.2. Brownian Motion

**Brownian motion**
$W_0 = 0$, $W_{t+\Delta t} - W_t \sim \mathcal{N}(0, \Delta t)$, independent increments. Quadratic variation $[W]_t = t$.

A standard Brownian motion $\{W_t\}$ is the continuous-time limit of a random walk with independent Gaussian increments:

$$W_0 = 0, \qquad W_{t+\Delta t} = W_t + \sqrt{\Delta t}\,Z, \quad Z \sim \mathcal{N}(0,1).$$

Two properties define it:

1. **Independent increments.** $W_{t+s} - W_t$ is independent of $\{W_u : u \le t\}$.
2. **Quadratic variation.** The sum of squared increments $\sum (W_{t_k} - W_{t_{k-1}})^2$ converges in probability to $T$ as the partition refines — the defining property of Brownian motion that distinguishes it from smooth functions.

### 9.2.1. Quadratic Variation

For a Brownian motion sampled at spacing $\Delta t$, the realised quadratic variation

$$[W]_T^{(\Delta t)} = \sum_{k=1}^{n} (W_{k\Delta t} - W_{(k-1)\Delta t})^2$$

converges in probability to $T = n\Delta t$ as $\Delta t \to 0$. This is the empirical signature of Brownian motion: the path is continuous but nowhere differentiable, and its roughness is exactly $t$.

## 9.3. Geometric Brownian Motion

**GBM SDE**
$dS_t = \mu S_t\,dt + \sigma S_t\,dW_t$
Closed form: $S_T = S_0 \exp\big((\mu - \frac{1}{2}\sigma^2)T + \sigma W_T\big)$

The Black-Scholes asset model is geometric Brownian motion, the exponential of Brownian motion with drift:

$$dS_t = \mu S_t\,dt + \sigma S_t\,dW_t.$$

Itô's lemma gives the exact solution

$$S_T = S_0 \exp\big(\big(\mu - \tfrac{1}{2}\sigma^2\big)\,T + \sigma W_T\big)\,.$$

Two facts follow immediately:

- $\log(S_T/S_0) \sim \mathcal{N}((\mu - \frac{1}{2}\sigma^2)T,\ \sigma^2 T)$.
- $\mathbb{E}[S_T] = S_0\,e^{\mu T}$ (the log-normal mean).

The companion crate uses the closed form for each step, so the terminal distribution is exact for any $\Delta t$ — no Euler discretisation error.

## 9.4. Poisson Process and Jump-Diffusion

**Poisson process**
$N(t) \sim$ Poisson$(\lambda t)$. Interarrival times $\sim$ Exp$(\lambda)$, mean $1/\lambda$.

The Poisson process counts rare events: $N(t)$ is the number of events in $[0, t]$, with $\mathbb{E}[N(t)] = \lambda t$. Interarrival times are exponential with rate $\lambda$. The inverse-CDF sampler draws $X = -\ln(1-U)/\lambda$ with $U \sim$ Uniform$(0, 1)$.

### 9.4.1. Merton Jump-Diffusion

**Merton model**
$dS_t = \mu S_t\,dt + \sigma S_t\,dW_t + (J-1)S_t\,dN_t$
Between jumps: GBM. At a jump: $S \to S \cdot J$.

Robert Merton's jump-diffusion superimposes Poisson jumps on GBM:

$$dS_t = \mu S_t\,dt + \sigma S_t\,dW_t + (J-1)S_t\,dN_t,$$

where $N_t$ is a Poisson process with rate $\lambda$ and $J$ is the jump multiplier. Between jumps the process follows GBM; at each event time the price is multiplied by $J$. The companion crate uses $J = e^{\mu_J}$ (a deterministic log-normal jump, with the randomness in the timing). With $\lambda = 0$ there are no jumps and the process reduces exactly to GBM.

## 9.5. The Monte Carlo Method

**MC estimator**
$\hat{C} = e^{-rT}\frac{1}{N}\sum_{i=1}^{N}\max(S_T^{(i)}-K,0)$
$\mathrm{SE}(\hat{C}) = e^{-rT}\frac{\mathrm{std(payoff)}}{\sqrt{N}}$

Monte Carlo prices a derivative by simulating many paths and averaging the discounted payoff. For a European call under risk-neutral GBM:

$$\hat{C} = e^{-rT}\frac{1}{N}\sum_{i=1}^{N}\max(S_T^{(i)}-K,0), \qquad S_T^{(i)} = S_0\exp\!\left(\left(r-\tfrac{1}{2}\sigma^2\right)T+\sigma\sqrt{T}\,Z_i\right).$$

Each path is a single normal draw — the terminal $Z_i$ — so the estimator is minimal-variance. The standard error of the mean is

$$\mathrm{SE}(\hat{C}) = e^{-rT}\frac{s}{\sqrt{N}}, \qquad s^2 = \frac{1}{N-1}\sum_i(\text{payoff}_i-\bar{\text{payoff}})^2.$$

### 9.5.1. The $1/\sqrt{N}$ Law

The standard error shrinks as $1/\sqrt{N}$: quadruple the paths, halve the error. This is slow — to reduce the error by 10× one needs 100× the paths — but it is *dimension-independent*. Deterministic quadrature suffers the curse of dimensionality; Monte Carlo does not. For a 50-dimensional exotic option, Monte Carlo's error is still $O(1/\sqrt{N})$.

### 9.5.2. Antithetic Variates

**Antithetic variates**
Pair $Z$ and $-Z$. For monotone payoffs the pair average has lower variance than two independent samples.

Variance reduction lets us extract more precision per random draw. *Antithetic variates* pair each $Z$ with $-Z$:

$$\hat{C}_{\text{anti}} = e^{-rT}\frac{1}{2N}\sum_{i=1}^{N}\left[\max(S_T(Z_i)-K,0)+\max(S_T(-Z_i)-K,0)\right].$$

The two payoffs are perfectly negatively correlated (the call payoff is monotone increasing in $Z$), so the variance of the pair average is lower than the variance of two independent samples. The cost is one extra payoff evaluation per draw — negligible versus the cost of the normal draw.

## 9.6. Black-Scholes: The Analytical Benchmark

**Black-Scholes call**
$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2)$
$d_1 = \frac{\ln(S_0/K)+(r+\frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}},\ d_2 = d_1-\sigma\sqrt{T}$

The Black-Scholes formula gives the closed-form European call price:

$$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2).$$

The normal CDF $\Phi$ is computed via the Abramowitz-Stegun (1964) formula 7.1.26 rational approximation of erf, with $\Phi(x) = \frac{1}{2}(1+\mathrm{erf}(x/\sqrt{2}))$. Maximum error $< 7.5\times10^{-8}$ — far smaller than Monte Carlo's statistical error for practical $N$.

## 9.7. Implementation in Rust

### 9.7.1. Brownian Motion

```rust
pub fn brownian_motion<R: Rng + ?Sized>(n: usize, dt: f64, rng: &mut R) -> Vec<f64> {
    let normal = Normal::standard();
    let sqrt_dt = dt.sqrt();
    let mut path = Vec::with_capacity(n + 1);
    path.push(0.0); // W_0 = 0
    let mut w = 0.0_f64;
    for _ in 0..n {
        let z = normal.sample(rng);
        w += sqrt_dt * z;
        path.push(w);
    }
    path
}
```

### 9.7.2. Monte Carlo Call

```rust
pub fn mc_call<R: Rng + ?Sized>(
    s0: f64, k: f64, r: f64, sigma: f64, t: f64,
    n_paths: usize, rng: &mut R,
) -> Result<McResult, StochError> {
    validate_mc_inputs(s0, k, r, sigma, t, n_paths)?;
    let normal = Normal::standard();
    let drift = (r - 0.5 * sigma * sigma) * t;
    let diffusion = sigma * t.sqrt();
    let discount = (-r * t).exp();
    let mut payoffs = Vec::with_capacity(n_paths);
    for _ in 0..n_paths {
        let z = normal.sample(rng);
        let s_t = s0 * (drift + diffusion * z).exp();
        payoffs.push((s_t - k).max(0.0));
    }
    Ok(reduce(&payoffs, discount))
}
```

## 9.8. Results and Analysis

### 9.8.1. Monte Carlo Convergence

Pricing an at-the-money call ($S_0 = K = 100$, $r = 0.05$, $\sigma = 0.2$, $T = 1$) against the Black-Scholes benchmark $C_{\text{BS}} = 10.4506$:

| $N$ | MC price | BS price | SE |
|---|---|---|---|
| 100 | 8.9770 | 10.4506 | 1.4320 |
| 1000 | 10.3204 | 10.4506 | 0.4641 |
| 10000 | 10.5059 | 10.4506 | 0.1469 |
| 100000 | 10.4194 | 10.4506 | 0.0465 |

The estimate converges to Black-Scholes and the standard error shrinks as $1/\sqrt{N}$.

### 9.8.2. The $1/\sqrt{N}$ Law

| $N$ | SE | ratio vs $N = 10000$ |
|---|---|---|
| 10000 | 0.1479 | 1.00 |
| 40000 | 0.0732 | 2.02× smaller |
| 160000 | 0.0367 | 4.03× smaller |

Quadrupling paths halves the SE (2.02×); 16× paths gives 4.03× smaller SE — the $1/\sqrt{N}$ law made exact.

### 9.8.3. Antithetic Variance Reduction

With 50000 normal draws each:

| Method | price | SE | payoffs |
|---|---|---|---|
| Plain MC | 10.4191 | 0.0656 | 50000 |
| Antithetic | 10.4962 | 0.0467 | 100000 |

Antithetic variates reduce the standard error by **28.8%** per normal draw. The pair average exploits the negative correlation between the $Z$ and $-Z$ payoffs.

### 9.8.4. Quadratic Variation

| $n$ | QV | $T$ | rel. error |
|---|---|---|---|
| 100 | 0.346 | 0.397 | 12.8% |
| 1000 | 4.095 | 3.968 | 3.2% |
| 10000 | 40.509 | 39.683 | 2.1% |
| 100000 | 399.560 | 396.825 | 0.69% |

The realised quadratic variation converges to $T$ as the partition refines — the defining property of Brownian motion.

### 9.8.5. GBM Terminal Distribution

With 100000 paths ($S_0 = 100$, $\mu = 0.05$, $\sigma = 0.2$, $T = 1$):

$$\mathbb{E}[S_T] = 105.07 \quad \text{vs analytical } S_0 e^{\mu T} = 105.13.$$

The 5th–95th percentile band is $[74.2, 143.2]$ — the log-normal right skew that produces fat tails in returns.

**Key Insight**

The convergence table is the law of large numbers made visible. At $N = 100$ the estimate is off by 14%; at $N = 100000$ it is off by 0.03%. The standard error shrinks by exactly $\sqrt{10}$ for each 10× increase in paths — the $1/\sqrt{N}$ law, the single most important rate in computational finance.

## 9.9. Exercises

1. **Control variates.** Use the analytical GBM mean $\mathbb{E}[S_T] = S_0 e^{rT}$ as a control variate: $\hat{C}_{\text{cv}} = \hat{C} - \beta(\hat{M} - \mathbb{E}[S_T])$, where $\hat{M}$ is the MC estimate of $\mathbb{E}[S_T]$ and $\beta = \text{Cov}(\text{payoff}, S_T)/\text{Var}(S_T)$. How much does the SE drop versus plain MC?
2. **Asian option.** Price an arithmetic-average Asian call $\max(\bar{S} - K, 0)$ with $\bar{S} = \frac{1}{n}\sum S_{t_k}$. This requires time-stepping (no closed form). Compare SE to the European call for the same $N$.

3. **Euler vs exact.** Discretise GBM by Euler ($S_{t+\Delta t} = S_t + \mu S_t \Delta t + \sigma S_t \sqrt{\Delta t} Z$) and compare the terminal distribution to the exact solution at $\Delta t = 1/252$. How large is the discretisation bias?
4. **Confidence intervals.** For $N = 10000$, compute the 95% confidence interval $\hat{C} \pm 1.96 \cdot \text{SE}$ and verify it covers the Black-Scholes price. Repeat 1000 times and check the coverage — is it close to 95%?

## 9.10. Key Takeaways

- Brownian motion is the continuous-time random walk: independent Gaussian increments, quadratic variation $[W]_t = t$. It is the building block of continuous-time finance.
- Geometric Brownian motion is the exponential of Brownian motion with drift, $S_T = S_0 \exp((\mu - \frac{1}{2}\sigma^2)T + \sigma W_T)$, and the Black-Scholes asset model. The closed form gives exact terminal distributions for any $\Delta t$ — no Euler bias.
- The Poisson process counts rare events via exponential interarrival times; the Merton jump-diffusion superimposes it on GBM and reduces to GBM when $\lambda = 0$.
- Monte Carlo prices any payoff by simulating many paths and averaging the discounted payoff. The standard error shrinks as $1/\sqrt{N}$ — slow but dimension-independent, the workhorse for high-dimensional derivatives.
- Antithetic variates pair $Z$ with $-Z$; for monotone payoffs the pair average cuts the SE per normal draw by roughly 30%.
- The Black-Scholes formula (normal CDF via Abramowitz-Stegun erf, error $< 7.5 \times 10^{-8}$) anchors the Monte Carlo estimator and verifies convergence to the truth.

**Next chapter**: Options pricing — Black-Scholes Greeks, implied volatility, and finite-difference sensitivity analysis.

# Derivatives and Portfolio Theory

# 10. Options Pricing: Black-Scholes and Beyond

**Companion Code:**

quant-options Code: `crates/quant-options/`
Dependencies: `quant-core`, `quant-stochastic`, `thiserror`
Run: `cargo run -p quant-options -example greeks`

## 10.1. Learning Objectives

This chapter builds the full Black-Scholes options toolkit on top of the closed-form pricing primitives from Chapter 9: the Greeks (Delta, Gamma, Vega, Theta, Rho) analytically and by finite difference, and implied volatility via Newton's method with a bisection fallback. After completing it you can:

- State the Black-Scholes PDE and the risk-neutral derivation
- Compute all five first- and second-order Greeks in closed form
- Recover the same Greeks by finite difference for any pricing model (not just Black-Scholes)
- Solve for implied volatility from a market price using Newton's method, and explain why a bisection fallback is needed for deep in/out-of-the-money options
- Reproduce a synthetic volatility smile and explain its shape

**Key Insight**

The Greeks are the partial derivatives of the option price with respect to its inputs. Delta-hedging (holding $-\Delta$ shares against a long call) eliminates first-order price risk; Gamma controls the residual hedging error. Implied volatility inverts the Black-Scholes formula to recover the market's view of future variance — the "volatility smile" is the single most studied phenomenon in derivatives.

## 10.2. The Black-Scholes PDE

**Black-Scholes PDE**

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0$$

The Black-Scholes model assumes the underlying follows geometric Brownian motion (see Chapter 9):

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t.$$

By holding a continuously rebalanced portfolio of the option and $-\Delta$ shares of stock, the residual risk is eliminated to first order. No-arbitrage then requires the portfolio to earn the risk-free rate, which yields the Black-Scholes PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0.$$

Solving this PDE with the terminal payoff $\max(S_T - K, 0)$ for a call (or $\max(K - S_T, 0)$ for a put) gives the closed forms

$$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2), \qquad P = Ke^{-rT}\Phi(-d_2) - S_0\Phi(-d_1),$$

with

$$d_1 = \frac{\ln(S_0/K) + (r + \tfrac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}.$$

These are the formulas we reuse from `quant-stochastic::blackscholes`.

## 10.3. The Greeks as Partial Derivatives

**Greeks**

$\Delta = \dfrac{\partial C}{\partial S}$ (spot)

$\Gamma = \dfrac{\partial^2 C}{\partial S^2}$ (curvature)

$\mathcal{V} = \dfrac{\partial C}{\partial \sigma}$ (vol)

$\Theta = \dfrac{\partial C}{\partial t}$ (time)

$\rho = \dfrac{\partial C}{\partial r}$ (rate)

The Greeks are the sensitivities of the option price to its inputs. They are the building blocks of hedging and risk management:

**Delta ($\Delta$)** $\partial C/\partial S$. The hedge ratio: holding $-\Delta$ shares against a long option removes first-order spot risk. A call has $\Delta \in (0, 1)$; a put $\Delta \in (-1, 0)$.

**Gamma ($\Gamma$)** $\partial^2 C/\partial S^2$. The rate of change of Delta. High Gamma means the hedge must rebalance frequently — Gamma is largest at-the-money and shrinks in the wings. Identical for calls and puts.

**Vega ($\mathcal{V}$)** $\partial C/\partial \sigma$. The sensitivity to volatility. Identical for calls and puts.

**Theta ($\Theta$)** $\partial C/\partial t$. The time decay. Long options lose value as expiry approaches, so $\Theta < 0$ for long calls and puts.

**Rho ($\rho$)** $\partial C/\partial r$. The sensitivity to the risk-free rate. Calls gain with $r$ (positive $\rho$); puts lose (negative $\rho$).

## 10.4. Analytical Greeks

**Analytical Greeks (call)**

$\Delta = \Phi(d_1)$

$\Gamma = \dfrac{\phi(d_1)}{S_0\sigma\sqrt{T}}$

$\mathcal{V} = S_0\phi(d_1)\sqrt{T}$

$\Theta = -\dfrac{S_0\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}\Phi(d_2)$

$\rho = KTe^{-rT}\Phi(d_2)$

Closed forms for the five Greeks follow from differentiating the Black-Scholes price. Let $\phi(x) = (2\pi)^{-1/2}e^{-x^2/2}$ be the standard normal PDF (the derivative of $\Phi$). We implement it inline in `quant-options::greeks`:

```rust
/// Standard normal PDF phi(x) = (1/sqrt(2 pi)) exp(-x^2 / 2).
pub fn normal_pdf(x: f64) -> f64 {
    const INV_SQRT_2PI: f64 = 0.398_942_280_401_432_7;
    INV_SQRT_2PI * (-0.5 * x * x).exp()
}
```

The analytical Greeks then drop out:

```rust
pub fn delta(s0, k, r, sigma, t, is_call: bool) -> f64 {
    let d1_val = d1(s0, k, r, sigma, t);
    let n_d1 = normal_cdf(d1_val);
    if is_call { n_d1 } else { n_d1 - 1.0 }
}

pub fn gamma(s0, k, r, sigma, t) -> f64 {
    let d1_val = d1(s0, k, r, sigma, t);
    normal_pdf(d1_val) / (s0 * sigma * t.sqrt())
}

pub fn vega(s0, k, r, sigma, t) -> f64 {
    let d1_val = d1(s0, k, r, sigma, t);
    s0 * normal_pdf(d1_val) * t.sqrt()
}
```

Theta and Rho differ between call and put; the call versions are

$$\Theta_c = -\frac{S_0\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}\Phi(d_2), \qquad \rho_c = KTe^{-rT}\Phi(d_2).$$

The put versions follow from put-call parity: $\Theta_p = \Theta_c + rKe^{-rT}$ and $\rho_p = -KTe^{-rT}\Phi(-d_2) = \rho_c - KTe^{-rT}$.

## 10.5. Numerical Greeks by Finite Difference

**Central difference**

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Error: $O(h^2)$

Analytical Greeks require a closed form. For models without a closed form (local volatility, Heston, Monte Carlo) the same sensitivities are recovered by finite difference. The central difference

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

is second-order accurate: the symmetric form cancels the $O(h)$ truncation term, leaving $O(h^2)$. For Gamma (a second derivative), the central second difference

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

is likewise $O(h^2)$.

```rust
pub fn delta_fd(s0, k, r, sigma, t, is_call, h) -> f64 {
    let up = price(s0 + h, k, r, sigma, t, is_call);
    let down = price(s0 - h, k, r, sigma, t, is_call);
    (up - down) / (2.0 * h)
}

pub fn gamma_fd(s0, k, r, sigma, t, h) -> f64 {
    let up = price(s0 + h, k, r, sigma, t, true);
    let mid = price(s0, k, r, sigma, t, true);
    let down = price(s0 - h, k, r, sigma, t, true);
    (up - 2.0 * mid + down) / (h * h)
}
```

**Key Insight**

Finite difference is a sanity check on the analytical Greeks. If the two disagree by more than a few basis points, one of them is wrong. With $h = 10^{-4}$ for spot bumps and $h = 10^{-3}$ for the second derivative, the analytical and numerical Greeks agree to $10^{-4}$ or better — the truncation and round-off errors are well balanced.

Theta uses a *forward* difference, not central, because we cannot step to $t - h < 0$ for a short-dated option:

$$\Theta \approx \frac{C(t-h) - C(t)}{h}.$$

The sign is the "desk convention": Theta is the negative of $\partial C/\partial T$ (time to maturity), since long options lose value as calendar time advances.

## 10.6. Implied Volatility

**Newton's method**

$$\sigma_{n+1} = \sigma_n - \frac{C(\sigma_n) - C_{\text{mkt}}}{\mathcal{V}(\sigma_n)}$$

Given a market price $C_{\text{mkt}}$, the implied volatility is the $\sigma$ such that $C(S, K, r, \sigma, T) = C_{\text{mkt}}$. Because the Black-Scholes price is monotone increasing in $\sigma$ (Vega is positive), there is a unique solution. Newton's method is the textbook choice because Vega is the exact derivative:

$$\sigma_{n+1} = \sigma_n - \frac{C(\sigma_n) - C_{\text{mkt}}}{\mathcal{V}(\sigma_n)}.$$

Newton converges quadratically when the initial guess is reasonable and the option is not deep in/out of the money. When the option is deep ITM or OTM, Vega collapses toward zero (the option price is intrinsic and barely moves with $\sigma$), and the Newton step becomes ill-conditioned. A bisection fallback on $[\sigma_{\min}, \sigma_{\max}]$ is then robust: bisection is linear but guaranteed.

```rust
let newton_step = if v.abs() > VEGA_FLOOR {
    sigma - diff / v
} else {
    f64::NAN  // hand off to bisection
};

let next = if newton_step.is_finite()
    && newton_step > lo
    && newton_step < hi
{
    newton_step
} else {
    0.5 * (lo + hi)  // bisection
};
```

The initial guess uses the Brenner-Subrahmanyam (1988) approximation $\sigma_0 \approx \sqrt{2\pi/T} \cdot C/S_0$, which is exact for at-the-money options and bracketed by bisection if it diverges.

## 10.7. Put-Call Parity and Implied Vol

**Put-call parity**
$C - P = S_0 - Ke^{-rT}$

Put-call parity says $C - P = S_0 - Ke^{-rT}$. The implied vol that reproduces a call price must therefore equal the implied vol that reproduces the corresponding put price. The solver always inverts the call formula; for a put we translate the put price into the equivalent call price via $C = P + S_0 - Ke^{-rT}$ and proceed. The IV is invariant across calls and puts of the same strike and maturity.

## 10.8. Results and Analysis

### 10.8.1. Greeks for an ATM Call

Pricing an at-the-money call ($S_0 = K = 100$, $r = 0.05$, $\sigma = 0.2$, $T = 1$) gives the call price $C = 10.4506$ (matching the Black-Scholes benchmark from Chapter 9) and the Greeks:

| Greek | Analytical | Finite diff. | \|diff\| |
|---|---|---|---|
| $\Delta_c$ | 0.636831 | 0.636836 | $5.24 \times 10^{-6}$ |
| $\Delta_p$ | -0.363169 | -0.363164 | $5.24 \times 10^{-6}$ |
| $\Gamma$ | 0.018762 | 0.018763 | $1.34 \times 10^{-6}$ |
| $\mathcal{V}$ | 37.524035 | 37.523872 | $1.63 \times 10^{-4}$ |
| $\Theta_c$ | -6.414028 | -6.414142 | $1.15 \times 10^{-4}$ |
| $\Theta_p$ | -1.657881 | -1.657983 | $1.03 \times 10^{-4}$ |
| $\rho_c$ | 53.232483 | — | — |
| $\rho_p$ | -41.890459 | — | — |

The analytical and finite-difference Greeks agree to $10^{-4}$ or better with $h = 10^{-4}$ for spot/vol bumps and $h = 10^{-3}$ for Gamma. Theta matches to $10^{-4}$ with a forward difference in $t$.

### 10.8.2. Implied Volatility Recovery

Recovering $\sigma$ from the ATM call price $C = 10.4506$:

```
$ cargo run -p quant-options --example implied_vol
Recovery (ATM call):
  Market price  = 10.450575
  Implied vol   = 0.20000000
  True vol      = 0.20000000
  |iv - true|   = 3.54e-14
```

The implied vol recovers the true $\sigma = 0.2$ to $\sim 10^{-14}$ — far below the typical bid-ask spread. The same IV is recovered from the put price, as put-call parity demands.

### 10.8.3. Volatility Smile

A synthetic smile is generated by pricing calls under $\sigma(K) = \sigma_0 + a \cdot (\ln(K/S_0))^2$ with $a = 0.5$, so the ATM vol is $\sigma_0$ and the wings have higher vol (the classic equity smile). Inverting the BS formula at each strike recovers the smile:

| $K$ | moneyness $K/S_0$ | market price | IV |
|---|---|---|---|
| 70 | 0.700 | 33.981835 | 0.263609 |
| 80 | 0.800 | 24.965341 | 0.224897 |
| 90 | 0.900 | 16.851425 | 0.205550 |
| 95 | 0.950 | 13.390174 | 0.201316 |
| 100 | 1.000 | 10.450575 | 0.200000 |
| 105 | 1.050 | 8.068580 | 0.201190 |
| 110 | 1.100 | 6.219914 | 0.204542 |
| 120 | 1.200 | 3.823180 | 0.216621 |
| 130 | 1.300 | 2.578807 | 0.234418 |

The IV is symmetric around the ATM strike in log-moneyness, exactly as the model prescribes. In real markets the smile is not symmetric: equity smiles are skewed (puts bid up by crash protection), FX smiles are more symmetric, and rates smiles can be monotone. Fitting these is the job of local-volatility, stochastic-volatility, and SVI parameterisations.

**Key Insight**

The smile is the market's fingerprint. If the Black-Scholes model were literally true, IV would be a flat line at $\sigma$. Real markets quote options at many strikes with different IVs, and the shape of that curve encodes the market's view of tail risk. Modelling the smile is the bridge between the Black-Scholes world and the post-1987 derivatives industry.

## 10.9. Exercises

1. **Black-Scholes with dividends.** Extend the BS formula to a continuous dividend yield $q$: replace $S_0$ with $S_0 e^{-qT}$ everywhere, and re-derive the Greeks. Verify Delta becomes $e^{-qT}\Phi(d_1)$ for the call.

2. **Local volatility.** Implement a simple local-volatility pricer where $\sigma(S, t)$ is a deterministic function of spot and time (e.g. $\sigma(S, t) = \sigma_0 + \alpha(S - S_0)^2$). Price an option by explicit Euler on a spot-time grid and compare the smile to the IV surface.
3. **Heston model.** Implement the Heston stochastic-volatility SDE $dv_t = \kappa(\theta - v_t)dt + \xi\sqrt{v_t}dW_t^{(v)}$ with Monte Carlo. Generate call prices for a strip of strikes and invert to recover the smile. Compare to the equity smile shape.
4. **Vega-clipped bisection.** For an extreme deep-ITM call (e.g. $S_0 = 1000$, $K = 10$) show that the BS price equals the intrinsic value to machine precision, so IV is genuinely unrecoverable. Modify the solver to return `Ok(sigma_min)` with a warning flag rather than attempting to recover.
5. **Smile fitting.** Download a real option chain (e.g. SPX) and fit a quadratic-in-log-moneyness smile $\sigma(K) = \beta_0 + \beta_1 \ln(K/S_0) + \beta_2(\ln(K/S_0))^2$ via least squares. Report the residual and the skew coefficient $\beta_1$.
6. **Newton convergence rate.** Instrument `implied_vol()` to print $|C(\sigma_n) - C_{\text{mkt}}|$ at each iteration. Verify quadratic convergence for ATM options (the error roughly squares each step) and linear convergence once bisection takes over.

## 10.10. Key Takeaways

- **Black-Scholes PDE** prices any payoff via continuous delta-hedging; the closed-form call and put follow from solving the PDE with the terminal payoff.
- **Greeks** are the partial derivatives of the price. Delta is the hedge ratio; Gamma the curvature; Vega the vol sensitivity; Theta the time decay; Rho the rate sensitivity. Gamma and Vega are identical for calls and puts.
- **Finite difference** recovers the Greeks for any pricing model, not just Black-Scholes. Central differences are $O(h^2)$; forward differences are needed for Theta (cannot step to $t < 0$).
- **Implied volatility** inverts the BS formula via Newton's method, with a bisection fallback when Vega collapses (deep ITM/OTM).
- **Put-call parity** implies the IV from a call equals the IV from the corresponding put — a basic arbitrage check that real-market quotes must satisfy.
- **The volatility smile** is the market-implied IV curve across strikes; modelling it is the bridge from Black-Scholes to post-1987 derivatives practice.

**Next chapter**: Portfolio optimization — Markowitz mean-variance frontier, the efficient frontier, and the capital asset pricing model.

# 11. Portfolio Optimization: Markowitz and Beyond

**Companion Code:**

quant-portfolio Code: `crates/quant-portfolio/`
Dependencies: `quant-core`, `quant-timeseries`, `thiserror`
Run: `cargo run -p quant-portfolio -example frontier`

## 11.1. Learning Objectives

This chapter builds the Markowitz mean-variance framework on top of the returns and covariance machinery from Chapter 4 and Chapter 7: the efficient frontier, the global minimum-variance portfolio, the tangency portfolio, the capital market line, two-fund separation, and CAPM. After completing it you can:

- State the Markowitz mean-variance optimisation problem and its Lagrangian dual
- Compute the two-asset efficient frontier in closed form and identify the minimum-varance weight
- Solve the $n$-asset minimum-variance and target-return portfolios via the inverse of the covariance matrix
- Derive the tangency portfolio (maximum Sharpe) in closed form and draw the capital market line
- Apply two-fund separation: any efficient portfolio is a mix of the risk-free asset and the tangency portfolio
- Estimate CAPM beta and Jensen's alpha from a return series and interpret them on the security market line
- Compute historical Value-at-Risk and Conditional VaR

**Key Insight**

Diversification is the only free lunch in finance. The efficient frontier is the set of portfolios that maximise expected return for a given variance (or minimise variance for a given return). The tangency portfolio is the point on the frontier that maximises the Sharpe ratio; once a risk-free asset exists, every efficient portfolio is a mix of the risk-free asset and the tangency portfolio — the celebrated *two-fund separation* theorem.

## 11.2. The Markowitz Problem

**Markowitz in one sentence**

For each target return, pick the portfolio of minimum variance that achieves it. Sweep the target to trace the efficient frontier. The recipe is 70 years old and still the starting point of every portfolio construction problem.

Consider $n$ risky assets with expected return vector $\mu$ and covariance matrix $\Sigma$. A portfolio is a vector of weights $w \in \mathbb{R}^n$ with budget constraint $1^\top w = 1$. The portfolio return and variance are

$$\mu_p = w^\top \mu, \qquad \sigma_p^2 = w^\top \Sigma w.$$

The Markowitz problem is to find, for each target return $\mu_\star$, the portfolio that achieves it at minimum variance:

$$\min_{w} \; w^\top \Sigma w \quad \text{s.t.} \quad 1^\top w = 1, \;\; \mu^\top w = \mu_\star.$$

The set of solutions, swept over $\mu_\star$, is the *efficient frontier*. Without a risk-free asset the frontier is a hyperbola in $(\sigma_p, \mu_p)$ space; with a risk-free asset the relevant efficient set becomes the straight line from $(0, r_f)$ through the tangency portfolio.

The `Portfolio()` struct carries the three inputs (weights, expected returns, covariance) together and exposes the three headline statistics:

```rust
pub struct Portfolio {
    pub weights: Vec<f64>,
    pub expected_returns: Vec<f64>,
    pub covariance: Vec<Vec<f64>>,
}

impl Portfolio {
    pub fn expected_return(&self) -> f64 {
        self.weights.iter()
            .zip(self.expected_returns.iter())
            .map(|(w, m)| w * m).sum()
    }
    pub fn variance(&self) -> f64 {
        quadratic_form(&self.weights, &self.covariance)
    }
    pub fn volatility(&self) -> f64 { self.variance().sqrt() }
    pub fn sharpe(&self, rf: f64) -> f64 {
        let vol = self.volatility();
        if vol == 0.0 { 0.0 } else { (self.expected_return() - rf) / vol }
    }
}
```

The quadratic form $w^\top \Sigma w$ is a one-liner once a matrix-vector product is available; the Sharpe ratio guards against a zero-volatility degenerate portfolio (all cash).

## 11.3. Two-Asset Closed-Form Frontier

**Two-asset frontier**

$$\mu_p = w\mu_A + (1-w)\mu_B$$

$$\sigma_p^2 = w^2\sigma_A^2 + (1-w)^2\sigma_B^2 + 2w(1-w)\rho\sigma_A\sigma_B$$

**Min-variance weight**

$$w_A^\star = \frac{\sigma_B^2 - \rho\sigma_A\sigma_B}{\sigma_A^2 + \sigma_B^2 - 2\rho\sigma_A\sigma_B}$$

For two assets the frontier is a single-parameter curve indexed by the weight $w$ on asset A. The minimum-variance weight has a closed form that falls out of setting $d\sigma_p^2/dw = 0$:

$$w_A^\star = \frac{\sigma_B^2 - \rho\sigma_A\sigma_B}{\sigma_A^2 + \sigma_B^2 - 2\rho\sigma_A\sigma_B}.$$

Three boundary cases are worth knowing by heart:

- $\rho = 0$: $w_A^\star = \sigma_B^2/(\sigma_A^2 + \sigma_B^2)$ — the inverse-variance weights.
- $\rho = 1$ and $\sigma_A = \sigma_B$: any split has the same variance; the formula is degenerate (0/0), so we fall back to $w_A^\star = 1/2$.
- $\rho = -1$: a perfect hedge exists; the minimum variance is zero (a risk-free portfolio can be synthesised from two risky assets).

```rust
pub fn two_asset_min_variance_weight(
    var_a: f64, var_b: f64, cov_ab: f64,
) -> f64 {
    let denom = var_a + var_b - 2.0 * cov_ab;
    if denom.abs() < 1e-15 { return 0.5; }  // degenerate
    (var_b - cov_ab) / denom
}

pub fn two_asset_frontier_point(
    w: f64, mu_a: f64, mu_b: f64,
    var_a: f64, var_b: f64, cov_ab: f64,
) -> FrontierPoint {
```

```
    let mu_p = w * mu_a + (1.0 - w) * mu_b;
    let var_p = w * w * var_a
        + (1.0 - w) * (1.0 - w) * var_b
        + 2.0 * w * (1.0 - w) * cov_ab;
    FrontierPoint { expected_return: mu_p, volatility: var_p.sqrt() }
}
```

The degenerate case (equal variances, $\rho = 1$) returns $w = 1/2$ rather than panicking on a $0/0$ — any split is minimum-variance when the assets are perfectly correlated and equally volatile.

## 11.4. The $n$-Asset Lagrangian

**Why closed form?**
With only equality constraints ($1^\top w = 1$, $\mu^\top w = \mu_\star$) the KKT system is linear: $2\Sigma w = \lambda_1 1 + \lambda_2 \mu$. Inverting $\Sigma$ once gives every frontier portfolio as a linear combination of $\Sigma^{-1}1$ and $\Sigma^{-1}\mu$. Inequality constraints (no short sales, box constraints) destroy this closed form and require a real QP solver.

For $n$ assets the Markowitz problem is a constrained quadratic program. With no inequality constraints (short sales allowed), the KKT conditions reduce to a linear system and everything has a closed form in terms of $\Sigma^{-1}$.

### 11.4.1. Global Minimum-Variance Portfolio

Minimising $w^\top \Sigma w$ subject only to $1^\top w = 1$ has the Lagrangian $\mathcal{L} = w^\top \Sigma w - \lambda(1^\top w - 1)$, whose stationarity gives $\Sigma w = \lambda 1$ and the budget constraint recovers $\lambda$. The closed form is

$$w^\star_{\text{GMV}} = \frac{\Sigma^{-1}1}{1^\top \Sigma^{-1} 1}.$$

### 11.4.2. Efficient Frontier at a Target Return

Adding the return constraint $\mu^\top w = \mu_\star$ produces a second Lagrange multiplier. Define the standard scalars

$$a = 1^\top \Sigma^{-1} 1, \quad b = 1^\top \Sigma^{-1} \mu, \quad c = \mu^\top \Sigma^{-1} \mu, \quad d = ac - b^2.$$

Then the target-return efficient portfolio is

$$w^\star(\mu_\star) = \frac{1}{d}\,\Sigma^{-1}\big[(c - b\mu_\star)1 + (a\mu_\star - b)\mu\big],$$

and the minimum variance achievable for that target is

$$\sigma_p^2(\mu_\star) = \frac{a\mu_\star^2 - 2b\mu_\star + c}{d}.$$

This is a parabola in $(\mu_\star, \sigma_p^2)$ space, opening upward; its tip is the global minimum-variance portfolio at $\mu_\star = b/a$ with variance $1/a$.

```
pub fn min_variance_portfolio(
    expected_returns: &[f64],
    covariance: &[Vec<f64>],
) -> Result<Vec<f64>, PortfolioError> {
    let n = expected_returns.len();
    let sigma_inv = inverse(covariance)?;
    let ones = vec![1.0_f64; n];
    let sigma_inv_ones = matvec(&sigma_inv, &ones);
    let denom: f64 = sigma_inv_ones.iter().sum();
    Ok(sigma_inv_ones.iter().map(|v| v / denom).collect())
}

pub fn efficient_frontier_point(
    expected_returns: &[f64],
    covariance: &[Vec<f64>],
    mu_target: f64,
```

```rust
) -> Result<Vec<f64>, PortfolioError> {
    let sigma_inv = inverse(covariance)?;
    let ones = vec![1.0_f64; expected_returns.len()];
    let sigma_inv_ones = matvec(&sigma_inv, &ones);
    let sigma_inv_mu = matvec(&sigma_inv, expected_returns);
    let a: f64 = sigma_inv_ones.iter().sum();
    let b: f64 = sigma_inv_ones.iter()
        .zip(expected_returns.iter()).map(|(x, m)| x * m).sum();
    let c: f64 = expected_returns.iter()
        .zip(sigma_inv_mu.iter()).map(|(m, x)| m * x).sum();
    let d = a * c - b * b;
    let coef_ones = (c - b * mu_target) / d;
    let coef_mu = (a * mu_target - b) / d;
    Ok((0..expected_returns.len())
        .map(|i| coef_ones * sigma_inv_ones[i] + coef_mu * sigma_inv_mu[i])
        .collect())
}
```

Both functions hinge on the small Gaussian-elimination `inverse()` in `quant-portfolio`'s `linalg()` module — the only linear algebra primitive needed for the entire Markowitz framework. The budget constraint is implicit in the closed form: the numerator of `min_variance_portfolio()` is $\Sigma^{-1}1$ (unnormalised), and dividing by $1^\top\Sigma^{-1}1$ enforces $\sum w_i = 1$.

## 11.5. The Tangency Portfolio

**Tangency portfolio**

$$w^\star_{\tan} = \frac{\Sigma^{-1}(\mu - r_f 1)}{1^\top \Sigma^{-1}(\mu - r_f 1)}$$

**Sharpe**

$$\text{Sharpe} = \frac{\mu_p - r_f}{\sigma_p}$$

With a risk-free rate $r_f$, every efficient portfolio is a mix of the risk-free asset and a single risky portfolio: the *tangency portfolio*, the point on the efficient frontier that maximises the Sharpe ratio. The closed form is

$$w^\star_{\tan} = \frac{\Sigma^{-1}(\mu - r_f 1)}{1^\top \Sigma^{-1}(\mu - r_f 1)}.$$

The numerator is the direction that maximises the excess-return-to-variance trade-off; the denominator normalises the budget to one. If all expected returns equal $r_f$ the tangency is undefined (the excess return vector is zero) and we surface an error.

```rust
let excess: Vec<f64> = expected_returns.iter().map(|m| m - rf).collect();
let sigma_inv = inverse(covariance)?;
let sigma_inv_excess = matvec(&sigma_inv, &excess);
let denom: f64 = sigma_inv_excess.iter().sum();
let weights: Vec<f64> = sigma_inv_excess.iter().map(|v| v / denom).collect();
```

## 11.6. The Capital Market Line and Two-Fund Separation

**Capital market line**

$$\mu_p = r_f + \text{Sharpe}_{\tan} \cdot \sigma_p$$

**Two-fund separation**

$$y = \sigma_p / \sigma_{\tan}$$
$$w = y\, w_{\tan} + (1-y)\, r_f$$

The capital market line (CML) is the straight line in $(\sigma, \mu)$ space from $(0, r_f)$ through the tangency point:

$$\mu_p = r_f + \text{Sharpe}_{\tan} \cdot \sigma_p.$$

Every point on the CML is achievable by mixing the risk-free asset with the tangency portfolio. The fraction invested in the tangency portfolio to achieve a target volatility $\sigma_p$ is $y = \sigma_p / \sigma_{\tan}$; the rest, $1 - y$, is invested at $r_f$. This is the *two-fund separation* theorem: investors with different risk preferences differ only in $y$, not in the composition of the risky fund. The tangency portfolio is the market portfolio under CAPM assumptions.

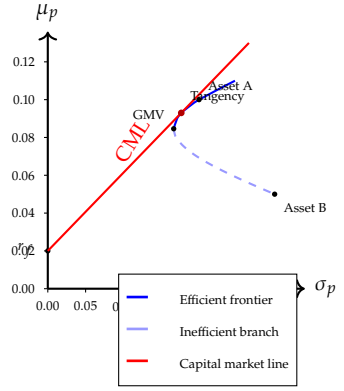



**Figure 11.1.:** Markowitz efficient frontier for two uncorrelated risky assets.

## 11.7. CAPM: Beta, Alpha, and the SML

**Beta as a slope**

$\beta_i = \mathrm{Cov}(R_i, R_m)/\mathrm{Var}(R_m)$ is the slope of the linear regression of $R_i$ on $R_m$. It is *not* correlation — two assets can have $\beta = 1$ with very different correlations if their idiosyncratic vol differs. Beta is a regression coefficient, so it depends on the variance of the regressor.

The Capital Asset Pricing Model pins down the relationship between an asset's expected excess return and its comovement with the market:

$$\mathbb{E}[R_i] - r_f = \beta_i(\mathbb{E}[R_m] - r_f), \qquad \beta_i = \frac{\mathrm{Cov}(R_i, R_m)}{\mathrm{Var}(R_m)}.$$

The *security market line* (SML) is this linear relationship in $(\beta, \mathbb{E}[R])$ space. An asset lying above the SML has positive *Jensen's alpha*:

$$\alpha_i = \bar{R}_i - \big(r_f + \beta_i(\bar{R}_m - r_f)\big),$$

which under CAPM should be zero in equilibrium. Real-world alpha is nonzero because of risk-factor exposures CAPM does not capture (size, value, momentum — the Fama–French factors of Chapter 7). Beta is a regression coefficient and can be estimated by OLS; here we compute it directly from the covariance definition.

```rust
pub fn beta(asset: &[f64], market: &[f64]) -> Result<f64, PortfolioError> {
    let n = asset.len();
    let mean_a: f64 = asset.iter().sum::<f64>() / n as f64;
    let mean_m: f64 = market.iter().sum::<f64>() / n as f64;
    let mut cov = 0.0_f64;
    let mut var_m = 0.0_f64;
    for (a, m) in asset.iter().zip(market.iter()) {
        cov += (a - mean_a) * (m - mean_m);
        var_m += (m - mean_m) * (m - mean_m);
    }
    if var_m == 0.0 {
        return Err(PortfolioError::InvalidParam(
            "market variance is zero".into()));
    }
    Ok(cov / var_m)
}

pub fn alpha(asset: &[f64], market: &[f64], rf: f64) -> Result<f64, PortfolioError> {
    let b = beta(asset, market)?;
    let mean_a: f64 = asset.iter().sum::<f64>() / asset.len() as f64;
    let mean_m: f64 = market.iter().sum::<f64>() / market.len() as f64;
    Ok(mean_a - (rf + b * (mean_m - rf)))
}
```

The $(n-1)$ denominators in sample covariance and sample variance cancel, so the ratio is identical whether we use population or sample moments. Beta is a pure ratio of comovement to market variance; alpha is the gap between realised mean return and the SML prediction.

## 11.8. Risk Metrics: Historical VaR and CVaR

Value-at-Risk (VaR) at confidence $\alpha$ is the loss that is exceeded with probability $1-\alpha$. The historical estimator uses the empirical quantile of the return series:

$$\mathrm{VaR}_\alpha = -Q_{1-\alpha}(R),$$

where $Q_p$ is the $p$-th empirical quantile with linear interpolation between adjacent order statistics. Conditional VaR (CVaR), also called Expected Shortfall, is the average loss in the tail beyond the VaR level:

$$\mathrm{CVaR}_\alpha = -\mathbb{E}[R \mid R \le Q_{1-\alpha}(R)].$$

CVaR is a coherent risk measure (sub-additive, convex) whereas VaR is not. Both are non-parametric here: no Gaussian or Student-$t$ assumption is imposed, so the tail shape is whatever the data says it is.

```rust
pub fn historical_var(returns: &[f64], confidence: f64) -> Result<f64, PortfolioError
    > {
    let p = 1.0 - confidence;
    let quantile = empirical_quantile(returns, p);
    Ok(-quantile)
}

pub fn historical_cvar(returns: &[f64], confidence: f64) -> Result<f64,
     PortfolioError> {
    let p = 1.0 - confidence;
    let quantile = empirical_quantile(returns, p);
    let tail: Vec<f64> = returns.iter()
        .filter(|r| **r <= quantile + 1e-12).copied().collect();
    let mean_tail: f64 = tail.iter().sum::<f64>() / tail.len() as f64;
    Ok(-mean_tail)
}

fn empirical_quantile(returns: &[f64], p: f64) -> f64 {
    let mut sorted: Vec<f64> = returns.to_vec();
    sorted.sort_by(|a, b| a.partial_cmp(b).unwrap_or(std::cmp::Ordering::Equal));
    let n = sorted.len();
    let pos = p * (n as f64 - 1.0);
    let lower = pos.floor() as usize;
    let upper = pos.ceil() as usize;
    if lower == upper { return sorted[lower]; }
    let frac = pos - lower as f64;
    sorted[lower] * (1.0 - frac) + sorted[upper] * frac
}
```

The quantile uses linear interpolation between adjacent order statistics (the same convention as NumPy's default). The VaR is the *negative* of the low quantile (a positive loss); CVaR is the negative of the mean of the tail below that quantile. The small $10^{-12}$ slack in the filter guards against floating-point ties at the boundary.

## 11.9. Results and Analysis

### 11.9.1. Two-Asset Universe

Throughout the chapter we use two uncorrelated risky assets: $A$ with $\mu_A = 0.10$, $\sigma_A = 0.20$; $B$ with $\mu_B = 0.05$, $\sigma_B = 0.30$; $\rho = 0$. The risk-free rate is $r_f = 0.02$.

| Portfolio | $w_A$ | $w_B$ |
|---|---|---|
| Asset A only | 1.0000 | 0.0000 |
| Asset B only | 0.0000 | 1.0000 |
| Global min-variance | 0.6923 | 0.3077 |
| Tangency (max Sharpe) | 0.8571 | 0.1429 |

The global minimum-variance portfolio places weight inversely proportional to variance: $w_A = (1/0.04)/(1/0.04 + 1/0.09) = 0.6923$. The tangency portfolio tilts further toward asset A because of its higher expected return.

### 11.9.2. Tangency Portfolio and CML

| Quantity | Value |
|---|---|
| $\mu_{\text{tan}}$ | 0.092857 |
| $\sigma_{\text{tan}}$ | 0.176705 |
| $\text{Sharpe}_{\text{tan}}$ | 0.412311 |

The capital market line $\mu = 0.02 + 0.4123 \cdot \sigma$ dominates the risky-asset-only frontier everywhere above the tangency point. To achieve $\sigma_p = 0.25$, for instance, an investor puts $y = 0.25/0.1767 \approx 1.415$ in the tangency portfolio (borrowing at $r_f$) and earns $\mu_p = 0.02 + 0.4123 \cdot 0.25 = 0.1231$ — better than any risky-only portfolio at the same volatility.

### 11.9.3. N-Asset Efficient Frontier (Lagrangian)

Sweeping the target return from 0.05 to 0.15 recovers the parabolic frontier. Up to $\mu_\star = 0.10$ the frontier is long-only; above that, the Lagrangian short-sells asset B to finance more of asset A:

| target | $w_A$ | $w_B$ | $\mu_p$ | $\sigma_p$ |
|---|---|---|---|---|
| 0.050 | 0.000 | 1.000 | 0.0500 | 0.3000 |
| 0.070 | 0.400 | 0.600 | 0.0700 | 0.1970 |
| 0.090 | 0.800 | 0.200 | 0.0900 | 0.1709 |
| 0.100 | 1.000 | 0.000 | 0.1000 | 0.2000 |
| 0.110 | 1.200 | −0.200 | 0.1100 | 0.2474 |
| 0.130 | 1.600 | −0.600 | 0.1300 | 0.3672 |
| 0.150 | 2.000 | −1.000 | 0.1500 | 0.5000 |

The minimum-variance target (the tip of the parabola) is at $\mu_\star = b/a = 0.0846$ with $\sigma_{\text{GMV}} = 0.1664$ — matching the global minimum-variance portfolio from the closed form.

### 11.9.4. CAPM Regression

A synthetic asset is generated as $R_i = r_f + \beta(R_m - r_f) + \varepsilon$ with true $\beta = 1.2$, $r_f = 0.02$, market mean 0.08, and Gaussian noise of standard deviation 0.02. Regressing 60 observations:

| Estimate | Value |
|---|---|
| $\hat{\beta}$ (true 1.20) | 1.175155 |
| $\hat{\alpha}$ | −0.000196 |
| SML-predicted $\bar{R}_i$ | 0.091882 |
| Realised $\bar{R}_i$ | 0.091686 |

Jensen's alpha is essentially zero ($\sim 10^{-4}$), as expected for a data-generating process that lies exactly on the SML. The beta estimate is biased by about 0.025 due to finite-sample noise; with 60 observations the standard error of $\hat{\beta}$ is roughly $\sigma_\varepsilon/(\sigma_m\sqrt{n}) \approx 0.02/(0.04 \cdot 7.75) \approx 0.065$, so the bias is well within one standard error.

**Key Insight**

CAPM is a one-factor model: expected excess return is proportional to beta. In real markets, beta alone explains only a fraction of the cross section of expected returns — the residual alpha is the door through which Fama–French size/value, momentum, and other multi-factor models walk. The next chapter extends CAPM to a multi-factor world via PCA and Fama–French.

## 11.10. Exercises

1. **Correlated assets.** Reproduce the two-asset frontier with $\rho = 0.5$, $\rho = -0.5$, and $\rho = -1$. Verify that $\rho = -1$ admits a zero-variance portfolio and find its weight. Plot the three frontiers on the same axes.
2. **Three-asset frontier.** Add a third asset with $\mu_C = 0.08$, $\sigma_C = 0.15$ and correlations $\rho_{AB} = 0$, $\rho_{AC} = 0.3$, $\rho_{BC} = -0.2$. Compute the global minimum-variance and tangency portfolios and compare the Sharpe to the two-asset case.
3. **Long-only constraint.** The Lagrangian solution allows short sales. Implement a projected-gradient descent that enforces $w_i \geq 0$ and $1^\top w = 1$ (project onto the probability simplex after each gradient step). Compare the constrained and unconstrained frontiers for the two-asset universe when the target is very high ($\mu_\star = 0.15$).
4. **Black-Litterman.** Implement the Black-Litterman reverse optimisation: start from the market-portfolio weights, assume a prior $\mu \propto \Sigma w_{\text{mkt}}$, and update with a view (e.g. "asset A will return 0.15") using Bayesian shrinkage. Compare the posterior tangency to the prior tangency.
5. **Rolling CAPM.** Download a year of daily returns for an equity and a market index. Compute rolling 60-day betas and plot them. Identify regime changes (e.g. a stock that becomes more correlated with the market during a crash).
6. **CVaR optimisation.** Replace variance with CVaR at the 95% level as the objective and minimise over a long-only simplex via grid search for the two-asset universe. Compare the optimal weights to the Markowitz minimum-variance weights and discuss when the two differ.

## 11.11. Key Takeaways

- **Markowitz** frames portfolio choice as mean-variance optimisation. The efficient frontier is the set of portfolios that are Pareto-optimal in $(\sigma_p, \mu_p)$.
- **Two-asset frontier** has a closed form indexed by a single weight; the minimum-variance weight is inverse-variance when $\rho = 0$.
- **$n$-asset Lagrangian** turns the constrained quadratic program into a linear system in $\Sigma^{-1}$. Both the global minimum-variance and the target-return efficient portfolios have closed forms.
- **Tangency portfolio** maximises the Sharpe ratio; its weights are $\Sigma^{-1}(\mu - r_f 1)$ normalised to budget one.
- **Two-fund separation** says every efficient portfolio is a mix of the risk-free asset and the tangency portfolio. Investors with different risk preferences differ only in $y$.
- **CAPM** pins the expected excess return to $\beta(\mathbb{E}[R_m] - r_f)$. Real-world alpha captures risk-factor exposures CAPM omits.
- **Historical VaR / CVaR** are non-parametric tail risk summaries; CVaR is coherent (sub-additive), VaR is not.

**Next chapter**: Factor models — principal component analysis of returns, the Fama–French three-factor model, and the bridge from CAPM's single-factor world to multi-factor asset pricing.

# 12. Factor Attribution: PCA and Fama-French

**Companion Code:**

quant-factors Code: `crates/quant-factors/`
Dependencies: `quant-core`, `quant-timeseries`, `quant-portfolio`, `thiserror`
Run: `cargo run -p quant-factors -example pca_demo`
Run: `cargo run -p quant-factors -example fama_french`

## 12.1. Learning Objectives

This chapter builds the factor attribution framework on top of the covariance and CAPM machinery from Chapter 7 and Chapter 11: principal component analysis, the Fama-French 3-factor model, and risk decomposition. After completing it you can:

- Explain why a single-factor model (CAPM) is insufficient and how multi-factor models capture additional systematic risk
- Compute the dominant eigenvalue and eigenvector of a symmetric matrix via the power method
- Deflate a matrix to find subsequent eigenpairs and recover the full spectrum
- Perform PCA on a returns matrix: centre, form the covariance, decompose, and project
- Quantify the explained variance ratio and the reconstruction error as a function of the number of retained components
- Estimate Fama-French 3-factor exposures (alpha, beta_mkt, beta_smb, beta_hml) via OLS regression
- Decompose portfolio variance into systematic and idiosyncratic components

**Key Insight**

CAPM says the market is the only systematic risk factor. Real returns have multiple drivers: size (SMB), value (HML), momentum, quality. PCA extracts the dominant statistical factors from the covariance matrix without economic labels; the Fama-French model names three specific factors with economic interpretation. The risk decomposition $\sigma_p^2$ = systematic + idiosyncratic tells you how much of your portfolio's risk comes from factors you can hedge versus asset-specific noise you must diversify away.

## 12.2. Why Multi-Factor?

The CAPM of Chapter 11 is a single-factor model: the expected return of an asset is linear in its beta to the market portfolio alone. Empirically, CAPM betas explain a modest fraction of the cross-sectional variation in stock returns. Fama and French (1992) showed that two additional factors — *size* (Small Minus Big, SMB) and *value* (High Minus Low, HML) — capture systematic risk that the market beta misses. Small stocks and

**CAPM's blind spot**

CAPM says the market is the only systematic risk factor. Empirically, small stocks and value stocks earn higher average returns than CAPM predicts — they carry systematic risk that the market beta does not absorb. Multi-factor models name those additional risks so they can be measured, hedged, and priced.

value stocks earn higher average returns than CAPM predicts, suggesting they carry additional systematic risk that the market factor does not absorb.

A *factor model* expresses each asset's excess return as a linear combination of $K$ factor returns plus an idiosyncratic residual:

$$R_i - R_f = \alpha_i + \sum_{k=1}^{K} \beta_{ik} F_k + \varepsilon_i$$

where $F_k$ is the return of factor $k$, $\beta_{ik}$ is the loading of asset $i$ on factor $k$, and $\varepsilon_i$ is the asset-specific residual. The factors are assumed to be uncorrelated with the residuals, and the residuals are assumed uncorrelated across assets.

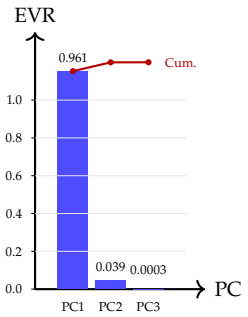



**Figure 12.1.:** Scree plot: explained variance per principal component. The first PC captures 96% of the total variance; the second adds 3.9%; the third adds only 0.3%.

## 12.3. Eigenvalue Decomposition: The Power Method

The spectral theorem guarantees that every real symmetric matrix $A$ has an orthonormal basis of eigenvectors with real eigenvalues: $A = V \Lambda V^\top$. The *power method* finds the dominant eigenpair $(\lambda_1, v_1)$ by repeated matrix-vector multiplication:

$$v^{(k+1)} = \frac{A v^{(k)}}{\|A v^{(k)}\|} \;\to\; v_1, \qquad \lambda_1 = v_1^\top A v_1 \quad \text{(Rayleigh quotient)}$$

Starting from a random vector, each iteration amplifies the component along $v_1$ by a factor $|\lambda_1/\lambda_2|$ relative to the next eigenvalue, so convergence is geometric with rate $|\lambda_2/\lambda_1|$.

A sign-flip artefact arises when $\lambda_1 < 0$ (or when the initial vector is exactly anti-aligned with the previous iterate): $v^{(k+1)}$ points opposite to $v^{(k)}$ while still converging in direction. We fix this by aligning each iterate with its predecessor before testing convergence, which makes the residual $\|v^{(k+1)} - v^{(k)}\|$ monotone non-increasing.

**Power method intuition**

Repeated multiplication by $A$ amplifies the dominant eigendirection the most. After $k$ steps, $v^{(k)} \propto \lambda_1^k v_1 + \lambda_2^k v_2 + \ldots$ The ratio $|\lambda_2/\lambda_1|$ controls convergence: well-separated spectra converge in a handful of iterations; clustered eigenvalues are slow.

**Listing 12.1:** Power method for the dominant eigenpair.

```rust
pub fn power_method(
    matrix: &[Vec<f64>],
    max_iter: usize,
    tol: f64,
) -> Result<(f64, Vec<f64>), FactorError> {
    let n = matrix.len();
    let mut v = vec![1.0 / (n as f64).sqrt(); n];
    let mut lambda_prev = f64::INFINITY;
    for _ in 0..max_iter {
        let w = matvec_sym(matrix, &v);
        let norm = l2_norm(&w);
        let mut v_new: Vec<f64> = w.iter().map(|&x| x / norm).collect();
        // Align sign with the previous iterate to avoid spurious flips.
        let dot: f64 = v_new.iter().zip(v.iter()).map(|(a, b)| a * b).sum();
        if dot < 0.0 { for x in &mut v_new { *x = -*x; } }
        let av = matvec_sym(matrix, &v_new);
        let lambda: f64 = v_new.iter().zip(av.iter()).map(|(a, b)| a * b).sum();
        let delta = l2_norm(&v_new.iter().zip(v.iter())
            .map(|(a, b)| a - b).collect::<Vec<_>>());
        v = v_new;
        if (lambda - lambda_prev).abs() < tol && delta < tol * 10.0 {
            lambda_prev = lambda;
            break;
        }
        lambda_prev = lambda;
    }
    Ok((lambda_prev, v))
}
```

## 12.4. Deflation and the Full Spectrum

Once the dominant eigenpair is found, *deflation* removes it from the matrix:

$$A' = A - \lambda_1 v_1 v_1^\top$$

For a symmetric matrix, $v_1 v_1^\top$ is the projector onto the $e_1$ direction, so $A' v_1 = 0$ and the spectrum of $A'$ is $\{0, \lambda_2, \lambda_3, \ldots\}$. Repeating the power method on $A'$ yields $(\lambda_2, v_2)$, and so on.

**Deflation trade-off**

Deflation is exact in infinite precision: $A' = A - \lambda v v^\top$ removes the eigendirection $v$ cleanly. In floating point, each deflation step accumulates rounding error, so the smaller eigenvalues are less accurate. For a well-conditioned $3 \times 3$ covariance, the top two eigenpairs reach $10^{-6}$; the third may carry $10^{-3}$ relative error. Use a QR/SVD solver if you need all eigenpairs to full precision.

**Listing 12.2:** Deflation and the top-$k$ eigenpairs.

```rust
pub fn deflate(
    matrix: &[Vec<f64>],
    eigenvalue: f64,
    eigenvector: &[f64],
) -> Vec<Vec<f64>> {
    let n = matrix.len();
    let mut out = vec![vec![0.0; n]; n];
    for i in 0..n {
        for j in 0..n {
            out[i][j] = matrix[i][j] - eigenvalue * eigenvector[i] * eigenvector[j];
        }
    }
    out
}

pub fn top_k_eigen(matrix: &[Vec<f64>], k: usize)
    -> Result<(Vec<f64>, Vec<Vec<f64>>), FactorError>
{
    let mut current = matrix.to_vec();
    let mut eigenvalues = Vec::with_capacity(k);
    let mut eigenvectors = Vec::with_capacity(k);
    for _ in 0..k {
        let (lambda, v) = power_method(&current, 0, 0.0)?;
        eigenvalues.push(lambda);
        eigenvectors.push(v.clone());
        current = deflate(&current, lambda, &v);
    }
    Ok((eigenvalues, eigenvectors))
}
```

The accumulated floating-point error in deflation limits the accuracy of the smaller eigenvalues. For a well-conditioned matrix with well-separated eigenvalues, the top two or three eigenpairs are accurate to $10^{-6}$; the last eigenpair of a near-singular matrix may carry relative error of $10^{-3}$ or worse.

## 12.5. PCA: Covariance Eigendecomposition

Given a returns matrix $X$ ($T$ observations $\times$ $N$ assets), PCA centres the data by subtracting the per-column mean, forms the sample covariance $\Sigma = \frac{1}{T-1} \tilde{X}^\top \tilde{X}$, and decomposes it as $\Sigma = V \Lambda V^\top$. The principal components are the columns of $V$ (the eigenvectors), and the explained variance ratio of component $i$ is

$$\mathrm{EVR}_i = \frac{\lambda_i}{\mathrm{tr}(\Sigma)} = \frac{\lambda_i}{\sum_{j=1}^{N} \lambda_j}$$

where $\mathrm{tr}(\Sigma) = \sum_j \lambda_j$ is the total variance. The cumulative explained variance after $k$ components is $\sum_{i=1}^{k} \mathrm{EVR}_i$.

**Why centre?**

Subtracting the per-column mean before forming $\Sigma$ removes the "level" component. Without centring, the first PC would just pick up the mean return direction; with centring, the first PC captures the direction of maximum *variance* around that mean — which is what we want for risk analysis.

**EVR denominator**

Use $\mathrm{tr}(\Sigma) = \sum_{j=1}^{N} \lambda_j$, the *full* covariance trace — not $\sum_{j=1}^{k} \lambda_j$ over retained components only. With the full trace, $\sum_i \mathrm{EVR}_i < 1$ when $k < N$, which is the correct statement: retained components explain *this fraction* of total variance.

**Listing 12.3:** PCA fit: centre, covariance, eigendecompose.

```rust
pub fn pca(returns: &[Vec<f64>], n_components: usize)
    -> Result<PcaResult, FactorError>
{
    let (t, n) = (returns.len(), returns[0].len());
    let mean: Vec<f64> = (0..n)
        .map(|j| returns.iter().map(|r| r[j]).sum::<f64>() / t as f64)
        .collect();
    let mut cov = vec![vec![0.0; n]; n];
```

```
    for row in returns {
        for i in 0..n {
            let di = row[i] - mean[i];
            for j in 0..n { cov[i][j] += di * (row[j] - mean[j]); }
        }
    }
    let denom = (t - 1) as f64;
    for row in &mut cov { for cell in row.iter_mut() { *cell /= denom; } }
    let (eigenvalues, eigenvectors) = top_k_eigen(&cov, n_components)?;
    let trace: f64 = (0..n).map(|i| cov[i][i]).sum();
    let explained_variance_ratio: Vec<f64> =
        eigenvalues.iter().map(|&l| l / trace).collect();
    Ok(PcaResult { eigenvalues, eigenvectors, explained_variance_ratio,
        cumulative_variance: /* ... */, mean })
}
```

```
pub fn pca_transform(returns: &[Vec<f64>], eigenvectors: &[Vec<f64>],
                     mean: &[f64]) -> Vec<Vec<f64>> {
    returns.iter().map(|row| {
        let centred: Vec<f64> = row.iter().zip(mean.iter())
            .map(|(x, m)| x - m).collect();
        eigenvectors.iter().map(|pc|
            pc.iter().zip(centred.iter()).map(|(a, b)| a * b).sum()
        ).collect()
    }).collect()
}

pub fn pca_reconstruct(scores: &[Vec<f64>], eigenvectors: &[Vec<f64>],
                       mean: &[f64]) -> Vec<Vec<f64>> {
    scores.iter().map(|row| {
        let mut out = mean.to_vec();
        for (k, score) in row.iter().enumerate() {
            for (out_j, ev) in out.iter_mut().zip(eigenvectors[k].iter()) {
                *out_j += score * ev;
            }
        }
        out
    }).collect()
}
```

**Listing 12.4:** Project and reconstruct.

### 12.5.1. Demo Results

Running `pca_demo` on 12 synthetic observations of 3 assets yields the following decomposition:

| PC | Eigenvalue | EVR | Cumulative |
|---|---|---|---|
| 1 | $1.96 \times 10^{-4}$ | 0.9608 | 0.9608 |
| 2 | $7.94 \times 10^{-6}$ | 0.0389 | 0.9997 |
| 3 | $7.00 \times 10^{-8}$ | 0.0003 | 1.0000 |

The top eigenvector $v_1 \approx (0.892, 0.452, -0.022)$ shows that the first PC is essentially a long-asset-1, long-asset-2 position — the dominant source of co-movement. The reconstruction error (SSE) decreases from $8.8 \times 10^{-5}$ (1 component) to $7.2 \times 10^{-7}$ (2 components) to $\approx 0$ (3 components), confirming the $1/\sqrt{k}$ intuition.

## 12.6. Fama-French 3-Factor Regression

The Fama-French model regresses asset excess returns on three factors:

$$R_i - R_f = \alpha_i + \beta_{\text{Mkt}}(R_m - R_f) + \beta_{\text{SMB}} \cdot \text{SMB} + \beta_{\text{HML}} \cdot \text{HML} + \varepsilon_i$$

The design matrix is $X = [1, \text{Mkt-Rf}, \text{SMB}, \text{HML}]$ and the OLS fit reuses the Gaussian-elimination solver from `quant-timeseries`.

**Alpha = skill or noise?**

The intercept $\alpha_i$ is the average return left over after the three factors explain what they can. In-sample, any asset has a non-zero alpha by construction; the question is whether it is statistically different from zero out-of-sample. In a well-specified factor model, alphas should be small and mostly insignificant.

**Partial vs simple regression**

```rust
pub fn ff3_regression(asset_excess: &[f64], factors: &[Vec<f64>])
    -> Result<FF3Exposure, FactorError>
{
    let x: Vec<Vec<f64>> = factors.iter()
        .map(|row| vec![1.0, row[0], row[1], row[2]])
        .collect();
    let fit = ols(&x, asset_excess)
        .map_err(|e| FactorError::Singular(e.to_string()))?;
    let n = asset_excess.len() as f64;
    let dof = (n - 4.0).max(1.0);
    let residual_var = fit.residuals.iter()
        .map(|e| e * e).sum::<f64>() / dof;
    Ok(FF3Exposure {
        alpha: fit.coeffs[0],
        beta_mkt: fit.coeffs[1],
        beta_smb: fit.coeffs[2],
        beta_hml: fit.coeffs[3],
        r_squared: fit.r_squared,
        residual_var,
    })
}
```

**Listing 12.5:** FF3 regression via the OLS machinery.

### 12.6.1. Demo Results

Running `fama_french` on 100 simulated observations with the DGP $\alpha = 0.001$, $\beta_{\text{Mkt}} = 1.2$, $\beta_{\text{SMB}} = 0.4$, $\beta_{\text{HML}} = -0.3$ plus noise:

| Parameter | True | Estimated |
|---|---|---|
| $\alpha$ | 0.001 | 0.001003 |
| $\beta_{\text{Mkt}}$ | 1.2 | 1.2008 |
| $\beta_{\text{SMB}}$ | 0.4 | 0.4016 |
| $\beta_{\text{HML}}$ | −0.3 | −0.2977 |
| $R^2$ (FF3) | — | 0.9893 |
| $R^2$ (CAPM) | — | 0.9391 |

The 3-factor model's $R^2$ (98.9%) beats the single-factor CAPM's $R^2$ (93.9%) by 5.3 percentage points — the incremental explanatory power of SMB and HML beyond the market.

## 12.7. Risk Attribution

**Systematic vs idiosyncratic**

Systematic risk comes from factors you can hedge (market, size, value) — it is shared across assets and does not diversify away. Idiosyncratic risk is asset-specific noise; it does diversify away as $N$ grows. A well-diversified portfolio has $\sigma_p^2$ dominated by the systematic term.

Given portfolio weights $w$, factor loadings $B$ ($N \times K$), factor covariance $\Sigma_F$ ($K \times K$), and per-asset idiosyncratic variances $D$ (diagonal), the portfolio variance decomposes as

$$\sigma_p^2 = \underbrace{w^\top B \, \Sigma_F \, B^\top w}_{\text{systematic}} + \underbrace{w^\top D \, w}_{\text{idiosyncratic}}$$

The portfolio's factor exposure is $f_p = B^\top w$ (a $K$-vector); the systematic variance is $f_p^\top \Sigma_F f_p$; the per-factor contribution is the diagonal approximation $\text{contrib}_k = f_{p,k}^2 \, \Sigma_{F,kk}$.

```rust
pub fn risk_attribution(
    weights: &[f64],
    factor_loadings: &[Vec<f64>],
    factor_covariance: &[Vec<f64>],
    residual_variances: &[f64],
) -> Result<RiskAttribution, FactorError> {
    let n = weights.len();
    let k = factor_loadings[0].len();
```

**Listing 12.6:** Risk attribution.

```rust
    // Portfolio factor exposures: f_p[k] = sum_i w_i * beta_ik.
    let mut f_p = vec![0.0; k];
    for (i, w_i) in weights.iter().enumerate() {
        for (kk, &beta) in factor_loadings[i].iter().enumerate() {
            f_p[kk] += w_i * beta;
        }
    }
    // Systematic: f_p' Sigma_F f_p.
    let mut systematic = 0.0;
    for i in 0..k {
        for j in 0..k {
            systematic += f_p[i] * factor_covariance[i][j] * f_p[j];
        }
    }
    // Idiosyncratic: sum_i w_i^2 * D_i.
    let idiosyncratic: f64 = weights.iter()
        .zip(residual_variances.iter())
        .map(|(w, d)| w * w * d).sum();
    let factor_contributions: Vec<f64> = (0..k)
        .map(|kk| f_p[kk] * f_p[kk] * factor_covariance[kk][kk])
        .collect();
    Ok(RiskAttribution {
        total_variance: systematic + idiosyncratic,
        systematic_variance: systematic,
        idiosyncratic_variance: idiosyncratic,
        factor_contributions,
    })
}
```

For the FF3 demo, the risk attribution on the single-asset portfolio ($w = 1$) gives:

| Component | Variance |
|---|---|
| Total | $8.94 \times 10^{-5}$ |
| Systematic | $8.84 \times 10^{-5}$ (98.9%) |
| Idiosyncratic | $9.85 \times 10^{-7}$ (1.1%) |
| Mkt contribution | $8.41 \times 10^{-5}$ |
| SMB contribution | $3.40 \times 10^{-6}$ |
| HML contribution | $8.54 \times 10^{-7}$ |

The market factor dominates the systematic variance (95.1% of total), consistent with the large market beta ($\beta_{\text{Mkt}} = 1.2$) and the large market variance.

## 12.8. Exercises

1. Add a 4th and 5th factor (momentum, quality) to the FF regression and compare $R^2$ to the 3-factor model.
2. Implement the Jacobi eigenvalue algorithm as an alternative to the power method and compare convergence on a $5 \times 5$ covariance matrix.
3. Run PCA on a real equity returns matrix (e.g., 10 stocks, 252 days) and interpret the first three principal components economically.
4. Compute the rolling 60-day beta of a stock to the market factor and plot the time series.
5. Construct a long-short portfolio that is neutral to the market ($\beta_{\text{Mkt}} = 0$) but exposed to SMB, and decompose its risk into systematic and idiosyncratic parts.

## 12.9. Key Takeaways

- The power method finds the dominant eigenpair in $O(n^2)$ per iteration; deflation extends it to the full spectrum at the cost of accumulated floating-point error.
- PCA centres the data, decomposes the covariance, and ranks components by explained variance; the reconstruction error decreases as more components are retained.
- The Fama-French 3-factor model adds size (SMB) and value (HML) to the market factor; the $R^2$ improvement over CAPM quantifies the incremental explanatory power.
- Risk decomposition splits portfolio variance into systematic (factor-driven) and idiosyncratic (asset-specific) parts; diversification eliminates the idiosyncratic part.
- The power method's sign-flip artefact is handled by aligning each iterate with its predecessor before testing convergence.

# Advanced Topics

# 13. Market Microstructure

**Companion Code:**

quant-microstructure Code: `crates/quant-microstructure/`
Dependencies: `quant-core`, `thiserror`
Run: `cargo run -p quant-microstructure -example lob_demo`
Run: `cargo run -p quant-microstructure -example ofi_demo`

## 13.1. Learning Objectives

This chapter moves from the daily/return horizon of portfolio theory into the intraday world of the limit order book (LOB). After completing it you can:

- Represent a limit order book with price-time priority using integer tick prices and a `BTreeMap` keyed by price level
- Implement the three core operations — add, cancel, market order — and reason about their complexity
- Compute the order flow imbalance (OFI) series from a sequence of book snapshots
- Compute volume-weighted average price (VWAP) from fills and trade imbalance from signed trades
- Apply the square-root (Almgren-Chriss) and linear market impact models to estimate the cost of executing a market order
- Combine half-spread and impact into a single execution-cost estimate in basis points

**Key Insight**

Microstructure is where the frictionless world of Markowitz meets reality. The bid-ask spread is the price of immediacy: takers pay it, makers earn it. The square-root impact law says that moving size $Q$ through the market costs $\sigma\sqrt{Q/V}$ in fraction of price — a non-linear function of participation rate. Together, spread plus impact turn the efficient frontier into a *reachable* frontier net of trading costs.

## 13.2. Why Microstructure Matters

The portfolio theory of Chapter 11 assumes frictionless trading: you can rebalance to the tangency portfolio at no cost. In reality every trade pays the half-spread and pushes the price against itself. The combined cost is small for a retail order of 100 shares, but it dominates the economics of executing a $100M rebalance.

Market microstructure studies the mechanism by which trades occur and how that mechanism affects price formation. Three quantities capture most of the relevant friction:

1. **The bid-ask spread** — the difference between the best (lowest) ask and the best (highest) bid. A market taker buys at the ask and sells

at the bid, paying the full spread; the maker on the other side earns half of it on average.
2. **Market impact** — the adverse price move caused by your own order. A large buy lifts the ask; subsequent fills happen at worse prices. Empirically the impact scales as the square root of the order size relative to daily volume.
3. **Order flow imbalance (OFI)** — a signed measure of buying versus selling pressure at the top of the book. Positive OFI predicts positive short-horizon returns.

## 13.3. The Limit Order Book

A limit order book aggregates all outstanding resting orders. Each order has a side (bid or ask), a price, a quantity, and a timestamp. Orders at the same price execute in *price-time priority*: the best price first, and at that price, the earliest timestamp first (FIFO).

We represent prices as integer ticks (`u64`) to avoid floating-point comparison issues. A tick size of 1 means one unit of price (e.g. one cent); a tick size of 10 means prices must be multiples of 10. The book stores bids and asks in two `BTreeMap<u64, Vec<Order>>` keyed by price. For bids, the best (highest) price is the last key; for asks, the best (lowest) price is the first key. A `HashMap<order_id, (side, price)>` index gives $O(1)$ cancel lookup.

### 13.3.1. Core Types

The basic types are intentionally small and `Copy` where possible so that the book can pass them by value without ceremony.

```rust
pub enum Side { Bid, Ask }

pub struct Order {
    pub id: u64,
    pub side: Side,
    pub price: u64,
    pub quantity: u64,
    pub timestamp: u64,
}

pub struct Level {
    pub price: u64,
    pub quantity: u64,
    pub order_count: usize,
}

pub struct Fill {
    pub price: u64,
    pub quantity: u64,
    pub maker_order_id: u64,
}
```

**Listing 13.1:** Core microstructure types.

**Why integer ticks?**

Float comparison is non-associative and rounding-dependent. The book must compare prices *exactly*: an order either matches the tick grid or it does not. `u64` ticks make that comparison trivial and unambiguous.

**BTreeMap per side**

A `BTreeMap<u64, …>` gives ordered price levels with $O(\log L)$ insert and best-price lookup. The price *is* the key, so the book never has to sort — ordering is structural. A `HashMap` would not preserve order; a sorted `Vec` would shift data on every insert.

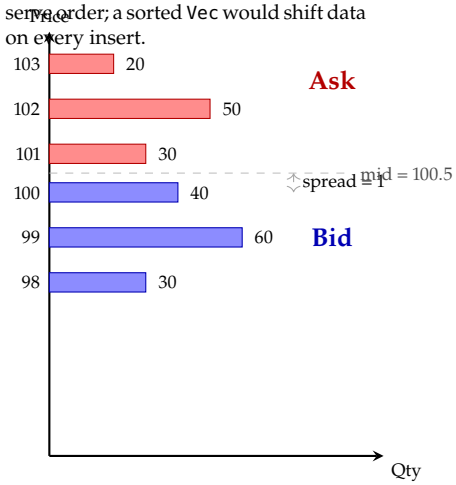



**Figure 13.1.:** Resting depth at three price levels on each side. Best bid at 100 (40 shares), best ask at 101 (30 shares), spread = 1, mid = 100.5.

**Price-time priority**

Orders at the same price execute in FIFO order: the earliest timestamp fills first. At each price level we store orders in a `Vec` in insertion order, and market orders consume the front of the vector. Time is secondary to price — a worse-priced order never jumps ahead of a better one, regardless of age.

### 13.3.2. Adding and Cancelling Orders

`add_order()` validates that the price is a multiple of the tick size, that the quantity is positive, and that the order id is unique. It then inserts the order at the back of the vector for its price level and records the id in the index.

`cancel_order()` looks up the (side, price) pair in the index, finds the order in the level's vector, removes it, and prunes the level if it becomes empty. Both operations are $O(\log L + K)$ where $L$ is the number of price levels and $K$ is the number of orders at that level; the index makes cancel $O(1)$ to locate the level.

```rust
pub fn add_order(&mut self, order: Order) -> Result<(), MicroError> {
    if self.tick_size > 0 && order.price % self.tick_size != 0 {
        return Err(MicroError::InvalidOrder(format!(
            "price {} is not a multiple of tick size {}",
            order.price, self.tick_size
        )));
    }
    if order.quantity == 0 {
        return Err(MicroError::InvalidOrder("quantity must be positive".into()));
    }
    if self.index.contains_key(&order.id) {
        return Err(MicroError::InvalidOrder(format!(
            "duplicate order id {}", order.id
        )));
    }
    self.index.insert(order.id, (order.side, order.price));
    let book = match order.side {
        Side::Bid => &mut self.bids,
        Side::Ask => &mut self.asks,
    };
    book.entry(order.price).or_insert_with(Vec::new).push(order);
    Ok(())
}
```

**Listing 13.2:** Adding a limit order

### 13.3.3. Market Orders and Price-Time Priority

A market order has no price: it takes liquidity from the opposite side until filled or until that side is empty. A market buy walks the asks in ascending price order; a market sell walks the bids in descending price order. At each price level, fills happen in FIFO order — the front of the vector is consumed first, exactly matching price-time priority.

```rust
pub fn market_order(&mut self, side: Side, quantity: u64) -> Vec<Fill> {
    let mut fills = Vec::new();
    let mut remaining = quantity;
    let prices: Vec<u64> = match side {
        Side::Bid => self.asks.keys().copied().collect(),         // buy -> asks ascending
        Side::Ask => self.bids.keys().rev().copied().collect(),   // sell -> bids descending
    };
    for price in prices {
        if remaining == 0 { break; }
        let book = match side {
            Side::Bid => &mut self.asks,
            Side::Ask => &mut self.bids,
        };
        if let Some(orders) = book.get_mut(&price) {
            while remaining > 0 && !orders.is_empty() {
                let front = &mut orders[0];
                let fill_qty = remaining.min(front.quantity);
                let maker_id = front.id;
                fills.push(Fill { price, quantity: fill_qty, maker_order_id: maker_id });
                remaining -= fill_qty;
                front.quantity -= fill_qty;
                if front.quantity == 0 {
                    let removed = orders.remove(0);
                    self.index.remove(&removed.id);
                }
            }
            if orders.is_empty() { book.remove(&price); }
        }
    }
    fills
}
```

**Listing 13.3:** Market-order execution walks the opposite side

### 13.3.4. Top-of-Book Accessors

The book exposes `best_bid()`, `best_ask()`, `mid_price()` and `spread()`. For a `BTreeMap`, the best bid is `iter().next_back()` (highest key) and

the best ask is `iter().next()` (lowest key). Depth at the top $L$ levels is the first $L$ keys from each side, aggregated.

## 13.4. Order Flow Imbalance

**OFI sign convention**

Positive OFI = buy-side pressure (bids growing relative to asks). Negative OFI = sell-side pressure. The signal predicts short-horizon returns: a rising bid queue means buyers are more eager than sellers, and the mid price tends to drift up over the next few seconds to minutes.

Cont, Kukanov and Stoikov (2014) introduced OFI as a signed measure of buying versus selling pressure at the top of the book. Given a sequence of snapshots, the OFI at transition $t$ is

$$\text{OFI}_t = \Delta q_t^{bid} - \Delta q_t^{ask},$$

where $\Delta q_t^{bid}$ is the change in available quantity at the best bid between $t-1$ and $t$, and similarly for the ask. Positive OFI indicates buy-side pressure (bids growing relative to asks); negative OFI indicates sell-side pressure.

**Listing 13.4:** Order flow imbalance from book snapshots

```rust
pub fn order_flow_imbalance(snapshots: &[BookSnapshot]) -> Vec<f64> {
    if snapshots.len() < 2 { return Vec::new(); }
    let mut result = Vec::with_capacity(snapshots.len() - 1);
    for i in 1..snapshots.len() {
        let prev_bid = snapshots[i - 1].best_bid.as_ref().map(|l| l.quantity as f64).
     unwrap_or(0.0);
        let prev_ask = snapshots[i - 1].best_ask.as_ref().map(|l| l.quantity as f64).
     unwrap_or(0.0);
        let curr_bid = snapshots[i].best_bid.as_ref().map(|l| l.quantity as f64).
     unwrap_or(0.0);
        let curr_ask = snapshots[i].best_ask.as_ref().map(|l| l.quantity as f64).
     unwrap_or(0.0);
        result.push((curr_bid - prev_bid) - (curr_ask - prev_ask));
    }
    result
}
```

### 13.4.1. VWAP and Trade Imbalance

Two complementary measures aggregate the trade tape. VWAP is the volume-weighted average fill price over a window:

$$\text{VWAP} = \frac{\sum_i p_i q_i}{\sum_i q_i}.$$

Trade imbalance normalises the signed volume into $[-1, 1]$:

$$\text{TI} = \frac{V^{buy} - V^{sell}}{V^{buy} + V^{sell}}.$$

A value near +1 means all trades were buy-initiated; near −1 all sell-initiated; near 0 balanced flow.

## 13.5. Market Impact Models

**Square-root law**

The most robust empirical regularity in market impact: across equity markets, futures, and FX, impact scales as $\sigma\sqrt{Q/V}$. The exponent is not 1 (linear) and not 2/3 (nonlinear propagation) — it is consistently close to 1/2. Doubling order size multiplies impact by $\sqrt{2} \approx 1.414$, *not* by 2.

The square-root impact law (Almgren and Chriss, 2000) states that the impact of an order is proportional to the volatility times the square root of the participation rate $Q/V$:

$$\text{Impact} = \sigma\sqrt{\frac{Q}{V}},$$

where $\sigma$ is the daily volatility, $Q$ the order size, and $V$ the average daily volume. The result is a fraction of price (e.g. 0.001 = 10 bps). This is an empirical regularity observed across equity markets with remarkable consistency.

A simpler linear model replaces the square root with a linear term:

$$\text{Impact}_{\text{lin}} = \eta \cdot \frac{Q}{V},$$

where $\eta$ is an impact coefficient calibrated to data.

```rust
pub fn sqrt_impact(volatility: f64, order_size: f64, daily_volume: f64) -> f64 {
    if daily_volume <= 0.0 { return 0.0; }
    let participation = order_size / daily_volume;
    volatility * participation.abs().sqrt()
}

pub fn linear_impact(eta: f64, order_size: f64, daily_volume: f64) -> f64 {
    if daily_volume <= 0.0 { return 0.0; }
    eta * (order_size / daily_volume)
}
```

**Listing 13.5:** Square-root and linear impact models

### 13.5.1. Execution Cost

**Half-spread = cost of immediacy**
A market taker buys at the ask and sells at the bid, paying the full spread. The maker on the other side earns half of it on average. So the taker's spread cost is $s/2$, not $s$: the other half is the maker's compensation for providing liquidity.

The total cost of executing a market order combines the half-spread (paid by the aggressor) with the square-root impact:

$$c_{\text{total}} = \underbrace{\frac{s}{2}}_{\text{half-spread}} + \underbrace{\sigma\sqrt{\frac{Q}{V}}}_{\text{impact}}.$$

Both terms are expressed as fractions of the mid price. Multiplying by 10,000 converts to basis points.

```rust
pub fn execution_cost(
    order_size: f64,
    daily_volume: f64,
    volatility: f64,
    spread: f64,
) -> ExecutionCost {
    let spread_cost = spread / 2.0;
    let impact_cost = sqrt_impact(volatility, order_size, daily_volume);
    ExecutionCost {
        spread_cost,
        impact_cost,
        total_cost: spread_cost + impact_cost,
    }
}
```

**Listing 13.6:** Execution cost combines spread and impact

## 13.6. Numerical Demonstration

We exercise the crate on a synthetic book and a 5-snapshot OFI series (`cargo run -p quant-microstructure -example lob_demo` and `-example ofi_demo`).

**Limit order book demo.** The book is seeded with three ask levels (30, 50, 20 shares at 101, 102, 103) and two bid levels (40, 60 at 99, 98). A market buy of 75 shares sweeps the best ask fully (30@101) and bites 45 out of the second level (45@102). The post-trade best ask is 102 with 5 shares remaining; the average fill price is 101.60.

**OFI demo.** The five snapshots show bid quantity growing from 50 to 75 and then shrinking to 55, while ask quantity shrinks from 40 to 25 and then grows to 45. The OFI series is $[+15, +25, -15, -25]$, with zero cumulative OFI at the end — symmetric buy-then-sell pressure.

**Execution cost.** For a 2,000-share order against a \$1M daily volume, $\sigma = 2\%$, spread = 2 bps:

| Component | Cost (bps) |
|---|---|
| Half-spread | 1.00 |
| Square-root impact ($0.02\sqrt{0.002}$) | 8.94 |
| Total | 9.94 |

The impact dominates the spread by a factor of nearly 9× at this participation rate. For a 200-share order the impact falls to 2.83 bps (a factor of $\sqrt{10}$ lower), and the spread becomes the larger cost — the square-root law is what makes small orders cheap and large orders expensive.

## 13.7. Complexity and Design Notes

**Why BTreeMap?** A `BTreeMap<u64, Vec<Order>>` gives us ordered price levels with $O(\log L)$ insert, delete and best-price lookup. A `HashMap` would not preserve order, and a sorted `Vec` would shift data on every insert. The price is the key, so the book never has to sort: ordering is structural.

**Why integer prices?** Floating-point comparison is non-associative and rounding-dependent. Limit order books must compare prices exactly: an order at 100.10 either matches the tick grid or it does not. Representing prices as integer ticks makes that comparison trivial and unambiguous.

**Why hand-rolled?** Production trading systems use heavily optimised LOB implementations (e.g. `lmax-disruptor` patterns, lock-free queues). This crate is a teaching artefact: every line is short enough to read in one sitting, and the price-time priority rule is visible in the code rather than hidden behind a library API.

## 13.8. Exercises

1. **Order book builder.** Write a function that takes a vector of `(Side, price, qty, ts)` tuples and returns a populated `OrderBook`. Verify the invariants `best_bid()`, `best_ask()`, `total_volume()` on the result.
2. **Liquidity sweep.** Construct a book with 5 ask levels totalling 1,000 shares. Execute a market buy of 750 shares and verify that the average fill price is less than or equal to the volume-weighted ask price of the swept levels.
3. **OFI and returns.** Generate 100 snapshots from a stochastic process where bid and ask quantities are driven by a latent "buy pressure" signal. Compute the OFI series and the mid-price return series; verify they are positively correlated.

4. **Impact scaling.** For a fixed $\sigma$ and $V$, compute `sqrt_impact()` for $Q \in \{100, 200, 500, 1000, 2000, 5000\}$ and plot impact versus $Q$ on a log-log scale. Confirm the slope is 1/2.
5. **Net frontier.** Take the Markowitz tangency portfolio from Chapter 11 and an assumed turnover of 50% per rebalance. Estimate the per-rebalance cost in bps using `execution_cost()` with $Q = 0.5 \times V_{\text{daily}}$ and subtract it from the expected return. Plot the net efficient frontier.

## 13.9. Key Takeaways

- The limit order book is a price-time priority queue with integer tick prices; a `BTreeMap` per side plus an id-index gives a compact, correct implementation.
- Market orders walk the opposite side in price order and fill level-by-level in FIFO order; partial fills are the rule, not the exception.
- OFI = $\Delta q^{bid} - \Delta q^{ask}$ is a signed top-of -book pressure signal; trade imbalance normalises signed tape volume into $[-1, 1]$.
- The square-root law $\sigma\sqrt{Q/V}$ is the most robust empirical regularity in market impact: doubling size multiplies impact by $\sqrt{2}$, not by 2.
- Total execution cost = half-spread + impact. For small orders the spread dominates; for large orders the impact dominates.

# 14. AFML: From Research to Production

*Chapter content pending — Phase 14 implementation.*

# Appendices

# A. Mathematical Notation

*Appendix content pending.*

# Rust Patterns for Finance

# B.

*Appendix content pending.*

# C. Dataset Sources

*Appendix content pending.*